Verifiable Computation for Approximate Homomorphic Encryption Schemes

Ignacio Cascudo[1], Anamaria Costache[2], Daniele Cozzo[1], Dario Fiore[1], Antonio Guimarães[1], and Eduardo Soria-Vazquez[3]

[1] IMDEA Software Institute, Spain.
[2] NTNU, Norway.
[3] Technology Innovation Institute, UAE.
firstname.lastname@imdea.org
firstname.lastname@ntnu.no
firstname.lastname@tii.ae

Abstract. We address the problem of proving the validity of computation on ciphertexts of homomorphic encryption (HE) schemes, a feature that enables outsourcing of data and computation while ensuring both data privacy and integrity. We propose a new solution that handles computations in RingLWE-based schemes, particularly the CKKS scheme for approximate arithmetic. Our approach efficiently handles ciphertext arithmetic in the polynomial ring R_q without emulation overhead and manages ciphertexts maintenance operations, such as modulus switching, key switching, and rescaling, with small cost. Our main result is a succinct argument that efficiently handles arithmetic computations and range checks over the ring R_q. To build this argument system, we construct new polynomial interactive oracle proofs (PIOPs) and multilinear polynomial commitments supporting polynomials over R_q, unlike prior work which focused on finite fields. We validate the concrete complexity of our approach through implementation and experimentation. Compared to the current state-of-the-art on verifiable HE for RNS schemes, we present similar performance for small circuits while being able to efficiently scale to larger ones, which was a major challenge for previous constructions as it requires verifying procedures such as relinearization.

1 Introduction

Homomorphic Encryption (HE) is a cryptographic primitive that allows performing computations directly on encrypted data without requiring decryption. This powerful capability enables data privacy in scenarios where sensitive data must be processed by untrusted third-party servers for storage and computation. HE has undergone significant advances, in terms of assumptions and efficiency [BV11, BGV12, FV12, Bra12, GSW13, CKKS17, CGGI16], that made HE more feasible for a broader range of applications.

Verifiable HE Due to the inherent malleability of homomorphically encrypted ciphertexts, HE cannot ensure the integrity of the computation. This is a crucial

limitation in the context of outsourcing computations, where both the correctness of the computation and the privacy of the data must be guaranteed. To tackle this problem, a line of research investigates how to add integrity properties to HE—a problem that lies at the intersection of two fundamental areas of research in cryptography: HE and Verifiable Computation (VC) [GGP10]. The latter (which includes the broader area of succinct zero-knowledge arguments, aka SNARKs) indeed deals with proving the correctness of computation outputs in a way that can be efficiently checked. HE and VC have each undergone tremendous efficiency advances that brought them to being used in practice. It is therefore natural to ask whether one can combine them to obtain a seemingly practical solution that addresses *both* privacy and integrity.

State-of-the-art The above combination is the core idea of many works in the state of the art, which we divide into two primary approaches: *proving plaintext computations under HE* and *proving ciphertext computations*. Both approaches have their merits and limitations (see Table 1). We are only interested in works where the verifier does sublinear work in the circuit size.

Proving plaintext computations. This approach, denoted HE-IOPs in [ACGS23], consists in generating an encrypted proof by executing a proof system on the plaintexts under HE. Although a naive application of this idea would lead to an expensive use of HE, recent works [GGW24, ACGS23, GBK+24] showed how to make it practical by executing under HE only the information-theoretic component of the SNARK, namely an interactive oracle proof (IOP) based on the FRI protocol [BBHR18]. The advantages of this approach are efficiency (especially compared to the very expensive solutions based on proving ciphertext computations – see below) and the support of "full" HE computations, including noise maintenance operations such as bootstrapping. On the other hand, HE-IOPs have two inherent limitations: they cannot support HE schemes for arithmetic of approximate numbers, like CKKS [CKKS17] (because the plaintext space of CKKS is not suitable to instantiate an IOP), and they ensure data privacy in a weak adversarial model where the verifier must keep secret whether verification passes. The reason is that in HE-IOPs verifying a proof requires decryption, and this can be exploited to extract a bit of information on the plaintext, defeating IND-CPA security. This security model is quite restrictive in practice, as in real life the verifier's acceptance bit may leak for multiple reasons (error messages, re-computation, etc.). In contrast, a private VC should guarantee security in the presence of verification oracles [FGP14].

Proving ciphertext computations. This approach, formalized in [FGP14], consists in using a verifiable computation scheme (e.g., a SNARK) to prove the correctness of the homomorphic computation on the HE ciphertexts. This has two main benefits. First, it ensures privacy in the strong adversarial model of verification oracles. Second, it does not impose any inherent restriction on the HE schemes that it supports (i.e. even [CKKS17] can be used). The main drawback of this approach is the expensive cost for the prover. This stems from two fundamental challenges involved in verifying operations on HE ciphertexts using a SNARK:

Scheme	Verif	Ctxt Arithm.	Ctxt Maint.	Bootstrapping	HE schemes
Generic SNARK	pub	○	○	○	any
[BCFK21]	pub	●	—	—	BV
[GNS23]	priv	●	○	○	any
HE-IOPs	priv, no VO	●	●	●	exact
Our work	pub	●	◑	○	any

Table 1: State of the art of verifiable computation for RingLWE-based HE. Notation: ○=emulated, ◑=efficiently emulated, ●=native support

1. HE ciphertexts are typically elements of the ring $R_q = \mathbb{Z}_q[X]/(X^N + 1)$, whereas SNARKs typically work best on computations over large finite fields.
2. Virtually all HE schemes require so-called ciphertexts maintenance operations (e.g., modulus-switch, key-switch, and rescaling), which involve non-algebraic operations such as real division and rounding.

General-purpose SNARKs can prove ciphertext arithmetic and maintenance operations by emulating them. This is unfortunately very costly. Knabenhans, Viand, and Hithnawi [KVH24, VKH23] recently provided an evaluation of these costs showing that, after various optimizations, the cost of proving one multiplication translates into ≈ 3 billion R1CS constraints.

Two works in the state of the art [BCFK21, GNS23] address the first challenge via proof systems specialized for R_q. However, [BCFK21] supports purely algebraic R_q operations (i.e., no maintenance) and thus applies only to a simple RingLWE-based HE [BV11] for constant-depth computations. Similarly, the Rinocchio SNARK of [GNS23] captures R1CS constraints over R_q, which cover well the arithmetic part of the HE computation but need expensive emulations based on bit decomposition to handle ciphertexts maintenance (challenge 2). Rinocchio has a few more limitations: it relies on new non-falsifiable assumptions, verification requires a secret key, and it needs a trusted setup that builds a CRS of size (at least) $2N(\log(q) + \lambda) \cdot S$ bits for S R1CS constraints. This is hardly scalable as $2N(\log(q) + \lambda)$ is $\approx 2^{23}$ even for small instances of CKKS, when proving non-algebraic maintenance operations, S will typically be $\geq N \cdot |C|$, where $|C|$ is the circuit size and $N \approx 2^{14}$. This makes the CRS size quadratically depend on N, which can quickly get to the order of many gigabytes of memory.

Other related work. A third category of results provide verifiable computation for applications where verification efficiency is not crucial [CKP+24, ABPS24]. These works require the verifier to run linearly in the witness size. Hence, their solutions do not scale for scenarios of (private and verifiable) outsourced computation, where the client would only be interested to outsource if it does sublinear work on the circuit size. Our results, on the other hand, provide a VC scheme for general computation where the verifier is succinct.

Our Contributions We propose a novel solution for verifiable homomorphic encryption, focusing on the second approach—proving ciphertext computations. To present our techniques we focus on the CKKS scheme because it offers unique advantages in supporting computations over approximate numbers, a feature that is increasingly important in many real-world applications. Also, CKKS is an interesting candidate as it cannot be captured by the recent HE-IOP approach and thus completely lacks efficient solutions. Nevertheless our proof techniques are flexible enough to be extended to other popular schemes like BGV and BFV.

Our approach allows us to overcome the challenges 1) and 2) mentioned above. First, it supports ciphertext arithmetic in R_q natively, i.e., without any emulation overhead. Second, it supports ciphertext maintenance operations such as modulus switching, key switching, and rescaling, with a small emulation overhead. And, unlike [GGW24, ACGS23, GBK+24], our scheme achieves security against verification oracle attacks. We design our succinct argument following the modular approach of combining polynomial interactive oracle proof (PIOPs) [BCS16, CHM+20, BFS20], an idealized information-theoretic protocol where the interactions between prover and verifier happens in form of polynomial evaluation oracles, and polynomial commitments [KZG10], that replace the oracles in the PIOP with actual commitments to the polynomials. In more detail, our new succinct argument stems from the combination of various contributions: a proof-friendly version of CKKS that allows good arithmetization, a PIOP consisting of a customized version of the GKR protocol and a novel look-up argument (both over R_q) and a polynomial commitment over R_q. For each of them, as detailed below, we provide efficient implementations.

Proof-friendly CKKS. We address the main incompatibilities between RNS-based HE schemes and proof systems:

- CKKS, similarly to many other HE schemes (e.g. BFV/BGV), relies on Residue Number System (RNS)-based arithmetic for efficiency, which in turn relies on rings that are typically unfriendly to the information-theoretic (IT) components of proof systems. Previous solutions relied on soundness amplification techniques (e.g. repetitions) which would introduce at least 2–4× slowdown. We implement CKKS directly over a proof-friendly ring, and show how to get efficient RNS arithmetic for it by exploiting techniques adapted from public key cryptography, such as incomplete NTTs [LS19].
- We redesign CKKS's mod switching operations to allow all proofs to be performed on a single ring, even if the scheme itself changes rings. This avoids problems such as connecting proofs over different arithmetic structures.
- We define and set up parameters to allow for core HE operations, such as key switchings and rescalings, to be performed in an approximate way such that we can relax the statements to be proven, which allows us to accelerate our proofs in up to 2 times.

PIOP for CKKS. We show how to take advantage of our proof-friendly CKKS to design an efficient characterization of homomorphic evaluations in terms of

a GKR arithmetic circuit and a collection of range checks. These range checks are crucial to efficiently express the non-algebraic maintenance operations, as they ensure that each element of a large vector $\boldsymbol{v} \in R_q^n$ is a polynomial with coefficients bounded by some $B < q$. Once we have this characterization, our main result is the construction of PIOPs for the two aforementioned relations, namely arithmetic circuits and range checks. While prior work shows PIOPs for such relations over *finite fields*, we construct them for the *ring* R_q. To build them we rely on the sum-check protocol [LFKN90] and its extension to rings with exceptional sets [CCKP19]. In particular, our contribution is twofold.

- For the case of arithmetic circuits, instead of using a naive PIOP version of GKR [GKR08] we take advantage of the structure induced by our arithmetization, which results in a circuit of constant depth (consisting of only 4 layers), independent of the size or depth of the HE circuit.
- For the case of range checks, we use the approach of table lookups [STW24], namely a succinct proof that convinces the verifier that a value belongs to a large table of values, for example this table may include all values within a certain range. However, since CKKS requires to check large bounds (e.g., B can be of 50 or 300 bits), we rely on the table decomposition technique of Lasso [STW24] which we extend to work for tables of R_q values.

Notice that neither our version of GKR nor our Lasso-style range proof are affected by the recent attacks on the Fiat-Shamir heuristic [KRS25]. For the former, notice that our GKR circuit has constant depth 4, whereas the the lowest-depth circuits that can be attacked by [KRS25] must have depth greater or equal than $d_{\texttt{comm}} + d_h + O(1)$ (for $\texttt{comm}$ a multi-linear polynomial commitment scheme computable by a depth $d_{\texttt{comm}}$ arithmetic circuit and h a hash function computable by a depth d_h arithmetic circuit). For the latter, the grand-product argument that we employ is computed using Quark's techniques [SL20]. In any case, for circuits with such canonical representation, the authors acknowledge their attack is not applicable [KRS25, Remark 3].

Multilinear Polynomial Commitment for R_q. The last piece to build our succinct argument is a suitable polynomial commitment (PC) for multilinear polynomials in $R_q[\boldsymbol{X}]$. We propose a new construction that fits this arithmetic. First, we show how to use the splitting of R_q into the product of (large enough) finite fields, so as to reduce the problem of designing a PC for $R_q[\boldsymbol{X}]$ into that of designing PCs for polynomials over finite fields. Second, we instantiate the latter using the flexible linear code-based construction from [BCG20, GLS$^+$23]. Brakedown [GLS$^+$23] instantiates this construction with an expander-graph based error correcting code that is agnostic to the finite field, a useful property in our case where the finite fields do not have a large number of roots of unity, preventing us from using PCs based on FRI [BBHR18]. On the other hand, instantiating this construction with Reed-Solomon codes (which leads to a polynomial commitment implicit in Ligero [AHIV17]) would provide smaller proof size and verifier time, and it is preferable as long as Reed-Solomon encoding can be performed fast. Such fast performance depends on whether one has a large enough set of roots of unity

in order to enable NTTs. Our fields have a "moderately large" set of roots of unity which, depending on the size of the polynomials, may not be enough to directly use Reed-Solomon codes. We find a balance in this situation and propose a new instantiation of this PC that uses a "piecewise" Reed-Solomon encoding, resulting in better performance in our setting, as confirmed by our experiments.

Implementation and Evaluation. We implement[4] and benchmark our proof-friendly version of CKKS and all the main building blocks required for implementing our construction. We show that our proof-friendly CKKS implementation only introduces an overhead of up to 20% for ciphertext multiplications over a "regular" instantiation of the scheme, being still faster than commonly used libraries such as HELib [HS14]. For all other components of our construction, we show that, by themselves, they enable concretely practical performance, and we estimate the costs of employing them on small applications. Compared to previous literature, our results for small (depth-1) circuits indicate similar performance levels as [VKH23][5], which is the state of the art on concrete performance for verifiable RNS HE schemes. However, contrary to [VKH23], we verify full-featured RNS-based leveled HE schemes, including key switching and relinearization operations, which enables our solution to scale to larger circuits, whereas performance in [VKH23]'s approach would deteriorate exponentially with the circuit depth.

On bootstrapping. In this work, we consider CKKS in "levelled" mode (*i.e.*, rescaling is the only noise-management technique), and do not focus on bootstrapping, as we see it as a natural extension of the techniques we provide. While the specific details may vary depending on which version of bootstrapping is considered [CHK$^+$18, BCC$^+$22, BMTH21, CCS19, HK20], most of them consist of similar main steps. Within these, rotations and modulus raising are the only building blocks that we do not explicitly describe here. Translating them into the operations we cover is nonetheless straightforward. Rotations are homomorphic evaluations of Galois automorphisms, which only require coefficient permutations and key switching (both operations covered by our framework). Modulus raising, in turn, can be verified analogously to the many other modulus switching operations we perform, with the only particularity that the cost of proving it would be proportional to the largest modulus it operates on.

Outline In Sec. 2 we provide preliminary notions needed for the rest of the paper. In Sec. 3 we describe our proof-friendly version of CKKS. In Sec. 4 we describe the arithmetization of our CKKS scheme, which reduces the problem of designing a PIOP for the homomorphic computation to a PIOP for an arithmetic circuit satisfiability relation and a PIOP for range checks. We instantiate the former in Sec. 4.2 with a GKR-style proof and the latter in Sec. 4.3 with a look-up argument for R_q. In Sec. 5 we present our improved version of the Brakedown polynomial commitment that allows compiling the previous PIOP

[4] The source code is available at `https://github.com/vfhe/proof-friendly-CKKS`.

[5] We can't have concrete comparisons for circuits of larger depth since [VKH23] is limited to depth 1 circuits

into a SNARK. Finally in Sec. 6 we give benchmarks for all the mentioned building blocks.

Changelog Previous versions of this paper presented a subprotocol for range checks that, as pointed out in [WZZ+26], had a soundness flaw. In this version, we changed this protocol (Section 4.3) in order to fix this flaw and updated its theoretical efficiency analysis. See Remarks 4.12 and 6.1 for details.

2 Preliminaries

Notation. For a positive integer n, $[n]$ denotes the set $\{0, \ldots, n-1\}$. In what follows R denotes a finite commutative ring with unity. Given $a \in R$, $[a]_b$ denotes reducing a modulo b. If $a \in R$ is an element of a polynomial ring, $[a]_b$ denotes the coefficient-wise reduction. We denote vectors in bold. For a vector $\mathbf{v} \in R^n$, we denote by $\mathbf{v}[i]$ its i-th coefficient. We write $[\mathbf{v}]_p$ to denote modular reduction of each component of $\mathbf{v}$.

We write $a \xleftarrow{\$} R$ to mean that a is sampled uniformly at random from the ring R and $e \leftarrow \chi$ to mean that e is sampled according to the distribution χ.

An *indexed relation* $\mathcal{R}$ is a set of tuples $(\mathtt{i}, \mathtt{x}; \mathtt{w})$ where $\mathtt{i}$ is the index, $\mathtt{x}$ the statement and $\mathtt{w}$ the witness. The language $\mathcal{L}(\mathcal{R})$ associated to a relation $\mathcal{R}$ is the set of pairs $(\mathtt{i}, \mathtt{x})$ for which there exists a witness $\mathtt{w}$ such that $(\mathtt{i}, \mathtt{x}; \mathtt{w}) \in \mathcal{R}$.

2.1 Background on rings

We denote by $R[X_1, \ldots, X_\ell]$ the set of polynomials of ℓ variables and coefficients in R and by $R^{(\leq d)}[X_1, \ldots, X_\ell]$ the subset of polynomials of individual degree $\leq d$.

Definition 2.1 (Exceptional sets). *An* exceptional set *is a subset $S \subset R$ with the property that $a - b$ is an invertible element of R for each pair of distinct elements $a, b \in S$.*

Lemma 2.2 (Generalized Schwartz-Zippel lemma). *Let $f \in R[X_1, \ldots, X_\ell]$ be a non-zero polynomial of total degree d. Then for an exceptional set $S \subset R$*

$$\Pr_{(r_1,\ldots,r_\ell) \leftarrow S^\ell} [f(r_1, \ldots, r_\ell) = 0] \leq \frac{d}{|S|}$$

Multilinear extensions. Given a function $f : \{0,1\}^\ell \to R$, its *multilinear extension* (MLE) is the (unique) multilinear polynomial $\tilde{f} : R^\ell \to R$ such that $f(\boldsymbol{b}) = \tilde{f}(\boldsymbol{b})$ for all $\boldsymbol{b} \in \{0,1\}^\ell$. The MLE of f is the following polynomial

$$\tilde{f}(X_1, \ldots, X_\ell) = \sum_{\boldsymbol{b} \in \{0,1\}^\ell} f(\boldsymbol{b}) \cdot \chi_{\boldsymbol{b}}(X_1, \ldots, X_\ell)$$

where $\chi_{\boldsymbol{b}}(X_1, \ldots, X_\ell) = \prod_{k=1}^{\ell} \chi_{b_k}(X_k)$, with $\chi_1(X) = X$ and $\chi_0(X) = 1 - X$. Analogously, the MLE of a vector $\boldsymbol{v} = (v_{0\cdots0}, \ldots, v_{1\cdots1}) \in R^{2^\ell}$ (conveniently indexed in binary) is the polynomial $\tilde{v}(X_1, \ldots, X_\ell) = \sum_{\boldsymbol{b}} v_{\boldsymbol{b}} \cdot \chi_{\boldsymbol{b}}(X_1, \ldots, X_\ell)$.

2.2 Polynomial IOPs for Rings

We recall the definition of polynomial interactive oracle proofs (PIOP), using the notion of (indexed) oracle relations of [CBBZ23].

Definition 2.3 (Oracle Relations). *An indexed oracle relation $\mathcal{R}$ is an indexed relation in which the index $\mathtt{i}$ and the statement $\mathtt{x}$ include pointers to oracle polynomials with coefficients in a ring R, and the polynomials are part of the witness $\mathtt{w}$. A pointer to a polynomial p is denoted by $[\![p]\!]$*

Definition 2.4 (Polynomial IOPs). *Let R be a ring and $\mathcal{R}$ be an indexed oracle relation for (multivariate) polynomials with coefficients in R. A polynomial IOP for $\mathcal{R}$ is an interactive protocol between a prover $\mathcal{P}$ and a verifier $\mathcal{V}$, compactly denoted by $\langle \mathcal{P}(\mathtt{i},\mathtt{x},\mathtt{w}), \mathcal{V}(\mathtt{i},\mathtt{x})\rangle$, such that:*

- *In each round the prover $\mathcal{P}(\mathtt{i},\mathtt{x},\mathtt{w})$ sends oracle polynomials and/or ring elements, and the verifier $\mathcal{V}(\mathtt{i},\mathtt{x})$ sends random challenges. Each oracle $[\![p]\!]$ specifies the number of variables ℓ and the degree in each variable.*
- *The verifier can query all the oracles, the ones sent by the prover and the ones included in $(\mathtt{i},\mathtt{x})$, at arbitrary points $\boldsymbol{r} \in R^\ell$. At the end of the execution, $\mathcal{V}$ accepts or rejects.*
- *Correctness: For any honest execution of $\langle \mathcal{P}(\mathtt{i},\mathtt{x},\mathtt{w}), \mathcal{V}(\mathtt{i},\mathtt{x})\rangle$ on $(\mathtt{i},\mathtt{x};\mathtt{w}) \in \mathcal{R}$, the verifier accepts.*
- *δ-Soundness: A PIOP is δ-sound if for any unbounded adversary Adv and any $(\mathtt{i},\mathtt{x}) \notin \mathcal{L}(\mathcal{R})$, $\Pr\left[\langle \mathsf{Adv}(\mathtt{i},\mathtt{x}), \mathcal{V}(\mathtt{i},\mathtt{x})\rangle = 1\right] \leq \delta$.*
- *δ-Knowledge Soundness: A PIOP is δ-knowledge-sound if there exists a PPT oracle machine Ext (which can query all the oracle polynomials at arbitrary points) such that for any adversary Adv and any $(\mathtt{i},\mathtt{x})$*

$$\Pr\left[b = 1 \ \wedge \ (\mathtt{i},\mathtt{x};\mathtt{w}) \notin \mathcal{R} \quad : \begin{array}{l} \langle \mathsf{Adv}(\mathtt{i},\mathtt{x}), \mathcal{V}(\mathtt{i},\mathtt{x})\rangle = b \\ \mathtt{w} \leftarrow \mathsf{Ext}^{\mathsf{Adv}}(\mathtt{i},\mathtt{x}) \end{array} \right] \leq \delta$$

Remark 2.5. As shown in [CBBZ23, Lemma 2.3], any δ-sound PIOP for an oracle relation $\mathcal{R}$ is δ-knowledge sound, if the witnesses of the relation $\mathcal{R}$ consist only of polynomials provided as oracles in the statement.

We measure the efficiency of a PIOP according to the following metrics: the running times $\mathsf{T}_\mathcal{P}$ and $\mathsf{T}_\mathcal{V}$ of prover and verifier, the number of rounds r, the query complexity q (i.e., number of queries made by the verifier), the number of oracles sent by the prover o, and the size s of the proof oracles (i.e., the sum of length of all the polynomials sent by the prover).

Virtual oracles Following [CBBZ23], we use the notion of *virtual oracles*. For an arithmetic function g, an oracle to $g([\![p_1]\!],\ldots,[\![p_k]\!])$ consists of the list of oracles $\{[\![p_1]\!],\ldots,[\![p_k]\!]\}$ and the description of g. A query of a virtual oracle $g([\![p_1]\!],\ldots,[\![p_k]\!])$ at any point $\boldsymbol{x}$ can be realized by querying each oracle $[\![p_j]\!]$ on some $\boldsymbol{x}_j$ and then, given $\{y_j = p_j(\boldsymbol{x}_j)\}$, computing $g(y_1,\ldots,y_k)$.

Sum-check protocol Let R be a ring, $p : R^\ell \to R$ be an ℓ-variate polynomial of degree at most d in each variable. The sum-check (oracle) relation $\mathcal{R}_{sum}$ is the set of tuples $(y; p)$ (resp. $((y, [\![p]\!]); p)$) such that $y = \sum_{\boldsymbol{i} \in \{0,1\}^\ell} p(\boldsymbol{i})$. The sum-check protocol [LFKN90], originally introduced for finite fields only, reduces for the verifier $\mathcal{V}$ the problem of computing y to that of evaluating p at a point $(r_1, \ldots, r_\ell)$ sampled from an exceptional set. Below we describe Π_{sum} as either an interactive proof or a PIOP. In our protocols, $\mathcal{V}$ might not be given (oracle access to) $p(r_1, \ldots, r_\ell)$ directly, but rather enough information to compute that on its own. For example, for $p = fg + h$, where $\mathcal{V}$ knows f, it might be given (oracle access to) $g(r_1, \ldots, r_\ell)$ and $h(r_1, \ldots, r_\ell)$.

The sum-check protocol/PIOP Π_{sum} for $\mathcal{R}_{sum}$

for $j = 1$ to ℓ

  $\mathcal{P}$ computes and sends $g_j(X) \leftarrow \sum_{\boldsymbol{i} \in \{0,1\}^{\ell - j}} p(r_1, \ldots, r_{j-1}, X, \boldsymbol{i})$.

  $\mathcal{V}$ sends $r_j \leftarrow\!\!\$\ S$.

$\mathcal{P}$ sends $p(r_1, \ldots, r_\ell)$. (In the PIOP version: $\mathcal{V}$ queries $[\![p]\!]$ on $(r_1, \ldots, r_\ell)$).

$\mathcal{V}$ accepts if the following checks are satisfied:

  $\forall j \in [\ell] : g_{j-1}(r_{j-1}) = g_j(0) + g_j(1)$, where $g_0(r_0) = y$

  $g_\ell(r_\ell) = p(r_1, \ldots, r_\ell)$

Theorem 2.6. *Let $S \subseteq R$ be an exceptional set. The PIOP for $\mathcal{R}_{sum}$ described above has perfect completeness and δ-knowledge-soundness with $\delta = d\ell/|S|$.*

Proof. It follows from the soundness analysis in [CCKP19] and Remark 2.5. □

Polynomial commitments and compilation to SNARKs. In order to use PIOPs in the real world one needs a polynomial commitment scheme. This is cryptographic primitive that allows one to commit to a polynomial p and to later convince a verifier that $y = p(a)$ for the committed p. Compiling a PIOP for relation $\mathcal{R}$ into a public-coin interactive argument of knowledge for the same relation one consists of two steps: replace every oracle sent by the prover with a commitment to the underlying polynomial, and replace every verifier's query to an oracle $[\![p]\!]$ at a point $\boldsymbol{a}$ with sending the actual value $p(\boldsymbol{a})$ and proving its correctness w.r.t. the commitment of p by using the $\langle \mathsf{ProveEval}, \mathsf{VerEval} \rangle$ protocol. To this standard compilation technique we add the observation that if we start from a PIOP for an oracle relation $\mathcal{R}$ then we can easily build a (preprocessing) commit-and-prove argument. For completeness we recall in appendix D the definitions of polynomial commitments and SNARKs, and we state the compilation theorem along with its efficiency results.

3 Proof-friendly CKKS

Practical implementations of homomorphic encryption schemes such as CKKS [CKKS17] and BGV [BGV12] commonly rely on reasonably specific arithmetic

structures for efficiency, most often based on Residue Number Systems (RNS) over polynomial rings that can be fully split and mapped to the product of word-sized prime fields. Conversely, most proof systems techniques have different arithmetic requirements, typically relying on larger (than word size) fields, for soundness reasons. Aligning these constraints (without forgoing HE performance or VC soundness) is an intricate task and often introduces substantial performance overhead over entire constructions. Other challenges arise from operations such as rescaling and mod switching procedures that change the computation ring, which is not natively supported by most proof systems. In this work, our first step is to define a proof-friendly version of full-RNS CKKS [CHK+19] that avoids these issues in a close to optimal way. Specifically, we instantiate CKKS with the following modifications, which are detailed later in this section.

1. The RNS arithmetic is implemented over rings that split into extension fields of configurable degree d. This enables us to fulfill the requirements for soundness but prevents the use of RNS with typical NTT-based arithmetic for fully splitting rings. Instead, we implement RNS using *incomplete NTTs* [LS19], which require asymptotically more expensive multiplications, but introduce minimal overall overhead in practice. In fact, they have been shown to even provide performance improvements for some use cases in the public-key cryptography literature [TS24, AHKS22].
2. The scheme is entirely defined over a single ring R_{q_0}. Any operations that would map to a ring different from R_{q_0} (*e.g.*, modulus switching) have their results embedded back into R_{q_0}. This embedding is defined with respect to ideals of R_{q_0} that enables reducing the number of RNS components throughout the computation, as usual in RNS-based HE implementations. We formalize this change via a slightly different CRT map (see Section 3.2).
3. Parameters are set up to allow for approximate rescalings and base decompositions, which are significantly faster to prove and verify.

3.1 Setting

Let N be a power of two and $q_0 = \prod_{i=0}^{L} p_i$ for some integer L, such that each p_i is a prime in the format $2aN/d + 1$ for some odd integer a and suitable power-of-two value d. Then, for each p_i, the cyclotomic polynomial $X^N + 1$ factors in $\mathbb{Z}_{p_i}[X]$ into $k = N/d$ irreducible factors

$$X^N + 1 = \prod_{j=0}^{k-1} (X^d - \zeta^{(2j+1)}),$$

where $\zeta \in \mathbb{Z}_{p_i}$ is a $2N/d$-th primitive root of unity. By the Chinese Remainder Theorem (CRT), the ring $R_{q_0} := \mathbb{Z}_{q_0}[X]/(X^N + 1)$ splits as $R_{q_0} = \prod_{i=0}^{L} R_{p_i} = \prod_{i=0}^{L} \left(\prod_{j=0}^{k-1} R_{i,j} \right)$, with each $R_{i,j} = \mathbb{Z}_{p_i}[X]/(X^d - \zeta^{(2j+1)})$ being a field of size p_i^d. By choosing an appropriate value for d, we have exceptional sets with enough elements for soundness security. Notice that the size of the largest exceptional

set in R_{q_0} is given by the size of its smallest field component. For example, $d = 4$ would be a safe choice for 32-bit primes. We refer to [LS19, TS24, AHKS22] and Section 6 for details on efficient arithmetic over these rings.

3.2 CRT

For all $0 \leq j \leq L$ we define $q_j = \prod_{i=0}^{L-j} p_i$ and in what follows we use l to compactly denote $l = L - j$. Similarly, we define $R \coloneqq \mathbb{Z}[X]/(X^N + 1)$ and $R_{q_j} = \mathbb{Z}_{q_j}[X]/(X^N + 1)$ for all $0 \leq j \leq L$. Let $\omega_j = (p_0, p_1, \ldots, p_l)$ be a CRT base for q_j and given $a \in R_{q_0}$, we define the inverse CRT map as follows:

$$\mathsf{CRT}^{-1}_{\omega_j}(a) \coloneqq \left([a]_{p_0}, [a]_{p_1}, \ldots, [a]_{p_l} \right) \in R^{l+1}_{q_0}. \tag{1}$$

Note that, in general, the inverse CRT for ω_j would be defined as a map $R_{q_0} \to R_{p_0} \times \ldots \times R_{p_l}$. We slightly modify this, by adding an embedding from $R_{p_0} \times \ldots \times R_{p_l}$ to $R^{l+1}_{q_0}$. More precisely, our mapping looks like:

$$\begin{aligned} R_{q_0} &\to R_{p_0} \times \ldots \times R_{p_l} && \hookrightarrow R^{l+1}_{q_0} \\ a \mapsto \boldsymbol{a} &\coloneqq \left([a]_{p_0}, [a]_{p_1}, \ldots, [a]_{p_l} \right) && \hookrightarrow \left(\boldsymbol{a'}^{\intercal}_0, \ldots, \boldsymbol{a'}^{\intercal}_l \right), \end{aligned}$$

where $\boldsymbol{a'}^{\intercal}_i = \left(\left[[a]_{p_i} \right]_{p_0}, \left[[a]_{p_i} \right]_{p_1}, \ldots, \left[[a]_{p_i} \right]_{p_l}, \underbrace{0 \ldots, 0}_{j \text{ times}} \right)$.

Let $Q_i = q_0/p_i$ and $\hat{Q}_i = \left[(q_0/p_i)^{-1} \right]_{p_i}$ be the usual constants for CRT recomposition, we define the generator of the ideal defining our embedding as $z_l \coloneqq \sum_{i=0}^{l} Q_i \hat{Q}_i = \mathsf{CRT}(\underbrace{1, \ldots, 1}_{l+1 \text{ times}}, 0, \ldots, 0) \in R_{q_0}$.

For each $0 \leq j \leq L$, the CRT recomposition vector for a given $a \in R_{q_0}$ is:

$$\mathsf{PW}_{\omega_l}(a) \coloneqq \left(\left[aQ_0\hat{Q}_0 \right]_{q_0}, \left[aQ_1\hat{Q}_1 \right]_{q_0}, \ldots, \left[aQ_l\hat{Q}_l \right]_{q_0} \right). \tag{2}$$

For any $a, b \in R_{q_0}$, for any level j, the following equivalence holds

$$a \cdot b \cdot z_l \equiv \langle \mathsf{PW}_{\omega_l}(a), \mathsf{CRT}^{-1}_{\omega_l}(b) \rangle \cdot z_l \pmod{q_0}. \tag{3}$$

This follows from a direct application of the CRT in the ideal defined by z_l.

3.3 CKKS

Let $q_0 > q_1 > \cdots > q_{D-1}$ be a chain of moduli for a circuit of depth D and $(\omega_0, \omega_1, \ldots, \omega_{D-1})$ their respective CRT bases, χ_{key} be the secret key distribution over R, and $\chi_{\text{err}}, \chi_{\text{enc}}$ be discrete Gaussian distributions over R_{q_0}. Below we present our version of the CKKS scheme. Algorithms SecretKeyGen, PublicKeyGen, Enc, Add, SAdd, and SMult are unchanged and thus omitted. For a full description of CKKS as presented in [CKKS17], as well as the algorithms mentioned above, see Appendix A.

$\mathsf{evk} \leftarrow \mathsf{KeySwitchGen}(s, s^2)$**:** For $\mathsf{sk} = (1, s)$, for $i = 0, \ldots, L$, sample $a_i \xleftarrow{\$} R_{q_0}$, sample $e_i \leftarrow \chi_{\mathrm{err}}$. Compute $b_i = -a_i \cdot s + e_i + \mathsf{PW}_{\omega_L}(s^2)[i] \pmod{q_0}$. Recall that $l = L - j$, for each level $j \in \{0, \ldots, D-1\}$, compute

$$\mathsf{evk} \coloneqq (\mathsf{evk}_{j,0}, \mathsf{evk}_{j,1}) \leftarrow ((z_l b_i)_{i=0,\ldots,l}, (z_l a_i)_{i=0,\ldots,l}) \in \left(R_{q_0}^2\right)^{l+1}.$$

Notice that all key switching keys are generated in R_{q_0} for all levels, but we "manually" change levels by moving them to the ideals defined by z_l. In practice, zeroed RNS components do not need to be processed, yielding similar performance as typical RNS implementations.

$\mu + e' \leftarrow \mathsf{Dec}(\mathsf{sk}, \mathsf{ct})$**:** Given a ciphertext $\mathsf{ct} \in R_{q_0}$ for some level j, parse $(c_0, c_1) = \mathsf{ct}$. Output the CRT recomposition of the first $l = L - j$ components of $\langle \mathsf{ct}, \mathsf{sk} \rangle$, namely $\mathsf{CRT}_{\omega_j}(\langle \mathsf{ct}, \mathsf{sk} \rangle)$.

$(d_0, d_1, d_2) \leftarrow \mathsf{PreMult}(\mathsf{ct}_0, \mathsf{ct}_1, j)$**:** For some level $j \in \{0, \ldots, D-1\}$, compute

$$(d_0, d_1, d_2) \coloneqq (\mathsf{ct}_0[0] \cdot \mathsf{ct}_1[0], \mathsf{ct}_0[0] \cdot \mathsf{ct}_1[1] + \mathsf{ct}_1[0] \cdot \mathsf{ct}_0[1], \mathsf{ct}_0[1] \cdot \mathsf{ct}_1[1]), \quad (4)$$

as an element of $R_{q_0}^3$. Then, we perform the key switching as follows.

$\mathsf{ct}' \leftarrow \mathsf{KeySwitch}(\mathsf{evk}, \mathsf{ct} = (d_0, d_1, d_2))$**:** output $\mathsf{ct}' = (c_0', c_1')$ where

$$c_i' \coloneqq d_i + \langle \mathsf{CRT}_{\omega_j}^{-1}(d_2), \mathsf{evk}_{j,i} \rangle \in R_{q_0}, \quad i = 0, 1. \quad (5)$$

$\mathsf{ct}'' \leftarrow \mathsf{Rescale}(\mathsf{ct}', j)$**:** Let $\mathsf{ct}' = (c_0', c_1')$, and $\mathsf{p}_l^{-1} = [q_{j+1}/q_j]_{q_{j+1}} \cdot z_l \in R_{q_0}$. We re-scale by computing $\mathsf{ct}'' \coloneqq (c_0'', c_1'') \in R_{q_0}^2$ as follows

$$c_i'' \coloneqq \left(c_i' - [c_i']_{p_l}\right) \mathsf{p}_l^{-1} \in R_{q_0}, \quad i = 0, 1. \quad (6)$$

3.4 Noise analysis

In principle, the noise analysis for our version of CKKS is the same as for usual instantiations of the scheme [CKKS17, CCH+24, KPP22, CHK+19]. However, to accelerate range proofs, we also allow for approximate versions of the rescaling and key switching procedures. The computation itself remains unchanged but, as we discuss in Section 4.1, we relax the proven statements as follows:

1. In the key switching: Instead of computing $\mathsf{CRT}_{\omega_j}^{-1}$, the prover could compute some other decomposition that still recomposes correctly with the factors of PW (as in Equation 3), but for which the decomposed values are bounded by some slightly larger constant M such that $\max(p_i) \leq M \leq 2\max(p_i)$.
2. In the rescaling: Instead of computing $[c_1']_{p_l}$, it computes $[c_1']_{p_l} \pm u \cdot p_l$, for some $u \in R_{q_0}$ such that $\|u\|_\infty \leq 1$.

These relaxations allow a malicious prover to introduce some additional noise to the results of these procedures. This noise, however, only increases the original noise of each procedure by a factor of at most 2. We detail this analysis and prove this bound in Appendix B.

4 PIOP for CKKS

We present a PIOP for proving a computation over CKKS ciphertexts. We begin by characterizing this as a combination of an arithmetic circuit and range checks. Then in Sections 4.2 and 4.3 we construct PIOPs for these two relations.

4.1 Relations for computations on CKKS ciphertexts

To arithmetize an HE computation, we think of it as being organized in multiplicative layers, where each layer consists of independent homomorphic multiplications. We denote by D the depth of the HE circuit, i.e., the number of multiplicative layers, and by W the width, i.e., the number of multiplication gates within the same layer. For ease of presentation we assume that all layers have the same width W, except for the output layer $l = D-1$, whose width is out, the number of ciphertexts output by the HE circuit. Also, we denote by in the number of input ciphertexts, the width of layer 0. Layer $l = 0, \ldots, D-1$ takes inputs $(\mathbf{c}^{(i)})_{i=0,\ldots,l}$, with $\mathbf{c}^{(i)} = (\mathbf{c}_0^{(i)}, \mathbf{c}_1^{(i)}) \in R_q^{2W}$, that are either the circuit's inputs $\mathbf{c}^{(0)}$ or outputs of previous layers, and it outputs $\mathbf{c}''^{(l+1)} = (\mathbf{c}_0''^{(l+1)}, \mathbf{c}_1''^{(l+1)}) \in R_q^{2W}$. For simplicity, we assume that the final circuit's outputs only come from the outputs of the last layer. Without loss of generality, we assume that the first part of the layer performs linear operations and then pre-multiplication.[6] Thus it can be described by vectors of quadratic polynomials $\mathbf{Q}_0^{(l)}, \mathbf{Q}_2^{(l)} : R_q^{lW+\mathsf{in}} \to R_q^W$ and $\mathbf{Q}_1^{(l)} : R_q^{2(lW+\mathsf{in})} \to R_q^W$ such that

$$(\mathbf{d}_0^{(l)}, \mathbf{d}_1^{(l)}, \mathbf{d}_2^{(l)}) = \left(\mathbf{Q}_0^{(l)}(\mathbf{c}_0^{(l)}, \ldots, \mathbf{c}_0^{(0)}), \mathbf{Q}_1^{(l)}(\mathbf{c}_0^{(l)}, \mathbf{c}_1^{(l)}, \ldots, \mathbf{c}_0^{(0)}, \mathbf{c}_1^{(0)}), \mathbf{Q}_2^{(l)}(\mathbf{c}_1^{(l)}, \ldots, \mathbf{c}_1^{(0)})\right) \tag{7}$$

Next is the key-switching, which takes the values $\mathbf{d}_0^{(l)}, \mathbf{d}_1^{(l)}, \mathbf{d}_2^{(l)}$ to compute

$$\mathbf{c}_b'^{(l)} = \mathbf{d}_b^{(l)} + \langle \mathsf{evk}_{l,b}, \mathsf{CRT}_{\omega_l}^{-1}(\mathbf{d}_2^{(l)}) \rangle \quad b = 0, 1, \tag{8}$$

where the vector $\mathsf{evk}_{l,b} \in R_q^{L+1-l}$ is constant to the circuit. Since we cannot prove eq. (8) algebraically due to the modular reductions within $\mathsf{CRT}_{\omega_l}^{-1}$, we use non-determinism and introduce values $\mathbf{w}_{\mathrm{ks},0}^{(l)}, \ldots, \mathbf{w}_{\mathrm{ks},L-l}^{(l)} \in R_q^W$ such that $\mathbf{c}_b'^{(l)} = \mathbf{d}_b^{(l)} + \sum_{i=0}^{L-l} \mathsf{evk}_{l,b}[i] \cdot \mathbf{w}_{\mathrm{ks},i}^{(l)}$, with $b = 0, 1$ or, more compactly, using eq. (7) we derived for $\mathbf{d}_0^{(l)}$ and $\mathbf{d}_1^{(l)}$, we can write $\mathbf{c}_b'^{(l)} = \mathbf{Q}_b^{(l)}((\cup_{b'=0}^{b} \mathbf{c}_{b'}^{(i)})_{i=0}^{l}) + \mathbf{L}_b^{(l)}((\mathbf{w}_{\mathrm{ks},i}^{(l)})_{i=0}^{L-l})$, $b = 0, 1$, for linear polynomials $\mathbf{L}_b^{(l)} : R_q^{W \times L-l} \to R_q^W$ that encode the inner product by $\mathsf{evk}_{l,b}$ for $b = 0, 1$. To ensure that the non-deterministic inputs $\mathbf{w}_{\mathrm{ks},0}^{(l)}, \ldots, \mathbf{w}_{\mathrm{ks},L-l}^{(l)}$ are indeed $\mathsf{CRT}_{\omega_l}^{-1}(\mathbf{d}_2^{(l)})$, we must ensure that their components are in the correct range and that they recompose to $\mathbf{d}_2^{(l)}$, i.e.

$$\left\| \mathbf{w}_{\mathrm{ks},i}^{(l)} \right\|_\infty < p_i, \ i \in [L + 1 - l]. \tag{9}$$

[6] We assume that we do not perform noise-maintenance operations after linear operations.

$$\mathbf{d}_2^{(l)} - \sum_{i=0}^{L-l} \mathsf{PW}_{\omega_l}(1)[i] \cdot \mathbf{w}_{\mathrm{ks},i}^{(l)} = 0, \tag{10}$$

where the vector $\mathsf{PW}_{\omega_l}(1)$ is constant to the circuit. Using the equations 7 for $\mathbf{d}_0^{(l)}, \mathbf{d}_1^{(l)}, \mathbf{d}_2^{(l)}$, then the system of equations 10 can be rewritten as:

$$\mathbf{Q}_2^{(l)}((\mathbf{c}_1^{(i)})_{i=0}^{l}) - \mathbf{L}_2^{(l)}(\mathbf{w}_{\mathrm{ks},0}^{(l)}, \ldots, \mathbf{w}_{\mathrm{ks},L-l}^{(l)}) = \mathbf{0} \tag{11}$$

for a linear polynomial $\mathbf{L}_2^{(l)} : R_q^{W \times L-l} \to R_q^W$ defined by the inner product by $\mathsf{PW}_{\omega_l}(1)$. The values $\mathbf{c}_0^{\prime(l)}, \mathbf{c}_1^{\prime(l)}$ are then inputs to the re-scaling procedure:

$$\mathbf{c}_b^{\prime\prime(l+1)} = (\mathbf{c}_b^{\prime(l)} - [\mathbf{c}_b^{\prime(l)}]_{p_{L-l}})\mathsf{p}_{L-l}^{-1}, \ b = 0, 1 \tag{12}$$

where p_{L-l}^{-1} is a constant in R_q. Again one proves eq. 12 using non-determinism. We introduce inputs $\mathbf{w}_{\mathrm{quot},0}^{(l+1)}, \mathbf{w}_{\mathrm{quot},1}^{(l+1)}, \mathbf{w}_{\mathrm{rmd},0}^{(l)}, \mathbf{w}_{\mathrm{rmd},1}^{(l)} \in R_q^W$ such that, for $b = 0, 1$

$$z_l \cdot \mathbf{w}_{\mathrm{quot},b}^{(l+1)} = \left(\mathbf{Q}_b^{(l)}((\cup_{b'=0}^{b'} \mathbf{c}_{b'}^{(i)})_{i=0}^{l}) + \mathbf{L}_b^{(l)}((\mathbf{w}_{\mathrm{ks},i}^{(l)})_{i \in [L+1-l]}) - \mathbf{w}_{\mathrm{rmd},b}^{(l)}\right) \mathsf{p}_{L-l}^{-1}, \tag{13}$$

with the conditions that

$$\left\|\mathbf{w}_{\mathrm{quot},0}^{(l+1)}\right\|_\infty, \left\|\mathbf{w}_{\mathrm{quot},1}^{(l+1)}\right\|_\infty < q/p_{L-l} \tag{14}$$

$$\left\|\mathbf{w}_{\mathrm{rmd},0}^{(l)}\right\|_\infty, \left\|\mathbf{w}_{\mathrm{rmd},1}^{(l)}\right\|_\infty < p_{L-l}. \tag{15}$$

The values $\mathbf{w}_{\mathrm{quot},0}^{(l+1)}, \mathbf{w}_{\mathrm{quot},1}^{(l+1)}$ are the outputs of the layer.

To summarize, the correct computation of layer l is characterized by the algebraic checks in eq. 11 and eq. 13, and the range checks of eq. 9, eq. 14 and eq. 15. We can compactly describe the algebraic part with a quadratic circuit check $\mathbf{C}^{(l)}\left((\mathbf{c}^{(i)})_{i=0}^{l}, (\mathbf{w}_{\mathrm{ks},i}^{(l)})_{i=0}^{L-l}, (\mathbf{w}_{\mathrm{rmd},b}^{(l)})_{b=0,1}, (\mathbf{w}_{\mathrm{quot},b}^{(l+1)})_{b=0,1}\right) = \mathbf{0}$. The fact that the output of each layer is input to the next layer allows arranging the whole computation into a single, flattened, quadratic polynomial that is the parallel execution, for $l = 0, \ldots, D-1$, of

$$\mathbf{C}^{(l)}\left((\mathbf{c}^{(0)}, (\mathbf{w}_{\mathrm{quot},}^{(i)})_{i=1}^{l}), (\mathbf{w}_{\mathrm{ks},i}^{(l)})_{i=0}^{L-l}, (\mathbf{w}_{\mathrm{rmd},b}^{(l)})_{b=0,1}, (\mathbf{w}_{\mathrm{quot},b}^{(l+1)})_{b=0,1}\right).$$

We compactly denote this polynomial with $\mathbf{C}(\mathbf{x}, \mathbf{w}_{\mathrm{ks}}, \mathbf{w}_{\mathrm{rmd}}, \mathbf{w}_{\mathrm{quot}}, \mathbf{y})$, where $\mathbf{x} := \{\mathbf{c}_b^{(0)}\}_b$, $\mathbf{w}_{\mathrm{ks}} := \{\mathbf{w}_{\mathrm{ks},i}^{(l)}\}_{i,l}$, $\mathbf{w}_{\mathrm{rmd}} := \{\mathbf{w}_{\mathrm{rmd},b}^{(l)}\}_{l,b}$, $\mathbf{w}_{\mathrm{quot}} := \{\mathbf{w}_{\mathrm{quot},b}^{(l)}\}_{l<D,b}$, $\mathbf{y} = \{\mathbf{w}_{\mathrm{quot},b}^{(D)}\}_b$. On the other hand, the range constraints can be verified separately.

Optimizations. For the range constraints, we can use the following optimization. In eq. 9 we can take the bound to be 2^{u_i}, where u_i is such that $2^{u_i-1} < p_i < 2^{u_i}$. Correctness of the values $\mathbf{w}_{\mathrm{ks},i}^{(l)}$ is still guaranteed by the recomposition check in eq. 11. Given our specific choice of the RNS primes p_i (Sec. 6), there is a single

u that satisfies the above inequalities for all p_i. This is particularly useful, as we can use a single batched range proof for all the elements $(\mathbf{w}^{(i)}_{\text{ks},0}, \ldots, \mathbf{w}^{(i)}_{\text{ks},L-i})_{i=1}^{D}$. The same optimization can be used for the constraints in eq. 15. Note that, the values $\mathbf{w}^{(l)}_{\text{rmd},b}$ will still give a small representative of $[\mathbf{c}'^{(l)}_b]_{p_{L-l}}$ in R_q, namely $\mathbf{w}^{(l)}_{\text{rmd},b} = [\mathbf{c}'^{(l)}_b]_{p_{L-l}} + k \cdot p_{L-l}$ with $k \in \{0, \pm 1\}$. This will only affect the noise by a small factor, as shown in Sec. 3.4 and Appendix. B.

PIOP for $\mathcal{R}_{CKKS}$ We present the relation for a computation on CKKS ciphertexts including the above optimization to do a single range check on all the values that share the same bound. Specifically, we define the following oracles:

- $[\![\tilde{\mathbf{x}}]\!]$ for the MLE of $\mathbf{x} := \mathbf{c}^{(0)}_0 \| \mathbf{c}^{(0)}_1$;
- $[\![\tilde{\mathbf{y}}]\!]$, for the MLE of $\mathbf{y} := \mathbf{w}^{(D)}_{\text{quot},0} \| \mathbf{w}^{(D)}_{\text{quot},1}$;
- $[\![\tilde{\mathbf{w}}_{2^u}]\!]$ for the MLE of $\mathbf{w}_{2^u} := \mathbf{w}^{(1)}_{\text{ks},0} \| \ldots \| \mathbf{w}^{(D-1)}_{\text{ks},0} \| \mathbf{w}^{(1)}_{\text{rmd},0} \| \mathbf{w}^{(1)}_{\text{rmd},1} \| \ldots \| \mathbf{w}^{(D-1)}_{\text{rmd},0} \| \mathbf{w}^{(D-1)}_{\text{rmd},1}$;
- $[\![\tilde{\mathbf{w}}_l]\!]$ for the MLE of $\mathbf{w}_l := \mathbf{w}^{(l)}_{\text{quot},0} \| \mathbf{w}^{(l)}_{\text{quot},1}$, for $l = 1, \ldots, D-1$;

Given a vector $\mathbf{w}$ of elements in $R_q := \mathbb{Z}_q[X]/(X^N + 1)$, we use $\|\mathbf{w}\|_\infty < B$ to denote that, for each element of the vector, all its monomial coefficients are in the range $[0, B-1]$. The relation that describes a computation of depth D on CKKS ciphertexts is:[7]

$$\mathcal{R}_{CKKS} = \Big\{ (\mathbf{C}, ([\![\tilde{\mathbf{x}}]\!], [\![\tilde{\mathbf{w}}_{2^u}]\!], \{[\![\tilde{\mathbf{w}}_l]\!]\}_{l=1}^{D-1}, [\![\tilde{\mathbf{y}}]\!]); (\mathbf{x}, \mathbf{w}_{2^u}, \{\mathbf{w}_l\}_{l=1}^{D-1}, \mathbf{y})) \ : \\ \mathbf{C}(\mathbf{x}, \mathbf{w}_{2^u}, \{\mathbf{w}_l\}_{l=1}^{D-1}, \mathbf{y}) = \mathbf{0} \wedge \|\mathbf{w}_{2^u}\|_\infty < 2^u \wedge \bigwedge_{l=1}^{D-1} \|\mathbf{w}_l\|_\infty < q/p_{L-l-1} \wedge \|\mathbf{y}\|_\infty < q/p_{D-1} \Big\}$$

Finally, we give a PIOP for $\mathcal{R}_{CKKS}$, which relies on PIOPs for the relations

$$\mathcal{R}_{range} = \{(\mathtt{i}, \mathtt{x}; \mathtt{w}) := (B, [\![\tilde{\mathbf{w}}]\!]; \mathbf{w}) \ : \ \|\mathbf{w}\|_\infty < B\}$$
$$\mathcal{R}_{AC} = \{(\mathtt{i}, \mathtt{x}; \mathtt{w}) := (\mathbf{C}, ([\![\tilde{\mathbf{x}}]\!], [\![\tilde{\mathbf{y}}]\!], [\![\tilde{\mathbf{w}}]\!]); (\mathbf{x}, \mathbf{w})) \ : \ \mathbf{C}(\mathbf{x}, \mathbf{y}; \mathbf{w}) = \mathbf{0}\}$$

PIOP Π_{CKKS} for $(\mathtt{i}, \mathtt{x}; \mathtt{w}) \in \mathcal{R}_{CKKS}$

$\mathcal{P}$ and $\mathcal{V}$ run Π_{AC} for $(\mathbf{C}, ([\![\tilde{\mathbf{x}}]\!], [\![\tilde{\mathbf{y}}]\!], [\![\tilde{\mathbf{w}}_{2^u}]\!], \{[\![\tilde{\mathbf{w}}_l]\!]\}_{l=1}^{D-1}); \mathbf{x}, \mathbf{y}, \mathbf{w}_{2^u}, \{\mathbf{w}_l\}_{l=1}^{D-1}) \in \mathcal{R}_{AC}$
$\mathcal{P}$ and $\mathcal{V}$ run Π_{range} for $(2^u, [\![\tilde{\mathbf{w}}_{2^u}]\!]; \mathbf{w}_{2^u}) \in \mathcal{R}_{range}$ and $(q/p_{D-1}, [\![\tilde{\mathbf{y}}]\!]; \mathbf{y}) \in \mathcal{R}_{range}$
For $l = 1, \ldots, D-1$,
  $\mathcal{P}$ and $\mathcal{V}$ run Π_{range} for $(q/p_{L-l-1}, [\![\tilde{\mathbf{w}}_l]\!]; \mathbf{w}_l) \in \mathcal{R}_{range}$.

Remark 4.1. (Automorphisms) The homomorphic evaluation of Galois automorphism is often seen as a challenging operation to prove due to its reliance on key switchings. Since we show how to prove key switching efficiently, the main challenge is solved. It remains to show that we are able to prove the (plaintext) automorphisms themselves. For completeness, we recall the Rotate algorithm in Appendix A. Here we give a high level idea how to use our techniques for

[7] The range check on $\mathbf{y}$ can be removed when $\mathbf{y}$ is known to the verifier.

arithmetizing the automorphism operation. Let $(c_0, c_1) \in R_q^2$. An automorphism indexed by $a \in \mathbb{Z}_{2N}^*$ acts on the ciphertexts as $(c_0(X^a), c_1(X^a)) \in R_q^2$. Notice that this is not an algebraic operation over the ring R_q, as it involves the coefficients of the ciphertexts. To prove it, we use a decomposition technique that we will employ later in Sec. 4.3 in our range proof. First, the prover inputs non-deterministic elements $(\mathbf{w}_{\text{aut},0}, \mathbf{w}_{\text{aut},1}) \in R_q^{2N}$, claimed to be the vectors whose components are the coefficients of c_0, c_1, seen as degree-0 polynomials. Then it computes

$$\mathbf{c}'_i := \mathbf{P}_a \cdot \mathbf{w}_{\text{aut},i} \in R_q^N, \qquad i = 0, 1$$

where $\mathbf{P}_a \in R_q^{N \times N}$ is the matrix defined by X^a. Finally, it re-composes the vectors $\mathbf{c}'_0, \mathbf{c}'_1$ to obtain elements $c'_0 \in R_q$ and $c'_1 \in R_q$ that correspond to $c_0(X^a)$ and $c_1(X^a)$. This is an algebraic operation over R_q. To summarize, proving an automorphism reduces to two relations, namely an arithmetic circuit relation given by the multiplication by $\mathbf{P}_a$, the re-composition procedure and a relation similar to $\mathcal{R}_{decomp}$ in Sec. 4.3, that decomposes an element in R_q into an array of coefficients interpreted as degree-0 elements.

4.2 PIOP for $\mathcal{R}_{AC}$

Π_{AC}, our PIOP for the arithmetic relation $\mathcal{R}_{AC}$, is based on an optimized PIOP version of the GKR protocol [GKR08, GKR15]. Towards building Π_{AC}, we first describe $\mathcal{R}_{AC}$ as a circuit consisting of the four following layers:

- Layer 3 consists of the input ciphertexts to the VC scheme, together with the non-deterministic inputs provided by the prover. These non-deterministic inputs include all intermediate ciphertexts that result from HE multiplication, as well as the claimed output HE ciphertexts.
- Layer 2 has the outputs of a series of *inner product gates*. These gates encode any linear operations performed on the input and/or intermediate ciphertexts, together with the subsequent pre-multiply step of CKKS, i.e. the quadratic polynomials $\mathbf{Q}_0, \mathbf{Q}_1$ and $\mathbf{Q}_2$ from Section 4.1.
- Layer 1 contains the outputs of the key-switching procedure.
- Layer 0 are the output wires, which are hardcoded to 0 for satisfiability of the "base decomposition consistency" and "rescaling consistency" gates.

NOTATION:

- We use $\#(x)$ to denote $\lceil \log_2(|x|) \rceil$ for a set of wires x.
- By abuse of notation, we use d_2 (resp. c') to refer to the set of wires holding such values, i.e. such output of the premultiply (resp. key switching) step.
- d_{01} refers to the set of wires holding values d_0, d_1, i.e. such output of premultiply.
- $I_{\mathcal{P}}$ refers to the wires holding the non-deterministic prover inputs. BD refers to the subset of $I_{\mathcal{P}}$ containing the base decomposition of d_2 values. *quot* refers to the "quotients" in the rescaling step, i.e. the outputs of the HE rescaling operation.
- We use *in* (resp. *out*) to refer to the wires holding the encrypted inputs (resp. outputs) of the verifiable computation system.
- *aux* refers to all the non-deterministic prover inputs which are not outputs, i.e. $aux = I_{\mathcal{P}} \setminus out$. In particular, it consists of the set of wires $aux, 2^u$ (for values $< 2^u$) and, for $j = 0, \ldots, D-1$, sets aux, j (for values $< q/q_{L-j-1}$).
- $V_i(\mathbf{x})$ (resp. $V_S(\mathbf{x})$) refers to the value on wire $\mathbf{x}$ of layer i (resp. the set S).

As in standard GKR, we will reduce a claim about the output layer to claims about the input layer by making use of sum-check. In more details, we introduce $\mathcal{R}_{AC}$-optimized *layer consistency equations* which relate a claim about the MLE of the values on some layer to a claim about the MLEs of values in previous ones. Eventually, all of these claims are reduced to claims that can be checked by querying the oracles present in $\mathcal{R}_{AC}$.

Layer consistency equations The consistency of the output layer with previous ones is described in Equation (16). Towards this, we introduce new predicates for the consistency of rescaling and base decomposition. Two thirds of the evaluations of $\tilde{V}_0(\mathtt{Z})$ in the boolean hypercube correspond to rescaling (one for c'_0, another for c'_1) and the other third to the base decomposition. For a relation $c''_b = \left(c'_b - [c'_b]_{p_l}\right) \mathsf{p}_l^{-1}$, $b \in \{0,1\}$ we call c''_b the quotient and $[c'_b]_{p_l}$ the remainder. The layer consistency equation for this layer is:

$$0 = \tilde{V}_0(\mathtt{Z}) = \sum_{\substack{\mathbf{x}\in\{0,1\}^{\#(c')}, \mathbf{y}\in\{0,1\}^{\#(I_{\mathcal{P}})} \\ \mathbf{t}\in\{0,1\}^{\#(d_2)}}} \Big(\widetilde{\mathtt{rescon}}_1(\mathtt{Z},\mathbf{x}) \cdot \tilde{V}_1(\mathbf{x}) + \widetilde{\mathtt{rescon}}_3(\mathtt{Z},\mathbf{y}) \cdot \tilde{V}_{3,I_{\mathcal{P}}}(\mathbf{y}) + \widetilde{\mathtt{bdcon}}_2(\mathtt{Z},\mathbf{t}) \cdot \tilde{V}_{2,d_2}(\mathbf{t}) + \widetilde{\mathtt{bdcon}}_3(\mathtt{Z},\mathbf{y}) \cdot \tilde{V}_{3,I_{\mathcal{P}}}(\mathbf{y}) \Big). \tag{16}$$

where predicates $\mathtt{rescon}_1, \mathtt{rescon}_3, \mathtt{bdcon}_2, \mathtt{bdcon}_3$ are 0 everywhere except in the following cases

$\mathtt{rescon}_1(\mathbf{z},\mathbf{x}) = 1$, if wire $\mathbf{x}$ holds the $\mathbf{z}$-th c' value.

$\mathtt{rescon}_3(\mathbf{z},\mathbf{y}) = -1$, if wire $\mathbf{y}$ holds the $\mathbf{z}$-th remainder.

$\mathtt{rescon}_3(\mathbf{z},\mathbf{y}) = -\mathsf{p}_l$, if $\mathbf{y}$ belongs to the l-th HE layer and holds the $\mathbf{z}$-th quotient.

$\mathtt{bdcon}_2(\mathbf{z},\mathbf{t}) = 1$, if wire $\mathbf{t}$ holds the $\mathbf{z}$-th d_2 value.

$\mathtt{bdcon}_3(\mathbf{z},\mathbf{y}) = -\mathsf{PW}_{\omega_l}(1)[i]$, if $\mathbf{y}$ belongs to the l-th HE layer and holds the i-th

decomposition of the $\mathbf{z}$-th d_2 value.

Consistency between layer 1 (which contains the outputs of key switching) and both the outputs of pre-multiply (in layer 2) together with the base decomposition of d_2 values is described as follows:

$$\tilde{V}_1(\mathtt{Z}) = \sum_{\mathbf{x}\in\{0,1\}^{\#(d_{01})},\mathbf{y}\in\{0,1\}^{\#(BD)}} \Big(\widetilde{\mathtt{add}}(\mathtt{Z},\mathbf{x})\cdot\tilde{V}_{2,d_{01}}(\mathbf{x}) + \widetilde{\mathtt{evk}}(\mathtt{Z},\mathbf{y})\cdot\tilde{V}_{3,BD}(\mathbf{y})\Big). \tag{17}$$

The corresponding predicates `add` (resp. `evk`) appear in Equation (30) (resp. Equation (31)) Appendix E.1.

Finally, consistency between layers 2 and 3 can be done through the predicate for inner products $\mathtt{Xmult}(\mathbf{z},\mathbf{x},\mathbf{y})$ from [LXZ21] (see Equation (32) in Appendix E.1). Notice how, given the "flattened" version of the HE circuit these inner products are enough to verify its satisfiability.

$$\tilde{V}_2(\mathtt{Z}) = \sum_{\substack{\mathbf{x}\in\{0,1\}^{\#(in)},\mathbf{u}\in\{0,1\}^{\#(quot)}\\ \mathbf{y}\in\{0,1\}^{\#(in)},\mathbf{v}\in\{0,1\}^{\#(quot)}}} \Big(\widetilde{\mathtt{Xmult}}(\mathtt{Z},\mathbf{x},\mathbf{y})\cdot\tilde{V}_{in}(\mathbf{x})\cdot\tilde{V}_{in}(\mathbf{y}) +$$
$$\widetilde{\mathtt{Xmult}}(\mathtt{Z},\mathbf{u},\mathbf{v})\cdot\tilde{V}_{3,quot}(\mathbf{u})\cdot\tilde{V}_{3,quot}(\mathbf{v}) + \widetilde{\mathtt{Xmult}}(\mathtt{Z},\mathbf{x},\mathbf{v})\cdot\tilde{V}_{in}(\mathbf{x})\cdot\tilde{V}_{3,quot}(\mathbf{v})\Big). \tag{18}$$

Progressing through the layers The Π_{AC} PIOP starts by invoking the sum-check *protocol* (not the PIOP) on Equation (16) and moving towards layer 3. At the end of the execution of the sum-check for each of the layer consistency equations, $\mathcal{P}$ sends to $\mathcal{V}$ what it claims are $\{\tilde{V}_{S_j}(\mathbf{r}_j)\}_j$, for random sum-check challenges $\mathbf{r}_j$ and the relevant sets of wires S_j in that equation.

After executing sum-check on Equation (17) and before executing it on Equation (18), we need to reduce claimed values $\tilde{V}_{2,d_{01}}(\mathbf{r}_{01})$ (from the sum-check on Equation (17)) and $\tilde{V}_{2,d_2}(\mathbf{r}_2)$ (from sum-check on Equation (16)) into a claim about $\tilde{V}_2(\mathtt{Z})$. In order to do this, we follow the approach from [ZLW+21]. For a set S among $\{d_{01}, d_2\}$, define predicates $\mathtt{C}_{2,S}(\mathtt{Y},\mathtt{Z})$ as follows:

$$\mathtt{C}_{2,S}(\mathbf{y},\mathbf{z}) = \begin{cases} 1, & \text{if wire } \mathbf{y} \text{ in } V_{2,S} \text{ is the } \mathbf{z}\text{-th wire in } V_2.\\ 0, & \text{otherwise.}\end{cases}$$

$\mathcal{V}$ samples $s_{01}, s_2 \leftarrow\$ S$ so as to randomly combine $\mathcal{P}$'s claims. Now, by running the sum-check protocol on Equation (19), $\mathcal{P}$ ends by claiming an evaluation $\tilde{V}_2(\mathbf{s})$ (for a random sum-check challenge $\mathbf{s} \leftarrow\$ S^{\#(V_2)}$) which can be substituted into Equation (18) so as to advance towards the input layer.

$$s_{01}\tilde{V}_{2,d_{01}}(\mathbf{r}_{01}) + s_2\tilde{V}_{2,d_2}(\mathbf{r}_2) =$$

$$s_{01}\Big(\sum_{\mathbf{z}\in\{0,1\}^{\#(V_2)}}\widetilde{\mathtt{C}}_{2,d_{01}}(\mathbf{r}_{01},\mathbf{z})\tilde{V}_2(\mathbf{z})\Big)+s_2\Big(\sum_{\mathbf{z}\in\{0,1\}^{\#(V_2)}}\widetilde{\mathtt{C}}_{2,d_2}(\mathbf{r}_2,\mathbf{z})\tilde{V}_2(\mathbf{z})\Big)=$$
$$\sum_{\mathbf{z}\in\{0,1\}^{\#(V_2)}}\tilde{V}_2(\mathbf{z})\Big(s_{01}\widetilde{\mathtt{C}}_{2,d_{01}}(\mathbf{r}_{01},\mathbf{z})+s_2\widetilde{\mathtt{C}}_{2,d_2}(\mathbf{r}_2,\mathbf{z})\Big). \tag{19}$$

After executing the sum-check protocol on Equation (18), we still need to match the syntax of Π_{AC} given in Π_{CKKS}. Hence, we want to reduce claims $\tilde{V}_{3,quot}(\mathbf{r}_{quot}), \tilde{V}_{3,quot}(\mathbf{r}'_{quot}), \tilde{V}_{3,BD}(\mathbf{r}_{BD})$ and $\tilde{V}_{3,I_\mathcal{P}}(\mathbf{r}_{I_\mathcal{P}})$ to claims about $D+1$ oracles: $[\![\tilde{V}_{out}]\!]$ for the claimed ciphertext outputs, $[\![\tilde{V}_{aux,2^u}]\!]$ for all non-deterministic inputs from the prover that need to be proven smaller than 2^u and finally, for $j=1,\ldots,D-1$, $[\![\tilde{V}_{aux,j}]\!]$ for non-deterministic inputs from the prover that need to be proven smaller than q/q_{L-j-1} . We generalize the strategy from [ZLW+21] that we applied in layer 2 so that we can reduce claims about sets S_i into sets T_k as long as $(\cup_i S_i)\subseteq(\cup_k T_k)$. In our concrete situation, notice that *out* and *aux* are disjoint sets of wires which contain $quot\cup BD\cup I_\mathcal{P}$. For a set S among $\{quot, BD, I_\mathcal{P}\}$ and $j\in\{1,\ldots,D-1\}\cup\{2^u\}$ we define predicates:

$$\mathtt{C}_{a,j,S}(\mathbf{y},\mathbf{z})=\begin{cases}1, & \text{if wire } \mathbf{y} \text{ in } V_S \text{ is the } \mathbf{z}\text{-th wire in } V_{aux,j}.\\ 0, & \text{otherwise.}\end{cases}$$
$$\mathtt{C}_{o,S}(\mathbf{y},\mathbf{t})=\begin{cases}1, & \text{if wire } \mathbf{y} \text{ in } V_S \text{ is the } \mathbf{t}\text{-th wire in } V_{out}.\\ 0, & \text{otherwise.}\end{cases}$$

$\mathcal{V}$ samples $s_{quot}, s'_{quot}, s_{BD}, s_{I_\mathcal{P}} \leftarrow\!\!\$\ S$, so that $\mathcal{P}$ and $\mathcal{V}$ execute the sum-check *PIOP* on Equation (20) to obtain the reduction we are looking for. We write $\mathtt{C}_{a,j,q}$ instead of $\mathtt{C}_{a,j,quot}$ due to space constraints.

$$s_{quot}\tilde{V}_{3,quot}(\mathbf{r}_{quot})+s'_{quot}\tilde{V}_{3,quot}(\mathbf{r}'_{quot})+s_{BD}\tilde{V}_{3,BD}(\mathbf{r}_{BD})+s_{I_\mathcal{P}}\tilde{V}_{3,I_\mathcal{P}}(\mathbf{r}_{I_\mathcal{P}})=$$
$$\sum_{\mathbf{x}\in\{0,1\}^{\#(aux,2^u)}}\tilde{V}_{aux,2^u}(\mathbf{x})\cdot\Big(s_{BD}\widetilde{\mathtt{C}}_{a,2^u,BD}(\mathbf{r}_{BD},\mathbf{x})+s_{I_\mathcal{P}}\widetilde{\mathtt{C}}_{a,2^u,I_\mathcal{P}}(\mathbf{r}_{I_\mathcal{P}},\mathbf{x})\Big)+\sum_{j=1}^{D-1}$$
$$\Big(\sum_{\mathbf{z}_j\in\{0,1\}^{\#(aux,j)}}\tilde{V}_{aux,j}(\mathbf{z}_j)\cdot\Big(s_{quot}\widetilde{\mathtt{C}}_{a,j,q}(\mathbf{r}_{quot},\mathbf{z}_j)+s'_{quot}\widetilde{\mathtt{C}}_{a,j,q}(\mathbf{r}'_{quot},\mathbf{z}_j)+s_{I_\mathcal{P}}\widetilde{\mathtt{C}}_{a,j,I_\mathcal{P}}(\mathbf{r}_{I_\mathcal{P}},\mathbf{z}_j)\Big)\Big)$$
$$+\sum_{\mathbf{t}\in\{0,1\}^{\#(out)}}\tilde{V}_{out}(\mathbf{t})\cdot\Big(s_{quot}\widetilde{\mathtt{C}}_{o,quot}(\mathbf{r}_{quot},\mathbf{t})+s'_{quot}\widetilde{\mathtt{C}}_{o,quot}(\mathbf{r}'_{quot},\mathbf{t})+s_{I_\mathcal{P}}\widetilde{\mathtt{C}}_{o,I_\mathcal{P}}(\mathbf{r}_{I_\mathcal{P}},\mathbf{t})\Big). \tag{20}$$

Putting all of the above together, we obtain Π_{AC}, the PIOP for $\mathcal{R}_{AC}$. The witness $\mathbb{w}$ is formed by $\tilde{V}_{aux,2^u}$ and $\{\tilde{V}_{aux,j}\}_{j=0}^{D-1}$. The statement $\mathbb{x}$ consists of pointers to the input and output oracles $[\![\tilde{V}_{in}]\!], [\![\tilde{V}_{out}]\!]$ as well as the witness $[\![\tilde{V}_{aux,2^u}]\!], \{[\![\tilde{V}_{aux,j}]\!]\}_{j=1}^{D-1}$. The index $\mathbb{i}$ consists of oracles to all the wiring predicates we have introduced, namely $[\![\widetilde{\mathtt{rescon}_1}]\!], [\![\widetilde{\mathtt{rescon}_3}]\!], [\![\widetilde{\mathtt{bdcon}_2}]\!], [\![\widetilde{\mathtt{bdcon}_3}]\!], [\![\widetilde{\mathtt{add}}]\!]$, $[\![\widetilde{\mathtt{evk}}]\!], [\![\widetilde{\mathtt{Xmult}}]\!], [\![\widetilde{\mathtt{C}}_{2,d_{01}}]\!], [\![\widetilde{\mathtt{C}}_{2,d_2}]\!], \{[\![\mathtt{C}_{a,j,q}]\!], [\![\mathtt{C}_{a,j,I_\mathcal{P}}]\!]\}_{j=1}^{D-1}, [\![\mathtt{C}_{a,2^u,BD}]\!], [\![\mathtt{C}_{a,2^u,I_\mathcal{P}}]\!], [\![\mathtt{C}_{o,quot}]\!]$ and $[\![\mathtt{C}_{o,I_\mathcal{P}}]\!]$. Due to space constraints, the precise Π_{AC} protocol and the proof of the following theorem appear in Appendix E.1.

Theorem 4.2. *Let $S \subseteq R_q$ be an exceptional set. Let Π_{sum} be a PIOP for $\mathcal{R}_{sum}$. Then Π_{AC} is a PIOP for $\mathcal{R}_{AC}$ with perfect completeness and knowledge soundness error $O(|\mathbf{C}|/|S|)$.*

4.3 A PIOP for range checks

We present a PIOP for proving that a polynomial is the multilinear extension of a vector of elements from the ring $R_q := \mathbb{Z}_q[X]/(X^N + 1)$ whose coefficients are all in the range $[0, B-1]$. Here we assume that B is a power of 2. At the end of the section we discuss how to deal with the more general case.

We model range checks as a table lookup and thus we construct a PIOP for the indexed oracle relation

$$\mathcal{R}_{range} = \{(\mathbb{i}, \mathbb{x}; \mathbb{w}) = (T_B, [\![\tilde{v}]\!]; \tilde{v}) : \forall \boldsymbol{i} \in \{0,1\}^\ell \ \tilde{v}(\boldsymbol{i}) \in T_B\} \tag{21}$$

where $\tilde{v}$ is a multilinear polynomial in $R_q[X_1, \ldots, X_\ell]$, and T_B is the table containing all the elements of R_q with coefficients in $[0, B-1]$. T_B has B^N entries that we index by tuples $(\mathsf{idx}_1, \ldots, \mathsf{idx}_N) \in [0, B-1]^N$ such that $T_B[\mathsf{idx}_1, \ldots, \mathsf{idx}_N] = \sum_{j=1}^N \mathsf{idx}_j \cdot X^{j-1} \in R_q$.

Below we present the idea of the construction, which is an extension of Lasso's techniques for table lookups [STW24] to polynomial rings.

Decomposing T_B. Let $B = 2^b$, $N = 2^\nu$, let $c = 2^\gamma$ be an integer such that $c \mid b$ and $\beta = B^{1/c}$. Let $\mathsf{dig}_k^{(\beta)}(\mathsf{idx})$ be the function that on input an integer idx returns the k-th digit of its base-β decomposition, i.e., idx_k such that $\mathsf{idx} = \sum_k \mathsf{idx}_k \beta^{k-1}$. Let $\mathsf{to\text{-}int}(\boldsymbol{j})$ be the function that on input a binary vector $\boldsymbol{j}$ returns the integer $\sum_k j_k 2^{k-1}$. We state the following lemma which shows how to "decompose" a lookup into T_B into $c \cdot N$ lookups in the smaller $\boldsymbol{t}_\beta = \{0, \ldots, \beta-1\}$ of size β.

Lemma 4.3. *Let $\tilde{v} \in R_q[X_1, \ldots, X_\ell]$ and $\boldsymbol{t}_\beta = \{0, \ldots, \beta-1\}$. Then $\forall \boldsymbol{i} \in \{0,1\}^\ell$, $\tilde{v}(\boldsymbol{i}) \in T_B$ iff $\exists \tilde{h} \in R_q[X_1, \ldots, X_{\ell+\nu+\gamma}]$ such that*

$$\forall \boldsymbol{i} \in \{0,1\}^\ell : \tilde{v}(\boldsymbol{i}) = \sum_{\substack{\boldsymbol{j} \in \{0,1\}^\nu \\ \boldsymbol{k} \in \{0,1\}^\gamma}} \tilde{h}(\boldsymbol{i}, \boldsymbol{j}, \boldsymbol{k}) \cdot X^{\mathsf{to\text{-}int}(\boldsymbol{j})} \cdot \beta^{\mathsf{to\text{-}int}(\boldsymbol{k})} \tag{22}$$

$$\forall \boldsymbol{i} \in \{0,1\}^\ell, \boldsymbol{j} \in \{0,1\}^\nu, \boldsymbol{k} \in \{0,1\}^\gamma : \tilde{h}(\boldsymbol{i}, \boldsymbol{j}, \boldsymbol{k}) \in \boldsymbol{t}_\beta \tag{23}$$

The proof of the lemma follows from the observation that every element of T_B can be expressed as the following combination of elements of $\boldsymbol{t}_\beta$.

$$\begin{aligned} T_B[\mathsf{idx}_1, \ldots, \mathsf{idx}_N] &= \sum_{j=1}^N \mathsf{idx}_j \cdot X^{j-1} = \sum_{j=1}^N \sum_{k=1}^c \mathsf{dig}_k^{(\beta)}(\mathsf{idx}_j) \cdot \beta^{k-1} \cdot X^{j-1} \\ &= \sum_{j=1}^N \sum_{k=1}^c \boldsymbol{t}_\beta[\mathsf{dig}_k^{(\beta)}(\mathsf{idx}_j)] \cdot X^{j-1} \cdot \beta^{k-1} \end{aligned}$$

Let $\tilde{p}_{\beta,N}$ be the multilinear polynomial such that $\forall \boldsymbol{j} \in \{0,1\}^{\nu}, \boldsymbol{k} \in \{0,1\}^{\gamma}$: $\tilde{p}_{\beta,N}(\boldsymbol{j},\boldsymbol{k}) = X^{\mathsf{to\text{-}int}(\boldsymbol{j})} \cdot \beta^{\mathsf{to\text{-}int}(\boldsymbol{k})}$. We can use Lemma 4.3 to derive

$$(T_B, [\![\tilde{v}]\!]; \tilde{v}) \in \mathcal{R}_{range} \iff \exists \tilde{h} : (\tilde{p}_{\beta,N}, ([\![\tilde{v}]\!], [\![\tilde{h}]\!]); (\tilde{v}, \tilde{h})) \in \mathcal{R}_{decomp} \\ \wedge (\boldsymbol{t}_\beta, [\![\tilde{h}]\!]; \tilde{h}) \in \mathcal{R}_{range}$$

$$\text{where } \mathcal{R}_{decomp} = \{ \ (\mathtt{i}, \mathtt{x}; \mathtt{w}) = ([\![\tilde{p}_{\beta,N}]\!], ([\![\tilde{v}]\!], [\![\tilde{h}]\!]); (\tilde{p}_{\beta,N}, \tilde{v}, \tilde{h})) : \\ \forall \boldsymbol{i} \in \{0,1\}^{\ell} \ \tilde{v}(\boldsymbol{i}) = \sum_{(\boldsymbol{j},\boldsymbol{k}) \in \{0,1\}^{\nu+\gamma}} \tilde{h}(\boldsymbol{i},\boldsymbol{j},\boldsymbol{k}) \cdot \tilde{p}_{\beta,N}(\boldsymbol{j},\boldsymbol{k}) \}$$

and to construct the following PIOP.

PIOP Π_{range} for $(T_B, [\![\tilde{v}]\!]; \tilde{v}) \in \mathcal{R}_{range}$

Let $B = 2^b$, $N = 2^\nu$, and let $c = 2^\gamma$ be an integer such that $c \mid b$ and $\beta = B^{1/c}$.

$\mathcal{P}$ computes the multilinear polynomial $\tilde{h} \in R_q[X_1, \ldots, X_{\ell+\nu+\gamma}]$ as the decomposition of $\tilde{v}$ according to equation (22), and sends the oracle $[\![\tilde{h}]\!]$.

$\mathcal{P}$ and $\mathcal{V}$ run

- a PIOP Π_{decomp} for $([\![\tilde{p}_{\beta,N}]\!], ([\![\tilde{v}]\!], [\![\tilde{h}]\!]); (\tilde{p}_{\beta,N}, \tilde{v}, \tilde{h})) \in \mathcal{R}_{decomp}$.
- a PIOP Π^{β}_{range} for $(\boldsymbol{t}_\beta, [\![\tilde{h}]\!]; \tilde{h}) \in \mathcal{R}_{range}$, where $\boldsymbol{t}_\beta = \{0, \ldots, \beta - 1\}$.

Theorem 4.4. *Let Π_{decomp} (resp. Π^{β}_{range}) be a PIOP for $\mathcal{R}_{decomp}$ (resp. $\mathcal{R}_{range}$) with perfect completeness and knowledge error δ_{decomp} (resp. δ_{range}). Then Π_{range} is a PIOP with perfect completeness and knowledge error $\delta_{decomp} + \delta_{range}$.*

The proof is immediate by definition of $\mathcal{R}_{range}$ and by the fact that the two PIOPs share the oracle $[\![\tilde{h}]\!]$. Therefore, we continue by showing a PIOP Π_{decomp} for $\mathcal{R}_{decomp}$, and a PIOP for $\mathcal{R}_{range}$, dubbed Π^{β}_{range}, in which the table $\boldsymbol{t}_\beta$ in the index only contains the positive integers $\{0, 1, \ldots, \beta - 1\}$.

PIOP for $\mathcal{R}_{decomp}$. To check that two multilinear polynomials $\tilde{v}, \tilde{h}$ satisfy equation (22), the verifier samples a random challenge $\boldsymbol{r}_i \leftarrow\!\!\$\ S^\ell$ and we use the sum-check protocol to show that $\tilde{v}(\boldsymbol{r}_i) = \sum_{(\boldsymbol{j},\boldsymbol{k}) \in \{0,1\}^{\nu+\gamma}} \tilde{h}(\boldsymbol{r}_i,\boldsymbol{j},\boldsymbol{k}) \cdot \tilde{p}_{\beta,N}(\boldsymbol{j},\boldsymbol{k})$.

PIOP Π_{decomp} for $([\![\tilde{p}_{\beta,N}]\!], ([\![\tilde{v}]\!], [\![\tilde{h}]\!]); (\tilde{p}_{\beta,N}, \tilde{v}, \tilde{h})) \in \mathcal{R}_{decomp}$

$\mathcal{V}$ sends a random challenge $\boldsymbol{r}_i \leftarrow\!\!\$\ S^\ell$

$\mathcal{P}$ sends $y_v = \tilde{v}(\boldsymbol{r}_i)$

Define the virtual oracle $[\![p^*]\!] = [\![\tilde{p}_{\beta,N}]\!] \cdot [\![\tilde{h}']\!]$ where
$\tilde{h}'(X_{\ell+1}, \ldots, X_{\ell+\nu+\gamma}) = \tilde{h}(\boldsymbol{r}_i, X_{\ell+1}, \ldots, X_{\ell+\nu+\gamma})$

$\mathcal{P}$ and $\mathcal{V}$ run the sum-check PIOP for $((y_v, [\![p^*]\!]); p^*) \in \mathcal{R}_{sum}$ where $\mathcal{V}$ answers any query to $\tilde{h}'(\boldsymbol{r}_j, \boldsymbol{r}_k)$ with a query to $\tilde{h}(\boldsymbol{r}_i, \boldsymbol{r}_j, \boldsymbol{r}_k)$.

// Alternatively, if a $O(c \cdot N)$ cost is affordable by $\mathcal{V}$, $\mathcal{V}$ can compute $\tilde{p}_{\beta,N}(\boldsymbol{r}_j, \boldsymbol{r}_k)$ on its own.

Theorem 4.5. *Let $S \subseteq R_q$ be an exceptional set. Π_{decomp} is a PIOP for $\mathcal{R}_{decomp}$ with perfect completeness and knowledge error $(\ell + 2\nu + 2\gamma)/|S|$.*

For lack of space the proof is in Appendix E.2.

PIOP for lookup in $\boldsymbol{t}_\beta$. To prove that a multilinear polynomial $\tilde{h}$ satisfies equation (23), that is a lookup of $\tilde{h}$ into $\boldsymbol{t}_\beta$, we proceed in two steps. First, we present a PIOP $\Pi^{\mathbb{F},\beta}_{range}$ for the lookup into $\boldsymbol{t}_\beta$ of a multilinear polynomial defined over a prime field $\mathbb{F} = \mathbb{Z}_p$; this protocol is based on a simplification of Spartan's memory checking techniques [Set20]. Second, we show how to build the PIOP Π^{β}_{range} for polynomials over R_q from $\Pi^{\mathbb{F},\beta}_{range}$: the prover sends an auxiliary polynomial $\tilde{h}'$ over $\mathbb{F}$ whose evaluations on the hypercube are the same integer entries of $\boldsymbol{t}_\beta$ as those of $\tilde{h}$, proves that $\tilde{h}$ and $\tilde{h}'$ are consistent, and then runs $\Pi^{\mathbb{F},\beta}_{range}$ on $\tilde{h}'$.

A PIOP for lookups in $\boldsymbol{t}_\beta$ over a finite field. First, we state the following lemma which reduces the lookup relation into an equality of multisets. See Appendix E.2 for the proof.

Lemma 4.6. *Let p, ℓ^* be positive integers such that $p > 2^{\ell^*}$. Let $\mathbb{F}$ be a field of characteristic p, let $\tilde{h}$ be an ℓ^*-variate multilinear polynomial over $\mathbb{F}$, and let $\tilde{t}_\beta$ be the MLE of a table of size $\beta = 2^{b/c}$ whose elements are all distinct. Then we have that $\forall \boldsymbol{i} \in \{0,1\}^{\ell^*}$ $\tilde{h}(\boldsymbol{i}) \in \{\tilde{t}_\beta(\boldsymbol{j}) : \boldsymbol{j} \in \{0,1\}^{b/c}\}$ if and only if there exist polynomials $\mathsf{read_ts} \in \mathbb{F}[X_1, \ldots, X_{\ell^*}], \mathsf{final_cts} \in \mathbb{F}[X_1, \ldots, X_{b/c}]$ such that $\mathsf{WS} = \mathsf{RS} \cup \mathsf{S}$ where*

$$\begin{aligned}
\mathsf{WS} &= \{(\tilde{t}_\beta(\boldsymbol{j}), 0) : \boldsymbol{j} \in \{0,1\}^{b/c}\} \cup \{(\tilde{h}(\boldsymbol{i}), \mathsf{read_ts}(\boldsymbol{i}) + 1) : \boldsymbol{i} \in \{0,1\}^{\ell^*}\} \\
\mathsf{RS} &= \{(\tilde{h}(\boldsymbol{i}), \mathsf{read_ts}(\boldsymbol{i})) : \boldsymbol{i} \in \{0,1\}^{\ell^*}\} \\
\mathsf{S} &= \{(\tilde{t}_\beta(\boldsymbol{j}), \mathsf{final_cts}(\boldsymbol{j})) : \boldsymbol{j} \in \{0,1\}^{b/c}\}
\end{aligned}$$

Following the lemma, the prover of the PIOP $\Pi^{\mathbb{F},\beta}_{range}$ builds the multilinear polynomials $\mathsf{read_ts}, \mathsf{final_cts}$ and aims to convince the verifier that $\mathsf{WS} = \mathsf{RS} \cup \mathsf{S}$.

To prove this multiset equality, the verifier sends random challenges $\sigma, \tau \leftarrow\!\!\$\ \mathbb{F}$, the prover computes and proves

$$\overline{\mathsf{WS}} = \overbrace{\prod_{\boldsymbol{j} \in \{0,1\}^{b/c}} (\tilde{t}_\beta(\boldsymbol{j}) - \tau)}^{\overline{\mathsf{WS}}_t} \overbrace{\prod_{\boldsymbol{i} \in \{0,1\}^{\ell^*}} (\tilde{h}(\boldsymbol{i}) + \sigma(\mathsf{read_ts}(\boldsymbol{i}) + 1) - \tau)}^{\overline{\mathsf{WS}}_h} \tag{24}$$

$$\overline{\mathsf{RS}} = \prod_{\boldsymbol{i} \in \{0,1\}^{\ell^*}} (\tilde{h}(\boldsymbol{i}) + \sigma \cdot \mathsf{read_ts}(\boldsymbol{i}) - \tau) \tag{25}$$

$$\overline{\mathsf{S}} = \prod_{\boldsymbol{j} \in \{0,1\}^{b/c}} (\tilde{t}_\beta(\boldsymbol{j}) + \sigma \cdot \mathsf{final_cts}(\boldsymbol{j}) - \tau) \tag{26}$$

and the verifier checks that $\overline{\mathsf{WS}}_t \cdot \overline{\mathsf{WS}}_h = \overline{\mathsf{RS}} \cdot \overline{\mathsf{S}}$. Finally, to prove the correctness of each of these four values, we use Quark's grand product technique.

Lemma 4.7 (Grand product Lemma [SL20]). *$P = \prod_{\boldsymbol{i}\in\{0,1\}^\ell} v(\boldsymbol{i})$ if and only if there exists a multilinear polynomial f in $\ell+1$ variables such that $f(1,\ldots,1,0) = P$, and $\forall \boldsymbol{i} \in \{0,1\}^\ell$ it holds $f(0,\boldsymbol{i}) = v(\boldsymbol{i})$ and $f(1,\boldsymbol{i}) = f(\boldsymbol{i},0)\cdot f(\boldsymbol{i},1)$.*

In Fig. 1 we give a full specification of the PIOP $\Pi^{\mathbb{F},\beta}_{range}$ for $\mathcal{R}_{range}$ where $\boldsymbol{t}_\beta = \{0,\ldots,\beta-1\}$ and $\tilde{h} \in \mathbb{F}^{(\leq 1)}[X_1,\ldots,X_{\ell^*}]$. Given a vector $\boldsymbol{x} = (x_1,\ldots,x_n)$, we denote by $\boldsymbol{x}'$ the vector $(x_2,\ldots,x_n)$.

PIOP $\Pi^{\mathbb{F},\beta}_{range}$ for $(\boldsymbol{t}_\beta, [\![\tilde{h}]\!]; \tilde{h}) \in \mathcal{R}_{range}$, with $\tilde{h} \in \mathbb{F}^{(\leq 1)}[X_1,\ldots,X_{\ell^*}]$

1: $\mathcal{P}$ constructs the multilinear polynomials $\mathsf{read_ts} \in \mathbb{F}^{(\leq 1)}[X_1,\ldots,X_{\ell^*}]$ and $\mathsf{final_cts} \in \mathbb{F}^{(\leq 1)}[X_1,\ldots,X_{b/c}]$ that encode the counters of the memory accesses when reading the values $\{\tilde{h}(\boldsymbol{i}) : \boldsymbol{i} \in \{0,1\}^{\ell^*}\}$ from the memory $\{\tilde{t}_\beta(\boldsymbol{j}) : \boldsymbol{j} \in \{0,1\}^{b/c}\}$ (see Lemma 4.6), and sends $[\![\mathsf{read_ts}]\!], [\![\mathsf{final_cts}]\!]$.

$\mathcal{V}$ replies with random $\sigma, \tau \leftarrow\!\$\ \mathbb{F}$.

2: $\mathcal{P}$ constructs multilinear polynomials $f^{(1)}_{\mathsf{WS}_h}, f^{(1)}_{\mathsf{RS}}, f^{(1)}_{\mathsf{WS}_t}, f^{(1)}_{\mathsf{S}}$ such that

$$\forall \boldsymbol{i} \in \{0,1\}^{\ell^*} : f^{(1)}_\alpha(\boldsymbol{i}) = f^{(i_1)}_\alpha(\boldsymbol{i}',0)\cdot f^{(i_1)}_\alpha(\boldsymbol{i}',1) \text{ for } \alpha = \mathsf{WS}_h, \mathsf{RS}$$

$$\forall \boldsymbol{j} \in \{0,1\}^{b/c} : f^{(1)}_\alpha(\boldsymbol{j}) = f^{(j_1)}_\alpha(\boldsymbol{j}',0)\cdot f^{(j_1)}_\alpha(\boldsymbol{j}',1) \text{ for } \alpha = \mathsf{WS}_t, \mathsf{S}$$

where for a given $\boldsymbol{i} \in \{0,1\}^n$, $\boldsymbol{i}'$ denotes $(i_2,\ldots,i_n)$, and the polynomials $f^{(0)}_\alpha$ are

$$f^{(0)}_{\mathsf{WS}_h} = \tilde{h} + \sigma\cdot(\mathsf{read_ts}+1) - \tau, \quad f^{(0)}_{\mathsf{RS}} = \tilde{h} + \sigma\cdot\mathsf{read_ts} - \tau,$$

$$f^{(0)}_{\mathsf{WS}_t} = \tilde{t}_\beta - \tau, \quad f^{(0)}_{\mathsf{S}} = \tilde{t}_\beta + \sigma\cdot\mathsf{final_cts} - \tau.$$

$\mathcal{P}$ sends $[\![f^{(1)}_{\mathsf{WS}_h}]\!], [\![f^{(1)}_{\mathsf{RS}}]\!], [\![f^{(1)}_{\mathsf{WS}_t}]\!], [\![f^{(1)}_{\mathsf{S}}]\!]$.

$\mathcal{V}$ replies with random $\chi \leftarrow\!\$\ \mathbb{F}$, $\boldsymbol{\rho} \leftarrow\!\$\ \mathbb{F}^{\ell^*}, \boldsymbol{\xi} \leftarrow\!\$\ \mathbb{F}^{b/c}$.

3: $\mathcal{P}$ and $\mathcal{V}$ run sum-check PIOPs for $((0,[\![h^*]\!]); h^*) \in \mathcal{R}_{sum}$ and $((0,[\![t^*]\!]); t^*) \in \mathcal{R}_{sum}$ where $[\![h^*]\!], [\![t^*]\!]$ are the virtual oracles: // see below for the definition of g

$$[\![h^*]\!] = g_{\chi,\boldsymbol{\rho}}\left([\![\tilde{h}]\!] + \sigma\cdot([\![\mathsf{read_ts}]\!]+1) - \tau, [\![\tilde{h}]\!] + \sigma\cdot[\![\mathsf{read_ts}]\!] - \tau, [\![f^{(1)}_{\mathsf{WS}_h}]\!], [\![f^{(1)}_{\mathsf{RS}}]\!]\right),$$

$$[\![t^*]\!] = g_{\chi,\boldsymbol{\xi}}\left([\![\tilde{t}_\beta]\!] - \tau, [\![\tilde{t}_\beta]\!] + \sigma\cdot[\![\mathsf{final_cts}]\!] - \tau, [\![f^{(1)}_{\mathsf{WS}_t}]\!], [\![f^{(1)}_{\mathsf{S}}]\!]\right)$$

$\mathcal{V}$ queries $[\![f^{(1)}_{\mathsf{WS}_h}]\!], [\![f^{(1)}_{\mathsf{RS}}]\!]$ on $(\mathbf{1},0) \in \mathbb{F}^{\ell^*}$ and $[\![f^{(1)}_{\mathsf{WS}_t}]\!], [\![f^{(1)}_{\mathsf{S}}]\!]$ on $(\mathbf{1},0) \in \mathbb{F}^{b/c}$, and checks if

$$f^{(1)}_{\mathsf{WS}_t}(\mathbf{1},0)\cdot f^{(1)}_{\mathsf{WS}_h}(\mathbf{1},0) = f^{(1)}_{\mathsf{RS}}(\mathbf{1},0)\cdot f^{(1)}_{\mathsf{S}}(\mathbf{1},0)$$

Function $g_{\chi,\boldsymbol{\rho}}(f^{(0)}_1, f^{(0)}_2, f^{(1)}_1, f^{(1)}_2)$

$f_k(X_0, \boldsymbol{X}) \leftarrow (1-X_0)\cdot f^{(0)}_k(\boldsymbol{X}) + X_0\cdot f^{(1)}_k(\boldsymbol{X}), k = 1,2$

return $(f_1(1,\boldsymbol{X}) - f_1(\boldsymbol{X},0)\cdot f_1(\boldsymbol{X},1) + \chi\cdot(f_2(1,\boldsymbol{X}) - f_2(\boldsymbol{X},0)\cdot f_2(\boldsymbol{X},1)))\cdot \tilde{eq}(\boldsymbol{\rho},\boldsymbol{X})$

Fig. 1: Construction of the PIOP $\Pi^{\mathbb{F},\beta}_{range}$ for lookups in $\boldsymbol{t}_\beta$ over a finite field $\mathbb{F}$

Theorem 4.8. *Let p be a prime number and let $\mathbb{F} = \mathbb{Z}_p$. Let ℓ^*, b, c be positive integers such that $\beta = 2^{b/c} < p$ and $2^{\ell^*} < p$. Then $\Pi^{\mathbb{F},\beta}_{range}$ is a PIOP for $\mathcal{R}_{range}$ over $\mathbb{F}$, with table $\boldsymbol{t}_\beta = \{0, \ldots, \beta - 1\}$, that has perfect completeness and knowledge error*

$$\delta^{\mathbb{F}}_{range} = \frac{2^{\ell^*} + 2^{b/c} + 4\max(\ell^*, b/c) + 1}{p} = O\left(\frac{2^{\ell^*} + 2^{b/c}}{p}\right).$$

In Appendix E.2 we give the proof of the theorem and we provide a detailed efficiency analysis of $\Pi^{\mathbb{F},\beta}_{range}$.

From $\mathbb{F}$ to R_q: the PIOP Π^{β}_{range}. We now show how to build the PIOP Π^{β}_{range} for $(\boldsymbol{t}_\beta, [\![\tilde{h}]\!]; \tilde{h}) \in \mathcal{R}_{range}$, where $\tilde{h} \in R_q^{(\leq 1)}[X_1, \ldots, X_{\ell^*}]$, from a PIOP $\Pi^{\mathbb{F},\beta}_{range}$ for the same relation over a prime field $\mathbb{F} = \mathbb{Z}_p$, for a prime p that is sufficiently larger than β, following a bound that we specify below. Recall that the entries of $\boldsymbol{t}_\beta = \{0, \ldots, \beta - 1\}$ are integers, which we identify with the corresponding constant polynomials of R_q. The idea is to let the prover send a multilinear polynomial $\tilde{h}'$ over $\mathbb{F}$ whose evaluations on the hypercube are the same integers as those of $\tilde{h}$, and then to check that $\tilde{h}$ and $\tilde{h}'$ are consistent by evaluating them on random points with small integer entries, so that the two evaluations can be compared over the integers without any overflow.

Preliminaries. Let us denote $\boldsymbol{X} = (X_1, \ldots, X_{\ell^*})$. Recall that $q = p_0 \cdot p_1 \cdots p_L$ is a product of distinct primes (see Section 3.1). For a positive integer a, let $\pi_a : \mathbb{Z} \to \mathbb{Z}_a$ be the canonical quotient map $\pi_a(x) = x \bmod a$. In particular, we see π_q as a map from $\mathbb{Z}$ to $\mathbb{Z}_q \subseteq R_q$, i.e., to the constant polynomials of R_q; following this convention, for a polynomial $\tilde{h}$ over R_q the condition $\tilde{h}(\boldsymbol{i}) \in \boldsymbol{t}_\beta$ of equation (23) means $\tilde{h}(\boldsymbol{i}) \in \pi_q(\boldsymbol{t}_\beta)$. Let p be an odd prime number and $I_p = \{-\lfloor p/2 \rfloor, \ldots, \lfloor p/2 \rfloor\} \subset \mathbb{Z}$. Let π_{I_p} be the restriction of π_p to I_p, which is a bijection, and let $\iota = \pi_{I_p}^{-1} : \mathbb{Z}_p \to I_p$ be its inverse. Abusing notation, we extend the definition of these maps to vectors by applying them coordinate-wise and to polynomials by applying them on all their coefficients. For example, given a polynomial $f \in \mathbb{Z}[X_1, \ldots, X_n]$ we denote by $\pi_p(f) \in \mathbb{Z}_p[X_1, \ldots, X_n]$ the polynomial obtained by applying π_p on all the coefficients of f.

We rely on the following proposition, which intuitively formalizes the fact that $\mathbb{Z}_p$ can emulate computations over signed integers, as long as no overflow occurs.

Proposition 4.9. *Let $f \in \mathbb{Z}[X_1, \ldots, X_n]$ and $\boldsymbol{x} \in \mathbb{Z}^n$ be such that $|f(\boldsymbol{x})| < p/2$. Then $\iota(\pi_p(f)(\pi_p(\boldsymbol{x}))) = f(\boldsymbol{x})$.*

Proof. Since π_p is a ring homomorphism we have $\pi_p(f(\boldsymbol{x})) = \pi_p(f)(\pi_p(\boldsymbol{x}))$. Applying ι on both sides we obtain $\iota(\pi_p(f)(\pi_p(\boldsymbol{x}))) = \iota(\pi_p(f(\boldsymbol{x}))) = f(\boldsymbol{x})$, since $f(\boldsymbol{x}) \in I_p$ and the restriction of $\iota \circ \pi_p$ to I_p is the identity function. □

Parameters. Let p_{min} be the smallest prime factor of q, let $\theta \leq p_{min}$ be a positive integer, and let $\Theta = \{0, \ldots, \theta - 1\} \subset \mathbb{Z}$. Notice that $\pi_q(\Theta) \subseteq \mathbb{Z}_q \subseteq R_q$ is

an exceptional set of R_q of size θ. Let $0 \leq m \leq \ell^*$ be an integer that represents an efficiency tradeoff parameter, and let $\kappa \geq 1$ be an integer that represents a number of repetitions. The parameters must satisfy the following conditions:

$$p > 2\beta \cdot (2\theta)^m \tag{27}$$

$$\left(\frac{m}{\theta}\right)^\kappa \leq 2^{-\lambda} \tag{28}$$

$$2^{\ell^*-m} = \mathsf{poly}(\ell^*) \tag{29}$$

Intuitively, the first condition ensures correctness of the protocol, the second one ensures soundness, and the third one succinctness, since the prover will send in the clear 2κ multilinear polynomials in $\ell^* - m$ variables.

The protocol. We describe the PIOP Π^{β}_{range} in Figure 2. The protocol involves oracle polynomials over two different rings, R_q and $\mathbb{F} = \mathbb{Z}_p$; the definitions of Section 2.2 extend straightforwardly to this setting and, when compiling the PIOP into an argument, the oracles over $\mathbb{F}$ are realized with a multilinear polynomial commitment over the field $\mathbb{F}$. Notice also that, for an honest prover, $\tilde{y}_\eta(\boldsymbol{\delta}) \in \mathbb{Z}_q$ for all $\boldsymbol{\delta} \in \{0,1\}^{\ell^*-m}$ (see the correctness proof below), and thus each $\tilde{y}_\eta$ can be sent as a multilinear polynomial with 2^{ℓ^*-m} coefficients in $\mathbb{Z}_q$, rather than in R_q; the security analysis is unaffected by this optimization, since it only uses the fact that $\tilde{y}_\eta - \tilde{h}(\hat{\boldsymbol{r}}_\eta, \cdot)$ is a nonzero polynomial over R_q whenever $\tilde{y}_\eta$ is not the correct partial evaluation of $\tilde{h}$.

The following lemma, which motivates the bound of equation (27), is used in the analysis of the protocol.

Lemma 4.10. *Let $\tilde{h}'' \in \mathbb{Z}^{(\leq 1)}[\boldsymbol{X}]$ be such that $\tilde{h}''(\boldsymbol{b}) \in \boldsymbol{t}_\beta$ for all $\boldsymbol{b} \in \{0,1\}^{\ell^*}$, and assume that equation (27) holds. Then for all $\boldsymbol{r} \in \Theta^m$ and $\boldsymbol{\delta} \in \{0,1\}^{\ell^*-m}$ it holds $\left|\tilde{h}''(\boldsymbol{r}, \boldsymbol{\delta})\right| < \beta(2\theta)^m < p/2$.*

Proof. By the multilinearity of $\tilde{h}''$ we have that $\tilde{h}''(\boldsymbol{r}, \boldsymbol{\delta}) = \sum_{\boldsymbol{\phi} \in \{0,1\}^m} \tilde{h}''(\boldsymbol{\phi}, \boldsymbol{\delta}) \cdot \chi_{\boldsymbol{\phi}}(\boldsymbol{r})$, where $\chi_{\boldsymbol{\phi}}(\boldsymbol{r}) = \prod_{i=1}^m (\phi_i r_i + (1-\phi_i)(1-r_i))$. Since $0 \leq r_i \leq \theta - 1$, every factor of $\chi_{\boldsymbol{\phi}}(\boldsymbol{r})$ has absolute value at most θ, and thus $|\chi_{\boldsymbol{\phi}}(\boldsymbol{r})| \leq \theta^m$. Combining this with $0 \leq \tilde{h}''(\boldsymbol{\phi}, \boldsymbol{\delta}) \leq \beta - 1$ we obtain $\left|\tilde{h}''(\boldsymbol{r}, \boldsymbol{\delta})\right| \leq 2^m(\beta-1)\theta^m < \beta(2\theta)^m$, which is smaller than $p/2$ by equation (27). □

Theorem 4.11. *Let $S \subseteq R_q$ be an exceptional set and let p_{min} be the smallest prime factor of q. Let $\boldsymbol{t}_\beta = \{0, \ldots, \beta - 1\}$, let p be an odd prime, and let the parameters $\theta \leq p_{min}$, m and κ satisfy equations (27) and (28). Let $\Pi^{\mathbb{F},\beta}_{range}$ be a PIOP for the relation $\mathcal{R}_{range}$ over the finite field $\mathbb{F} = \mathbb{Z}_p$ that is perfectly complete and has knowledge error $\delta^{\mathbb{F}}_{range}$. Then the protocol Π^{β}_{range} of Figure 2 is a PIOP for $\mathcal{R}_{range}$ with perfect completeness and knowledge error*

$$\left(\frac{m}{\theta}\right)^\kappa + \frac{\ell^*-m}{|S|} + \frac{\ell^*-m}{p} + \delta^{\mathbb{F}}_{range} \;\leq\; 2^{-\lambda} + \frac{\ell^*-m}{|S|} + \frac{\ell^*-m}{p} + \delta^{\mathbb{F}}_{range}.$$

PIOP Π^{β}_{range} for $(\boldsymbol{t}_\beta, [\![\tilde{h}]\!]; \tilde{h}) \in \mathcal{R}_{range}$, with $\tilde{h} \in R_q^{(\leq 1)}[X_1, \ldots, X_{\ell^*}]$

1: $\mathcal{P}$ computes $\tilde{h}'' \in \mathbb{Z}^{(\leq 1)}[\boldsymbol{X}]$ such that, for all $\boldsymbol{b} \in \{0,1\}^{\ell^*}$, $\tilde{h}''(\boldsymbol{b}) \in \boldsymbol{t}_\beta$ and $\pi_q(\tilde{h}''(\boldsymbol{b})) = \tilde{h}(\boldsymbol{b})$, i.e., the MLE of the integer entries of $\boldsymbol{t}_\beta$ looked up by $\tilde{h}$. Then $\mathcal{P}$ computes $\tilde{h}' = \pi_p(\tilde{h}'') \in \mathbb{Z}_p^{(\leq 1)}[\boldsymbol{X}]$ and sends $[\![\tilde{h}']\!]$.

$\mathcal{V}$ samples $\boldsymbol{r}_1, \ldots, \boldsymbol{r}_\kappa \leftarrow\!\!\$\ \Theta^m$ and sends them to $\mathcal{P}$.

2: For $\eta \in [\kappa]$, let $\hat{\boldsymbol{r}}_\eta = \pi_q(\boldsymbol{r}_\eta) \in \mathbb{Z}_q^m$ and $\boldsymbol{r}'_\eta = \pi_p(\boldsymbol{r}_\eta) \in \mathbb{Z}_p^m$. $\mathcal{P}$ sends the multilinear polynomials $(\tilde{y}_1, \tilde{y}'_1, \ldots, \tilde{y}_\kappa, \tilde{y}'_\kappa)$, where for all $\eta \in [\kappa]$:

$$\tilde{y}_\eta(X_{m+1}, \ldots, X_{\ell^*}) = \tilde{h}(\hat{\boldsymbol{r}}_\eta, X_{m+1}, \ldots, X_{\ell^*}) \in R_q^{(\leq 1)}[X_{m+1}, \ldots, X_{\ell^*}],$$
$$\tilde{y}'_\eta(X_{m+1}, \ldots, X_{\ell^*}) = \tilde{h}'(\boldsymbol{r}'_\eta, X_{m+1}, \ldots, X_{\ell^*}) \in \mathbb{Z}_p^{(\leq 1)}[X_{m+1}, \ldots, X_{\ell^*}].$$

$\mathcal{V}$ samples $\hat{\boldsymbol{s}} \leftarrow\!\!\$\ S^{\ell^*-m}$ and $\boldsymbol{s}' \leftarrow\!\!\$\ \mathbb{Z}_p^{\ell^*-m}$. For all $\eta \in [\kappa]$, $\mathcal{V}$ queries $[\![\tilde{h}]\!]$ on $(\hat{\boldsymbol{r}}_\eta, \hat{\boldsymbol{s}}) \in R_q^{\ell^*}$ and $[\![\tilde{h}']\!]$ on $(\boldsymbol{r}'_\eta, \boldsymbol{s}') \in \mathbb{Z}_p^{\ell^*}$, and checks if

$$\tilde{y}_\eta(\hat{\boldsymbol{s}}) = \tilde{h}(\hat{\boldsymbol{r}}_\eta, \hat{\boldsymbol{s}}) \in R_q \quad \text{and} \quad \tilde{y}'_\eta(\boldsymbol{s}') = \tilde{h}'(\boldsymbol{r}'_\eta, \boldsymbol{s}') \in \mathbb{Z}_p.$$

Then $\mathcal{V}$ checks if, for all $\eta \in [\kappa]$, $\boldsymbol{\delta} \in \{0,1\}^{\ell^*-m}$ and $l \in \{0, \ldots, L\}$,

$$\tilde{y}_\eta(\boldsymbol{\delta}) \bmod p_l = \iota(\tilde{y}'_\eta(\boldsymbol{\delta})) \bmod p_l.$$

3: $\mathcal{P}$ and $\mathcal{V}$ run the PIOP $\Pi^{\mathbb{F},\beta}_{range}$ for $(\boldsymbol{t}'_\beta, [\![\tilde{h}']\!]; \tilde{h}') \in \mathcal{R}_{range}$ over $\mathbb{F} = \mathbb{Z}_p$, where $\boldsymbol{t}'_\beta = \pi_p(\boldsymbol{t}_\beta)$.

Fig. 2: Construction of the PIOP Π^{β}_{range} for lookups in $\boldsymbol{t}_\beta$ over R_q from the PIOP $\Pi^{\mathbb{F},\beta}_{range}$ over $\mathbb{F} = \mathbb{Z}_p$

When $\Pi^{\mathbb{F},\beta}_{range}$ is instantiated with the PIOP of Figure 1, by Theorem 4.8 the prime p should additionally satisfy $p > 2^{\ell^*}$ (whereas $p > \beta$ is already implied by equation (27)). In Appendix E.2 we give the proof of Theorem 4.11.

Remark 4.12. In a previous version of this paper, the PIOP for $(\boldsymbol{t}_\beta, [\![\tilde{h}]\!]; \tilde{h}) \in \mathcal{R}_{range}$ was identical to the current $\Pi^{\mathbb{F},\beta}_{range}$ of Figure 1, except that it was defined over R_q, with the random challenges sampled from the exceptional set $S \subseteq R_q$. As observed in [WZZ+26] and communicated to us by Yuanju Wei, such a protocol only ensured independent range checks on the CRT components of $\tilde{h}(\boldsymbol{i})$: writing R_q as a product of finite fields, what it guaranteed is that the projection of $\tilde{h}(\boldsymbol{i})$ in each of these fields lies in $\boldsymbol{t}_\beta$. Even though, since $\beta < p_{min}$, the table $\boldsymbol{t}_\beta$ is the same in every component, these independent checks do not imply that $\tilde{h}(\boldsymbol{i})$ itself lies in $\boldsymbol{t}_\beta$.[8] In this version, we fix this flaw in two steps. First, we instantiate the lookup PIOP over a prime field $\mathbb{F} = \mathbb{Z}_p$, on an auxiliary polynomial $\tilde{h}'$ sent by the prover; this is the protocol $\Pi^{\mathbb{F},\beta}_{range}$. Second, we give a protocol ensuring that, for every $\boldsymbol{i}$, the evaluations $\tilde{h}'(\boldsymbol{i}) \in \mathbb{F}$ and $\tilde{h}(\boldsymbol{i}) \in R_q$ are the same over the integers; this is the protocol Π^{β}_{range} of Figure 2. We also notice that an alternative solution for Π^{β}_{range} could be through the techniques of [WZZ+26], which we leave for future exploration.

[8] For instance, $6 \in \mathbb{Z}_{15}$ is not in $\boldsymbol{t}_2$ whereas its CRT components in $\mathbb{Z}_3$ and $\mathbb{Z}_5$ are in $\boldsymbol{t}_2$.

Efficiency of Π_{range}. The efficiency of the PIOP Π_{range} is that of Π^{β}_{range} (instantiated with $\ell^* = \ell+\nu+\gamma$), i.e., the protocol of Figure 2 that internally invokes $\Pi^{\mathbb{F},\beta}_{range}$ on $\tilde{h}'$, with the additional costs stemming from Π_{decomp}. To summarize:

- the number of rounds is $\mathsf{r} = \max(\ell + \nu + \gamma, b/c) + 2$;
- the prover sends $\mathsf{o} = 8$ oracles: $[\![\tilde{h}]\!]$ over R_q, and $[\![\tilde{h}']\!]$ and the 6 oracles of $\Pi^{\mathbb{F},\beta}_{range}$ over $\mathbb{F}$; the total size s of the oracles is $2^{\ell+\nu+\gamma}$ elements of R_q and $2^{\ell+\nu+\gamma+2} + 3 \cdot 2^{b/c}$ elements of $\mathbb{F}$; in addition, the prover sends $\kappa \cdot 2^{\ell+\nu+\gamma-m}$ elements of $\mathbb{Z}_q$ and as many elements of $\mathbb{F}$;
- the proving time is $\mathsf{T}_{\mathcal{P}} = O(\kappa \cdot 2^{\ell+\nu+\gamma} + 2^{b/c})$;
- the verifier time is $\mathsf{T}_{\mathcal{V}} = O(\ell + \nu + \gamma + b/c + \kappa \cdot 2^{\ell+\nu+\gamma-m})$, where the last term is $\mathsf{poly}(\ell + \nu + \gamma)$ by equation (29);
- the number of verifier's queries is $\mathsf{q} = 25 + 2\kappa$, which includes the 22 queries of $\Pi^{\mathbb{F},\beta}_{range}$, the κ queries to $[\![\tilde{h}]\!]$ and the κ queries to $[\![\tilde{h}']\!]$ of Π^{β}_{range}, plus the three queries $\tilde{v}(\boldsymbol{r}_i)$, $\tilde{h}(\boldsymbol{r}_i, \boldsymbol{r}_j, \boldsymbol{r}_k)$ and $\tilde{p}_{\beta,N}(\boldsymbol{r}_j, \boldsymbol{r}_k)$ in Π_{decomp}.[9]

Arbitrary bounds The PIOP Π_{range} assumes the bound B to be a power of two. To model arbitrary integer bounds, we can use standard techniques from literature, see e.g. [CCs08], to reduce them to the one with a power-of-two bound. Let $2^{u-1} < B < 2^u$, then for an integer x:

$$x \in [0, B) \quad \iff x \in [0, 2^u) \ \wedge \ x - B + 2^u \in [0, 2^u).$$

Thus a statement $(T_B, [\![\tilde{v}]\!]; \tilde{v}) \in \mathcal{R}_{range}$ for a non-power-of-two bound B is equivalent to checking $(T_{2^u}, [\![\tilde{v}]\!]; \tilde{v}) \in \mathcal{R}_{range} \wedge (T_{2^u}, [\![\tilde{v}]\!] - B + 2^u; \tilde{v}) \in \mathcal{R}_{range}$.

5 Multilinear Polynomial Commitments for CKKS Rings

To compile our PIOP into an argument system (cf. Section 2.2), we need a polynomial commitment for multilinear polynomials in $R_q[X_1, \ldots, X_\ell]$, and we are not aware of solutions that work natively with this type of rings. In this section, we first observe that we can use the decomposition of R_q as product of fields (see Sec. 3) to reduce the problem to designing polynomial commitments for multilinear polynomials over finite fields. Most existing PC schemes in the literature work either for specific prime fields [KZG10], or for finite fields with a large number of roots of unity [BBHR18]. However, none of these categories match the fields yielded by our decomposition of R_q. For this reason, we resort to a polynomial commitment scheme described in Brakedown [GLS+23] and implicit in [BCG20]. These are transparent-setup commitments which only require error correcting codes with large enough relative minimum distance, and they only require the underlying finite field to be large, but we do not strictly need additional properties such as having many 2^k-th roots of unity.

[9] The oracle $[\![\tilde{p}_{\beta,N}]\!]$ is part of the index.

Polynomial commitments for $(R_q)_{\leq 1}[X_1, \ldots, X_\ell]$ from polynomial commitments over finite fields. Since R_q is isomorphic to a direct product of finite fields $R_q \simeq \mathbb{F}^{(1)} \times \cdots \times \mathbb{F}^{(\mu)}$ as detailed in Section 3, we can obtain a polynomial commitment scheme PC for $(R_q)_{\leq 1}[X_1, \ldots, X_\ell]$ from a family of polynomial commitment schemes $\mathsf{PC}^{(i)}$ for $\mathbb{F}^{(i)}_{\leq 1}[X_1, \ldots, X_\ell]$, $i \in [\mu]$. To alleviate notation we denote $\boldsymbol{X} = (X_1, \ldots, X_\ell)$. Let $\Phi : R_q \to \mathbb{F}^{(1)} \times \cdots \times \mathbb{F}^{(\mu)}$ be a ring isomorphism and $\Phi^{(i)} : R_q \to \mathbb{F}^{(i)}$ its projection to the i-th component for $i \in [\mu]$. These notations extend naturally to $\Phi : R_q[\boldsymbol{X}] \to \mathbb{F}^{(1)}[\boldsymbol{X}] \times \cdots \times \mathbb{F}^{(\mu)}[\boldsymbol{X}]$ and $\Phi^{(i)} : R_q[\boldsymbol{X}] \to \mathbb{F}^{(i)}[\boldsymbol{X}]$. Note that f is multilinear if and only if for all $i \in [\mu]$, $\Phi^{(i)}(f)$ is multilinear; moreover Φ commutes with polynomial evaluation: given $\boldsymbol{a} \in R_q^\ell$, we have $\Phi(f(\boldsymbol{a})) = (f^{(1)}(\boldsymbol{a}^{(1)}), \ldots, f^{(\mu)}(\boldsymbol{a}^{(\mu)}))$ where $f^{(i)} = \Phi^{(i)}(f)$ and $\boldsymbol{a}^{(i)} = \Phi^{(i)}(\boldsymbol{a})$ (here $\Phi^{(i)}$ is applied componentwise to $\boldsymbol{a}$).

Given a polynomial f, the prover defines its commitment to be the vector of commitments to each $f^{(i)} = \Phi^{(i)}(f)$ using $\mathsf{PC}^{(i)}$. Later, to prove $f(\boldsymbol{a}) = y$, the prover shows $f^{(i)}(\boldsymbol{a}^{(i)}) = y^{(i)}$ using the evaluation proof procedure in $\mathsf{PC}^{(i)}$, where $\boldsymbol{a}^{(i)} = \Phi^{(i)}(\boldsymbol{a})$ and $y^{(i)} = \Phi^{(i)}(y)$. We give the formal description in Appendix F. It is immediate that if all $\mathsf{PC}^{(i)}$ are complete (respectively binding, knowledge sound) then PC is complete (respectively binding, knowledge sound).

The polynomial commitments for $\mathbb{F}^{(i)}_{\leq 1}[X_1, \ldots, X_\ell]$ We sum up here the polynomial commitment construction described in [GLS+23] and implicit in [BCG20]. It applies to multilinear polynomials in $\mathbb{F}[X_1, \ldots, X_\ell]$ where $\mathbb{F}$ is an arbitrary (large enough) finite field. We only describe the version with proof size and verifier time $O(2^{\ell/2})$ (i.e. square root in the input size 2^ℓ) but note this is generalized in the aforementioned papers to a construction where both complexities are $O(2^{\ell/t})$. Moreover, we assume ℓ is even and define $m = 2^{\ell/2}$. Then $f \in \mathbb{F}_{\leq 1}[X_1, \ldots, X_\ell]$ is characterized by the m^2 evaluations $\{f(\boldsymbol{b}) : \boldsymbol{b} \in \{0,1\}^\ell\}$, and we can arrange these in a square matrix $U_f \in \mathbb{F}^{m \times m}$ such that the following holds: for any $\boldsymbol{a} \in \mathbb{F}^\ell$, there exist $\boldsymbol{q}_1 \in \mathbb{F}^m, \boldsymbol{q}_2 \in \mathbb{F}^m$ with $f(\boldsymbol{a}) = \langle \boldsymbol{q}_1 \cdot U_f, \boldsymbol{q}_2 \rangle$.[10] The commitment scheme makes use of a linear error-correcting code C of length M, dimension m, and minimum distance γM, with some encoding function $\mathsf{Enc} : \mathbb{F}^m \to C \subseteq \mathbb{F}^M$.

In the committing phase, the prover encodes each row of U_f with Enc resulting in $\hat{U}_f \in \mathbb{F}^{m \times M}$ and then commits to $\hat{U}_f$ with a Merkle tree. In fact, since later the prover needs to open full columns, we can take the leaves of the Merkle tree to be each of the columns of the matrix.

Given $\boldsymbol{a}$ and y for which the prover claims $f(\boldsymbol{a}) = y$, the prover and verifier engage in a two-phase protocol. In the testing phase, the prover shows the consistency of the commitment by opening a random linear combination $\boldsymbol{v} = \boldsymbol{r} \cdot U$ of the rows of U_f, and a random set of columns Q (of a pre-agreed size $|Q| = \mathfrak{t}$) of

[10] Concretely, given $\boldsymbol{a} \in \mathbb{F}^\ell$, let $\boldsymbol{q}_1 = \bigotimes_{i=1}^{\ell/2}(a_i, 1-a_i)$ and $\boldsymbol{q}_2 = \bigotimes_{i=1}^{\ell/2}(a_{\ell/2+i}, 1-a_{\ell_2+i})$, with $\otimes$ denoting the Kronecker product. If $\boldsymbol{a} \in \{0,1\}^\ell$, both $\boldsymbol{q}_1$ and $\boldsymbol{q}_2$ are unit vectors. The (i,j)-th position of U_f is then $f(\boldsymbol{b})$ for the unique $\boldsymbol{b} \in \{0,1\}^\ell$ such that the $\boldsymbol{q}_1$, $\boldsymbol{q}_2$ associated to that $\boldsymbol{b}$ are respectively the i-th and j-th unit vectors.

$\hat{U}_f$ (resulting in a submatrix $\hat{U}_f^Q$) where $\boldsymbol{r}$ and Q are queried by the verifier. Now the verifier checks that $\mathsf{Enc}(\boldsymbol{v})|_Q = \boldsymbol{r} \cdot \hat{U}_f^Q$. In the evaluation phase the prover wants to show that $\langle \boldsymbol{q}_1 \cdot U_f, \boldsymbol{q}_2 \rangle = y$. The prover sends a claimed $\boldsymbol{v}' = \boldsymbol{q}_1 \cdot U_f$ and proves its correctness similarly to the testing phase, i.e. opening a random subset Q' of columns of $\hat{U}_f$ of size $\mathfrak{t}$. The verifier checks $\mathsf{Enc}(\boldsymbol{v}')|_{Q'} = \boldsymbol{q}_1 \cdot \hat{U}_f^{Q'}$. and $\langle \boldsymbol{v}', \boldsymbol{q}_2 \rangle = y$. See Appendix F for a more formal description of this scheme.

Proposition 5.1 (from [GLS+23]). *Let γ be the relative minimum distance of the error correcting code C (i.e. its minimum distance is γN) if the testing phase passes with probability at least $N/|\mathbb{F}| + (1-\gamma/3)^{\mathfrak{t}}$, then there exists a multilinear polynomial $f' \in \mathbb{F}_{\leq 1}[X_1, \ldots, X_\ell]$ such that on query $(\boldsymbol{a}, y)$ either $f'(\boldsymbol{a}) = y$ or the verifier rejects the evaluation phase with probability at least $1 - (1 - 2\gamma/3)^{\mathfrak{t}}$.*

Choosing Enc *in practice* In our case our fields are of the form $\mathbb{F} = \mathbb{F}_{p^4}$ for p of 49 bits. Brakedown [GLS+23] introduced a code based on expander graphs that is linear-time encodable and does not require a large set of roots of unity in the field. On the other hand, if we do have a large set of 2^k-th roots of unity, and furthermore this set is contained in the base field $\mathbb{Z}_p$, then Reed-Solomon codes (which lead to a construction implicit in Ligero [AHIV17]) are a preferable choice for Enc, as in this case encoding is fast and we can get much larger relative minimum distance γ than in the expander-graph solution, and hence the size of the proof and verifier times are much smaller [11]. For our parameters, we have a "moderately large" set containing exactly $N/2$ roots of unity in $\mathbb{F}_p$, but depending on the application it may be that $m \geq N/2$. In that case, we suggest to use a piecewise Reed-Solomon code, where we split the input message in $4m/N$ blocks of length $N/4$ and encode each block independently with a $[N/2, N/4, N/4+1]$-RS code (more details in Appendix F). The resulting code has rate $1/2$ and relative minimum distance $\gamma = N/8m = N/2^{\frac{\ell}{2}+3}$. For example, for $\ell = 2^{26}, N = 2^{14}$, this leads to $\gamma = 1/4$; achieving soundness error 2^{-128} then requires opening $\mathfrak{t} = 1020$ columns, as opposed to 6593 in the parameter set analyzed in Brakedown, leading to roughly $6\times$ shorter evaluation proofs.

6 Practical performance

The efficiency analyses of Sections 4.2 and 4.3 allow one to characterize performance for our construction (summarized in Table 2) and establish its asymptotic advantages over alternative approaches. Notwithstanding, assessing concrete efficiency requires additional experimental data, as we rely on new proof components as well as custom instantiations of existing ones. Considering this, we implement and benchmark all basic building blocks required by our construction. Our goal is not to provide a full solution, which we believe should be tailored to application-specific needs, but to demonstrate the efficiency of each subcomponent of our proposal and, ultimately, show that its practicality boils down

[11] in fact, for Reed-Solomon codes there is a refined analysis [BCI+20, GLS+23] that improves the soundness error from Proposition 5.1 to $N/|\mathbb{F}| + (1-\gamma)^{\mathfrak{t}}$, which further reduces $\mathfrak{t}$ for a given security parameter

	Maximum size	Sum-checks	Commitments	Eval proofs
Π_{AC}	$WD(L+8)$	5	$3+D$	$4+D$
Π_{range}	$Nc(W(D+1)(L-D/2)+2DW)$	$3D$	$8D$	$(25+2\kappa)D$

Table 2: Summary of costs for our construction for a circuit of width W and depth D, with κ repetitions in Π^{β}_{range} (Figure 2). Maximum size refers to the input size, in number of R_{q_0} elements, of all sum-check and commitments.

almost entirely to the choice of some basic and well-studied proof tools, such as polynomial commitment schemes, for which we present a candidate solution.

We experiment with two sets of HE parameters, $(N = 2^{13}, L = 3)$ and $(N = 2^{14}, L = 6)$, extracted from applications using CKKS and similar schemes. These parameters allow for several choices of prime size, which should be made according to the noise and secret key distributions for security. For generality, we benchmark with 49-bit primes, which is the largest size allowing for optimal performance on IFMA-based polynomial arithmetic implementations [BKS+21]. We note our results can be easily extrapolated to other parameter sets with the same dimension N and number of RNS components L. We have made the source code available at `https://github.com/vfhe/proof-friendly-CKKS`. We run all experiments on an `m7i.metal-48xl` instance on AWS (Intel Xeon Platinum 8488C at 3.2GHz and 768GB memory) using at most 48 threads for multithreaded experiments.

6.1 Proof-friendly CKKS

The first component of our construction is our proof-friendly version of CKKS (Section 3), which works over a single ring R_{q_0} that splits into extension fields of configurable degree d. We implement full RNS arithmetic for it using incomplete NTTs [LS19] evaluated as d independent executions of negacyclic NTTs of dimension N/d, for which, in turn, we implement using HEXL [BKS+21]. We consider $d \in \{2, 4\}$ and perform quadratic-time polynomial multiplication in the non-splittable components (since d is small). Using this arithmetic backend, we implement our proof-friendly CKKS in Python and compare it with CKKS in HELib [HS14] instantiated with similar parameters (same ring dimension, number of RNS components, and modulus size). We also compare it with our own implementation of "regular" CKKS using HEXL directly (without our special ring). Table 3 shows the results. The difference in performance with HELib is explained by implementation factors unrelated to our modifications[12]. Compared to HEXL, our proof-friendly CKKS multiplication is at most 20% more expensive while the performance of RNS polynomial multiplication deteriorates linearly with the degree of the field (as expected for our quadratic-time multiplications).

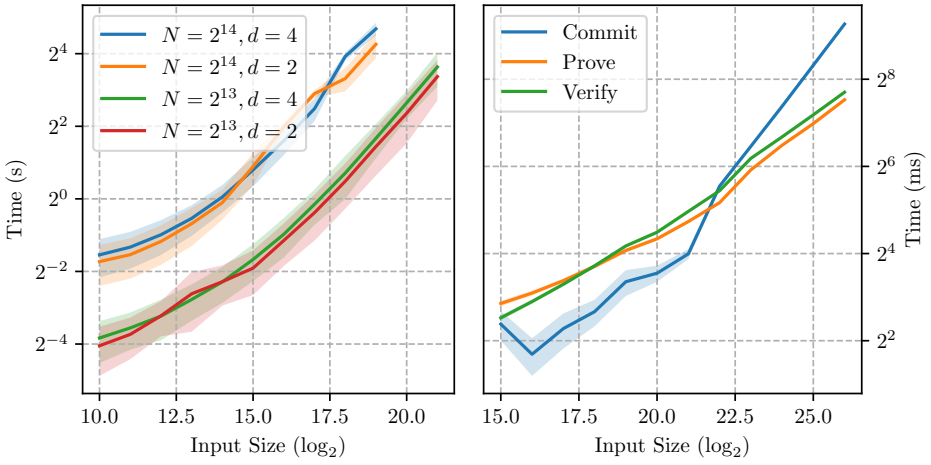
6.2 Proof and verification performance

The consistency check circuits for rescalings and base decompositions, introduced in Section 4.1, are duals of the respective CKKS computations, hence presenting

[12] We consider the standard *"package build"* of HELib.

	$N, L = 8192, 3$				$N, L = 16384, 6$			
	Proof-friendly CKKS		CKKS		Proof-friendly CKKS		CKKS	
	$d = 2$	$d = 4$	HEXL	HELib	$d = 2$	$d = 4$	HEXL	HELib
NTT	0.032	0.030	0.035	-	0.147	0.127	0.210	-
Poly Mul	0.029	0.054	0.013	-	0.140	0.250	0.080	-
CKKS Mul	1.107	1.205	1.023	18.316	7.394	8.457	7.197	48.407

Table 3: Performance of the proof-friendly version of CKKS. Time in milliseconds for a single-threaded execution.



(a) Sum-check (b) Brakedown (single component)

Fig. 3: Performance of our sum-check and Brakedown [GLS+23] polynomial commitments. Time for a multi-threaded execution with up to 48 threads. Brakedown commitment time is per R_{q_0} component.

performance similar as already reported in Table 3. Once the circuit is evaluated, proving it consists just of sequences of sum-check protocol executions and various uses of the polynomial commitment. For the former, we take the linear-time sum-check protocol from Libra [XZZ+19]. We implement it over the same ring defined for our proof-friendly CKKS and benchmark it for inputs of size 2^{10} to 2^{21}. Figure 3a presents the results. For the polynomial commitment scheme, our main choice is Brakedown [GLS+23], for which we benchmark both the original implementation and specific optimizations enabled by our choice of parameters. Figure 3b presents the results for the original implementation. We measure commit, prove, and verification times considering only arithmetic in the base field $\mathbb{F}_p$, since extension fields are not supported by the original implementation.

Improvements to Brakedown. A single Brakedown commitment runs in less than a second for all of our parameters, but that is only for a single component of R_{q_0}. As discussed in Section 3.1, R_{q_0} splits in LN/d components, thus requiring LN/d Brakedown commitments, which would take between 1 minute (for input size 2^{15}, $N = 2^{13}$, and $L = 3$) and 16 hours (for input size 2^{26}, $N = 2^{14}$, and

$L = 6$), based on the results of Figure 3b. We improve these results by proposing different choices of encoding, as discussed in Section 5. Particularly, Brakedown relies on expander-graph codes to be field-agnostic, whereas our choice of field enables the efficient computation of Reed-Solomon codes of moderate size. Considering that, we implemented a version of Brakedown's encoding using an optimized implementation of RS codes. For our largest parameter set, we could reduce encoding time to just 8.9 minutes, an improvement of two orders of magnitude over [GLS+23]. Table 4 in Appendix G presents the complete results. This improvement is a consequence not only of replacing the encoding but also of employing a significantly more optimized implementation. Addressing whether proof and verification could also be accelerated to this extent is not as straightforward and diving into specific optimizations for Brakedown would fall outside our scope, especially when using it is ultimately an option within our proposal. On the other hand, we note that proof and verification only require procedures that are asymptotically much cheaper than commitments while Brakedown implements them at the same level of optimization. This could indicate that similar improvements might be possible, but we leave such assessment as future work.

6.3 Application-level estimates

Remark 6.1. Note that the experiments-based efficiency estimates of the range proofs provided below consider the PIOP Π_{range} presented in a previous version of the paper (see Remark 4.12) and have not been updated with the revised version of Π^{β}_{range}. For experimental results reflecting the current state of this line of work, we refer to [CCC+26] which introduces several improvements over the constructions presented here.

Putting together the performance summary of Table 2 with the experimental results presented in this section, we can estimate concrete costs for our construction and compare it with previous literature. Let us take, for example, the "Medium circuit" from [VKH23], which evaluates a noise flooding operation with RNS parameters ($N = 2^{13}, L = 3$), the same we consider in our experiments[13]. This circuit is composed of a single ciphertext-ciphertext multiplication, followed by mod-switching, and a noise flooding realized via a linear combination of $O(\lambda)$ ciphertexts with binary coefficients. Fixing the size of the lookup table to 20-bit, the circuit for mod-switching and multiplication yields $(W, D, L, N, c) = (1, 1, 3, 8192, 5)$. For our experiments, we use $d = 4$. Linear combinations have a negligible cost in our approach, but, for completeness, we take $\lambda = 128$ and add it the maximum size of Π_{AC}. With these values, we have that Π_{AC} has maximum size $11 + 128 = 139$ while Π_{range} has maximum size 286720. It is clear that the cost is dominated by the latter whereas the former is negligible. From Figure 3, we have that, for this maximum size, each sum-check takes 4 seconds while each R_{q_0} commitment takes 4.9 minutes in the original

[13] [VKH23] uses the BGV scheme instead of CKKS, but ciphertext computations should be similar and our framework could also capture BGV.

implementation of Brakedown, or just 1.9 seconds in our optimized RS encoding (see Table 4 for details). Supposing all sum-checks and commitments of Π_{range} are at the maximum size, which is a very loose upper bound, this would result in 13.3s for all commitments and 12s for all sum-checks. Proof and verification are asymptotically much cheaper than commitments and sum-checks, as discussed in Section 6.2[14]. For reference, [VKH23] reports a minimum of 443 seconds to prove this circuit with Rinocchio and 83 minutes with Aurora. Adjusting for differences in implementation and execution environments, this should put the performance of both works at the same order of magnitude. Our major advantage, however, is that our estimate includes the cost for proving key switchings, which are not supported in [VKH23]. Concretely, we could prove two (or many) consecutive multiplications at costs of similar order whereas performance in [VKH23] would degrade exponentially since ciphertexts cannot be re-linearized.

It should be noted that this estimate includes the costs of all main procedures required by our construction, but does not account for secondary factors like data representation and other implementation challenges that may arise when implementing a complete framework. Examining these implementation aspects is a necessary, nontrivial, step that we leave for future work. As such, we limit our experiments and proof of concept implementation to the goal of providing enough evidence of practical improvements over previous literature, and, in this regard, [VKH23] is currently the most practical solution for RNS schemes.

Acknowledgments

We would like to thank Yuanju Wei for reporting a flaw in an earlier version of the PIOP Π^{β}_{range} in Section 4.3, which has been fixed and allowed us to improve the presentation of our result with more precise claims.

This work is supported by the PICOCRYPT project that has received funding from the European Research Council (ERC) under the European Unions Horizon 2020 research and innovation programme (Grant agreement No. 101001283), partially supported by projects PRODIGY (TED2021-132464B- I00) and ESPADA (PID2022-142290OB-I00) funded by MCIN/AEI/10.13039/501100011033/. This work is part of the grant JDC2023-050791-I, funded by MCIN/AEI/10.13039/501100011033 and the ESF+. This work is also supported by the Smart Networks and Services Joint Undertaking (SNS JU) under the European Union's Horizon Europe research and innovation programme in the scope of the CONFIDENTIAL6G project under Grant Agreement 101096435. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Commission. Neither the European Union nor the European Commission can be held responsible for them

[14] Estimating time for proof and verifications would also require an optimized implementation of Brakedown for our specific fields, which we leave as future work, as discussed in the last paragraph of Section 6.2.

References

ABPS24. Shahla Atapoor, Karim Baghery, Hilder V. L. Pereira, and Jannik Spiessens. Verifiable FHE via lattice-based snarks. *IACR Commun. Cryptol.*, 1(1):24, 2024.

ACGS23. Diego F. Aranha, Anamaria Costache, Antonio Guimarães, and Eduardo Soria-Vazquez. HELIOPOLIS: Verifiable computation over homomorphically encrypted data from interactive oracle proofs is practical. Cryptology ePrint Archive, Paper 2023/1949, 2023.

AHIV17. Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. Ligero: Lightweight sublinear arguments without a trusted setup. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017: 24th Conference on Computer and Communications Security*, pages 2087–2104, Dallas, TX, USA, October 31 – November 2, 2017. ACM Press.

AHKS22. Amin Abdulrahman, Vincent Hwang, Matthias J. Kannwischer, and Amber Sprenkels. Faster Kyber and Dilithium on the Cortex-M4. In Giuseppe Ateniese and Daniele Venturi, editors, *Applied Cryptography and Network Security*, pages 853–871, Cham, 2022. Springer International Publishing.

BBHR18. Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast reed-solomon interactive oracle proofs of proximity. In Ioannis Chatzigiannakis, Christos Kaklamanis, Dániel Marx, and Donald Sannella, editors, *ICALP 2018: 45th International Colloquium on Automata, Languages and Programming*, volume 107 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 14:1–14:17, Prague, Czech Republic, July 9–13, 2018. Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik.

BCC+22. Youngjin Bae, Jung Hee Cheon, Wonhee Cho, Jaehyung Kim, and Taekyung Kim. META-BTS: Bootstrapping precision beyond the limit. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022: 29th Conference on Computer and Communications Security*, pages 223–234, Los Angeles, CA, USA, November 7–11, 2022. ACM Press.

BCFK21. Alexandre Bois, Ignacio Cascudo, Dario Fiore, and Dongwoo Kim. Flexible and efficient verifiable computation on encrypted data. In Juan Garay, editor, *PKC 2021: 24th International Conference on Theory and Practice of Public Key Cryptography, Part II*, volume 12711 of *Lecture Notes in Computer Science*, pages 528–558, Virtual Event, May 10–13, 2021. Springer, Cham, Switzerland.

BCG20. Jonathan Bootle, Alessandro Chiesa, and Jens Groth. Linear-time arguments with sublinear verification from tensor codes. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020: 18th Theory of Cryptography Conference, Part II*, volume 12551 of *Lecture Notes in Computer Science*, pages 19–46, Durham, NC, USA, November 16–19, 2020. Springer, Cham, Switzerland.

BCI+20. Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. Proximity gaps for reed-solomon codes. In *61st Annual Symposium on Foundations of Computer Science*, pages 900–909, Durham, NC, USA, November 16–19, 2020. IEEE Computer Society Press.

BCS16. Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive oracle proofs. In Martin Hirt and Adam D. Smith, editors, *TCC 2016-B: 14th Theory of Cryptography Conference, Part II*, volume 9986 of *Lecture Notes*

in Computer Science, pages 31–60, Beijing, China, October 31 – November 3, 2016. Springer Berlin Heidelberg, Germany.

BFS20. Benedikt Bünz, Ben Fisch, and Alan Szepieniec. Transparent SNARKs from DARK compilers. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020, Part I*, volume 12105 of *Lecture Notes in Computer Science*, pages 677–706, Zagreb, Croatia, May 10–14, 2020. Springer, Cham, Switzerland.

BGV12. Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (Leveled) fully homomorphic encryption without bootstrapping. In Shafi Goldwasser, editor, *ITCS 2012: 3rd Innovations in Theoretical Computer Science*, pages 309–325, Cambridge, MA, USA, January 8–10, 2012. Association for Computing Machinery.

BKS+21. Fabian Boemer, Sejun Kim, Gelila Seifu, Fillipe D.M. de Souza, and Vinodh Gopal. Intel HEXL: Accelerating Homomorphic Encryption with Intel AVX512-IFMA52. In *Proceedings of the 9th on Workshop on Encrypted Computing Applied Homomorphic Cryptography*, WAHC '21, pages 57–62, New York, NY, USA, 2021. Association for Computing Machinery.

BMTH21. Jean-Philippe Bossuat, Christian Mouchet, Juan Ramón Troncoso-Pastoriza, and Jean-Pierre Hubaux. Efficient bootstrapping for approximate homomorphic encryption with non-sparse keys. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part I*, volume 12696 of *Lecture Notes in Computer Science*, pages 587–617, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland.

Bra12. Zvika Brakerski. Fully homomorphic encryption without modulus switching from classical GapSVP. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology – CRYPTO 2012*, volume 7417 of *Lecture Notes in Computer Science*, pages 868–886, Santa Barbara, CA, USA, August 19–23, 2012. Springer Berlin Heidelberg, Germany.

BV11. Zvika Brakerski and Vinod Vaikuntanathan. Fully homomorphic encryption from ring-LWE and security for key dependent messages. In Phillip Rogaway, editor, *Advances in Cryptology – CRYPTO 2011*, volume 6841 of *Lecture Notes in Computer Science*, pages 505–524, Santa Barbara, CA, USA, August 14–18, 2011. Springer Berlin Heidelberg, Germany.

CBBZ23. Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. HyperPlonk: Plonk with linear-time prover and high-degree custom gates. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023, Part II*, volume 14005 of *Lecture Notes in Computer Science*, pages 499–530, Lyon, France, April 23–27, 2023. Springer, Cham, Switzerland.

CCC+26. Ignacio Cascudo, Anamaria Costache, Daniele Cozzo, Dario Fiore, Antonio GuimarÃ£es, and Eduardo Soria-Vazquez. Batch, pack, and prove: More efficient verifiable computation for ckks. In *ASIACRYPT 2026*, 2026.

CCH+24. Anamaria Costache, Benjamin R. Curtis, Erin Hales, Sean Murphy, Tabitha Ogilvie, and Rachel Player. On the precision loss in approximate homomorphic encryption. In Claude Carlet, Kalikinkar Mandal, and Vincent Rijmen, editors, *SAC 2023: 30th Annual International Workshop on Selected Areas in Cryptography*, volume 14201 of *Lecture Notes in Computer Science*, pages 325–345, Fredericton, Canada, August 14-18, 2024. Springer, Cham, Switzerland.

CCKP19. Shuo Chen, Jung Hee Cheon, Dongwoo Kim, and Daejun Park. Verifiable computing for approximate computation. Cryptology ePrint Archive, Report 2019/762, 2019.

CCs08. Jan Camenisch, Rafik Chaabouni, and abhi shelat. Efficient protocols for set membership and range proofs. In Josef Pieprzyk, editor, *Advances in Cryptology – ASIACRYPT 2008*, volume 5350 of *Lecture Notes in Computer Science*, pages 234–252, Melbourne, Australia, December 7–11, 2008. Springer Berlin Heidelberg, Germany.

CCS19. Hao Chen, Ilaria Chillotti, and Yongsoo Song. Improved bootstrapping for approximate homomorphic encryption. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2019, Part II*, volume 11477 of *Lecture Notes in Computer Science*, pages 34–54, Darmstadt, Germany, May 19–23, 2019. Springer, Cham, Switzerland.

CGGI16. Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *Advances in Cryptology – ASIACRYPT 2016, Part I*, volume 10031 of *Lecture Notes in Computer Science*, pages 3–33, Hanoi, Vietnam, December 4–8, 2016. Springer Berlin Heidelberg, Germany.

CHK+18. Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. Bootstrapping for approximate homomorphic encryption. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018, Part I*, volume 10820 of *Lecture Notes in Computer Science*, pages 360–384, Tel Aviv, Israel, April 29 – May 3, 2018. Springer, Cham, Switzerland.

CHK+19. Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. A full RNS variant of approximate homomorphic encryption. In Carlos Cid and Michael J.: Jacobson, Jr., editors, *SAC 2018: 25th Annual International Workshop on Selected Areas in Cryptography*, volume 11349 of *Lecture Notes in Computer Science*, pages 347–368, Calgary, AB, Canada, August 15–17, 2019. Springer, Cham, Switzerland.

CHM+20. Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Psi Vesely, and Nicholas P. Ward. Marlin: Preprocessing zkSNARKs with universal and updatable SRS. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020, Part I*, volume 12105 of *Lecture Notes in Computer Science*, pages 738–768, Zagreb, Croatia, May 10–14, 2020. Springer, Cham, Switzerland.

CKKS17. Jung Hee Cheon, Andrey Kim, Miran Kim, and Yong Soo Song. Homomorphic encryption for arithmetic of approximate numbers. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology – ASIACRYPT 2017, Part I*, volume 10624 of *Lecture Notes in Computer Science*, pages 409–437, Hong Kong, China, December 3–7, 2017. Springer, Cham, Switzerland.

CKP+24. Sylvain Chatel, Christian Knabenhans, Apostolos Pyrgelis, Carmela Troncoso, and Jean-Pierre Hubaux. VERITAS: Plaintext encoders for practical verifiable homomorphic encryption. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024: 31st Conference on Computer and Communications Security*, pages 2520–2534, Salt Lake City, UT, USA, October 14–18, 2024. ACM Press.

FGP14. Dario Fiore, Rosario Gennaro, and Valerio Pastro. Efficiently verifiable computation on encrypted data. In Gail-Joon Ahn, Moti Yung, and Ninghui Li,

editors, *ACM CCS 2014: 21st Conference on Computer and Communications Security*, pages 844–855, Scottsdale, AZ, USA, November 3–7, 2014. ACM Press.

FV12. Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Report 2012/144, 2012.

GBK+24. Mariana Gama, Emad Heydari Beni, Jiayi Kang, Jannik Spiessens, and Frederik Vercauteren. Blind zkSNARKs for private proof delegation and verifiable computation over encrypted data. Cryptology ePrint Archive, Report 2024/1684, 2024.

GGP10. Rosario Gennaro, Craig Gentry, and Bryan Parno. Non-interactive verifiable computing: Outsourcing computation to untrusted workers. In Tal Rabin, editor, *Advances in Cryptology – CRYPTO 2010*, volume 6223 of *Lecture Notes in Computer Science*, pages 465–482, Santa Barbara, CA, USA, August 15–19, 2010. Springer Berlin Heidelberg, Germany.

GGW24. Sanjam Garg, Aarushi Goel, and Mingyuan Wang. How to prove statements obliviously? In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024, Part X*, volume 14929 of *Lecture Notes in Computer Science*, pages 449–487, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland.

GKR08. Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. Delegating computation: interactive proofs for muggles. In Richard E. Ladner and Cynthia Dwork, editors, *40th Annual ACM Symposium on Theory of Computing*, pages 113–122, Victoria, BC, Canada, May 17–20, 2008. ACM Press.

GKR15. Shafi Goldwasser, Yael Tauman Kalai, and Guy N Rothblum. Delegating computation: interactive proofs for muggles. *Journal of the ACM (JACM)*, 62(4):1–64, 2015.

GLS+23. Alexander Golovnev, Jonathan Lee, Srinath T. V. Setty, Justin Thaler, and Riad S. Wahby. Brakedown: Linear-time and field-agnostic SNARKs for R1CS. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part II*, volume 14082 of *Lecture Notes in Computer Science*, pages 193–226, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.

GNS23. Chaya Ganesh, Anca Nitulescu, and Eduardo Soria-Vazquez. Rinocchio: SNARKs for ring arithmetic. *Journal of Cryptology*, 36(4):41, October 2023.

GSW13. Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology – CRYPTO 2013, Part I*, volume 8042 of *Lecture Notes in Computer Science*, pages 75–92, Santa Barbara, CA, USA, August 18–22, 2013. Springer Berlin Heidelberg, Germany.

HK20. Kyoohyung Han and Dohyeong Ki. Better bootstrapping for approximate homomorphic encryption. In Stanislaw Jarecki, editor, *Topics in Cryptology – CT-RSA 2020*, volume 12006 of *Lecture Notes in Computer Science*, pages 364–390, San Francisco, CA, USA, February 24–28, 2020. Springer, Cham, Switzerland.

HS14. Shai Halevi and Victor Shoup. Algorithms in HElib. In Juan A. Garay and Rosario Gennaro, editors, *Advances in Cryptology – CRYPTO 2014, Part I*, volume 8616 of *Lecture Notes in Computer Science*, pages 554–571, Santa Barbara, CA, USA, August 17–21, 2014. Springer Berlin Heidelberg, Germany.

KPP22. Andrey Kim, Antonis Papadimitriou, and Yuriy Polyakov. Approximate homomorphic encryption with reduced approximation error. In Steven D. Galbraith, editor, *Topics in Cryptology – CT-RSA 2022*, volume 13161 of *Lecture Notes in Computer Science*, pages 120–144, Virtual Event, March 1–2, 2022. Springer, Cham, Switzerland.

KRS25. Dmitry Khovratovich, Ron D. Rothblum, and Lev Soukhanov. How to prove false statements: Practical attacks on fiat-shamir. Cryptology ePrint Archive, Paper 2025/118, 2025.

KVH24. Christian Knabenhans, Alexander Viand, and Anwar Hithnawi. Towards robust fhe for the real world. Real World Crypto 2024, 2024.

KZG10. Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In Masayuki Abe, editor, *Advances in Cryptology – ASIACRYPT 2010*, volume 6477 of *Lecture Notes in Computer Science*, pages 177–194, Singapore, December 5–9, 2010. Springer Berlin Heidelberg, Germany.

LFKN90. Carsten Lund, Lance Fortnow, Howard J. Karloff, and Noam Nisan. Algebraic methods for interactive proof systems. In *31st Annual Symposium on Foundations of Computer Science*, pages 2–10, St. Louis, MO, USA, October 22–24, 1990. IEEE Computer Society Press.

LS19. Vadim Lyubashevsky and Gregor Seiler. NTTRU: Truly fast NTRU using NTT. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2019(3):180–201, 2019.

LXZ21. Tianyi Liu, Xiang Xie, and Yupeng Zhang. zkCNN: Zero knowledge proofs for convolutional neural network predictions and accuracy. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021: 28th Conference on Computer and Communications Security*, pages 2968–2985, Virtual Event, Republic of Korea, November 15–19, 2021. ACM Press.

Set20. Srinath Setty. Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020, Part III*, volume 12172 of *Lecture Notes in Computer Science*, pages 704–737, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Cham, Switzerland.

SL20. Srinath Setty and Jonathan Lee. Quarks: Quadruple-efficient transparent zkSNARKs. Cryptology ePrint Archive, Report 2020/1275, 2020.

STW24. Srinath T. V. Setty, Justin Thaler, and Riad S. Wahby. Unlocking the lookup singularity with Lasso. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024, Part VI*, volume 14656 of *Lecture Notes in Computer Science*, pages 180–209, Zurich, Switzerland, May 26–30, 2024. Springer, Cham, Switzerland.

TS24. Tolun Tosun and Erkay Savas. Zero-Value Filtering for Accelerating Non-Profiled Side-Channel Attack on Incomplete NTT-Based Implementations of Lattice-Based Cryptography. *IEEE Transactions on Information Forensics and Security*, 19:3353–3365, 2024. Conference Name: IEEE Transactions on Information Forensics and Security.

VKH23. Alexander Viand, Christian Knabenhans, and Anwar Hithnawi. Verifiable fully homomorphic encryption, 2023.

WZZ+26. Yuanju Wei, Zhelei Zhou, Xinxuan Zhang, Songyu Wu, Binwu Xiang, Cheng Hong, Yi Deng, and Xiaofeng Chen. GlueLUT: Efficient lookup table arguments over residue rings. Cryptology ePrint Archive, Paper 2026/494, 2026.

XZZ+19. Tiancheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, and Dawn Song. Libra: Succinct zero-knowledge proofs with optimal prover computation. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part III*, volume 11694 of *Lecture Notes in Computer Science*, pages 733–764, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Cham, Switzerland.

ZLW+21. Jiaheng Zhang, Tianyi Liu, Weijie Wang, Yinuo Zhang, Dawn Song, Xiang Xie, and Yupeng Zhang. Doubly efficient interactive proofs for general arithmetic circuits with linear prover time. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021: 28th Conference on Computer and Communications Security*, pages 159–177, Virtual Event, Republic of Korea, November 15–19, 2021. ACM Press.

A Homomorphic Encryption

Definition A.1. *A public-key Homomorphic Encryption scheme* HE *over a set of admissible circuits* C *consists of the following algorithms.*

- $(\mathsf{pp}, C) \leftarrow \mathsf{HE.Setup}(1^\lambda, \mathcal{M}, \mathsf{C})$: *Given a message space* $\mathcal{M}$*, a set of admissible circuits* C*, output the public parameters* pp *and the ciphertext space* $\mathcal{C}$ *such that the scheme is semantically secure.*
- $(\mathsf{sk}, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathsf{HE.KeyGen}(1^\lambda, \mathcal{C}, \mathsf{pp})$: *Given the public parameters* pp *and the ciphertext space* $\mathcal{C}$*, output the secret key* sk*, the public key* pk *and the evaluation key* evk*.*
- $\mathsf{ct} \leftarrow \mathsf{HE.Enc}(\mathsf{pk}, m)$: *Given a message* $m \in \mathcal{M}$*, output its encryption* ct*.*
- $m' \leftarrow \mathsf{HE.Dec}(\mathsf{sk}, \mathsf{ct})$: *Given a ciphertext* ct *and its corresponding secret key* sk*, output the decryption of* ct*,* m'*.*
- $\mathsf{ct}' \leftarrow \mathsf{HE.Eval}(\mathsf{evk}, \mathsf{ct}, C)$: *Given an admissible circuit* $C \in \mathsf{C}$*, such that* $C : \mathcal{M} \to \mathcal{M}$*, output a ciphertext* ct' *corresponding to an encryption of* $C(m)$*.*

When the context is clear, we will omit specifying the $\mathsf{sk}, \mathsf{pk}, \mathsf{pp}, \mathcal{C}$ parameters.

Remark. In the definition of the Eval algorithm above, we have written $C : \mathcal{M} \to \mathcal{M}$ for simplicity. We note that the definition can be trivially extended to any (admissible) circuit $C : \mathcal{M}^\kappa \to \mathcal{M}^{\kappa'}$, for any $\kappa, \kappa' \in \mathbb{N}$.

Definition A.2. *(Semantic Security) Let* $\mathsf{HE} = (\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{Eval})$ *be a (public-key) homomorphic encryption scheme as defined above, let* $(\mathsf{pp}, \mathcal{C}) \leftarrow \mathsf{Setup}(1^\lambda, \mathcal{M}, \mathsf{C})$*, and let* $\mathcal{A}$ *be an adversary. The advantage of* $\mathcal{A}$ *with respect to* HE *is defined as follows, with* $(\mathsf{sk}, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathsf{KeyGen}(1^\lambda, \mathcal{C}, \mathsf{pp})$*:*

$$\begin{aligned}\mathsf{Adv}^{\mathsf{he}}_{\mathsf{Adv}}(\lambda) := \big| &\Pr[\mathsf{Adv}(\mathsf{pk}, \mathsf{ct}) = 1 : \mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{pk}, 1)] \\ &- \Pr[\mathsf{Adv}(\mathsf{pk}, \mathsf{ct}) = 1 : \mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{pk}, 0)]\big|.\end{aligned}$$

HE *is semantically secure if* $\mathsf{Adv}^{\mathsf{he}}_{\mathsf{Adv}}(\lambda) = \mathsf{negl}(\lambda)$ *for every PPT adversary* $\mathcal{A}$*.*

A.1 CKKS

Let q, q' be two ciphertext moduli, let P be the "special modulus" used in key-switching and let χ be the error distribution, and S be the secret-key one. We note that we are presenting the CKKS scheme as in the original paper [CKKS17], as opposed to other versions, such as the RNS one [CHK+19]. Note in particular that we omit the specifics of levels – this is on purpose, since algorithms (for example Enc are typically presented "at the top level", but can be defined at any level).

$\mathsf{sk} \leftarrow \mathsf{SecretKeyGen}(\lambda)$**:** Sample $s \leftarrow S$ and output $\mathsf{sk} = (1, s)$.

$\mathsf{pk} \leftarrow \mathsf{PublicKeyGen}(\mathsf{sk})$**:** Parse $\mathsf{sk} = (1, s)$ and sample $a \leftarrow R_q$ uniformly at random and $e \leftarrow \chi$. Output $\mathsf{pk} = ([-as + e]_q, a)$.

$\mathsf{evk} \leftarrow \mathsf{EvaluationKeyGen}(\mathsf{sk}, \mathsf{sk}')$**:** Parse $\mathsf{sk} = (1, s)$, $\mathsf{sk}' = (1, s')$. Note that for applying $\mathsf{KeySwitch}$ after a multiplication, $s' = s^2$. Sample $a' \leftarrow R_{Pq}$ uniformly at random and $e' \leftarrow \chi$. Output $\mathsf{evk} = ([-a's + e' + Ps']_{Pq}, a')$.

$\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{pk}, m)$**:** For a message $m \in R$. Let $\mathsf{pk} = (p_0, p_1)$, sample $v \leftarrow S$ and $e_1, e_2 \leftarrow \chi$. Output $\mathsf{ct} = ([m + p_0 v + e_1]_q, [p_1 v + e_2]_q)$.

$m \leftarrow \mathsf{Dec}(\mathsf{sk}, \mathsf{ct})$**:** Let $s = \mathsf{sk}$ and $\mathsf{ct} = (c_0, c_1)$. Output $m = [c_0 + c_1 s]_q$.

$\mathsf{ct} \leftarrow \mathsf{Add}(\mathsf{ct}_0, \mathsf{ct}_1)$**:** Output $\mathsf{ct} = ([\mathsf{ct}_0[0] + \mathsf{ct}_1[0]]_q, [\mathsf{ct}_0[1] + \mathsf{ct}_1[1]]_q)$.

$\mathsf{ct}' \leftarrow \mathsf{SAdd}(\mathsf{ct}, \alpha)$**:** Given a level l, a scalar $\alpha \in R_{q_l}$ for some level l, compute $\mathsf{ct}' \coloneqq (c_0 + \alpha, c_1) \pmod{q_l}$. Return ct'.

$\mathsf{ct}' \leftarrow \mathsf{SMult}(\mathsf{ct}, \alpha)$**:** Given a level l, a scalar $\alpha \in R_{q_l}$ for some level l, compute $\mathsf{ct}' \coloneqq (c_0 \cdot \alpha, c_1 \cdot \alpha) \pmod{q_l}$. Return ct'.

$\mathsf{ct} \leftarrow \mathsf{PreMult}(\mathsf{ct}_0, \mathsf{ct}_1)$**:** Set $d_0 = [\mathsf{ct}_0[0]\mathsf{ct}_1[0]]_q$, $d_1 = [\mathsf{ct}_0[0]\mathsf{ct}_1[1] + \mathsf{ct}_0[1]\mathsf{ct}_1[0]]_q$, and $d_2 = [\mathsf{ct}_0[1]\mathsf{ct}_1[1]]_q$. Output $\mathsf{ct} = (d_0, d_1, d_2)$.

$\mathsf{ct} \leftarrow \mathsf{Rotate}(\mathsf{ct}', a)$**:** For $a \in \mathbb{Z}_{2N}^*$, and $\mathsf{ct}' = (\mathsf{ct}'_0, \mathsf{ct}'_1)$ a ciphertext encrypted under $\mathsf{sk}' = (1, s)$, apply the automorphism $\mathsf{ct}' \mapsto \mathsf{ct} = (\mathsf{ct}'_0(X^a), \mathsf{ct}'_1(X^a))$ and $\mathsf{sk} = (1, s(X^a))$. Then, compute $\mathsf{evk} = \mathsf{EvaluationKeyGen}(\mathsf{sk}, \mathsf{sk}')$ and output $\mathsf{ct} = \mathsf{KeySwitch}(\mathsf{ct}', \mathsf{evk})$.

$\mathsf{ct}' \leftarrow \mathsf{KeySwitch}(\mathsf{ct}, \mathsf{evk})$**:** Let $\mathsf{ct}[0] = d_0$, $\mathsf{ct}[1] = d_1$ and $\mathsf{ct}[2] = d_2$. Let $\mathsf{evk}[0] = -a's + e' + Ps^2$ and $\mathsf{evk}[1] = a'$. Set $c'_0 = [d_0 + \lfloor P^{-1} \cdot d_2 \cdot (-a's + e' + Ps^2) \rceil]_q$, and $c'_1 = [d_1 + \lfloor P^{-1} \cdot d_2 \cdot a' \rceil]_q$. Output $\mathsf{ct}' = (c'_0, c'_1)$.

$\mathsf{ct}'' \leftarrow \mathsf{Rescale}(\mathsf{ct}', q')$ **:** Let $\mathsf{ct}' = (c'_0, c'_1)$. Set $c''_0 = \left[\left\lfloor \frac{q'}{q} c'_0 \right\rceil\right]_{q'}$ and $c''_1 = \left[\left\lfloor \frac{q'}{q} c'_1 \right\rceil\right]_{q'}$. Output $\mathsf{ct}'' = (c''_0, c''_1)$.

$\mathsf{ct} \leftarrow \mathsf{Mult}(\mathsf{ct}_0, \mathsf{ct}_1, \mathsf{evk}, q')$**:** Output $\mathsf{ct}' = \mathsf{Rescale}(\mathsf{KeySwitch}(\mathsf{PreMult}(\mathsf{ct}_0, \mathsf{ct}_1), \mathsf{evk}), q')$.

B Noise analysis for proof-friendly CKKS

Definition B.1. *Let* $\mathsf{ct} \in R_q^2$ *be a CKKS ciphertext and* sk *its associated secret key. Recall we can write*

$$\mathsf{Dec}(\mathsf{ct}, \mathsf{sk}) = \langle \mathsf{ct}, \mathsf{sk} \rangle = m + e_{\mathsf{ct}} \in R_q.$$

We refer to e_{ct} *as the* noise *of the ciphertext.*

Before our noise analysis, we recall the following two facts. These can be found, for example in the original CKKS paper [CKKS17].

- Let ct be the output of the Key-Switching algorithm. Then, we have that

$$\langle \mathsf{ct}, \mathsf{sk} \rangle = m + e_{\mathsf{ct}} + e_{\mathsf{KS}}.$$

- Let ct be the output of the $\mathsf{scale}(\mathsf{ct}', p)$ algorithm. Then, we have that

$$\langle \mathsf{ct}, \mathsf{sk} \rangle = m/p + e_{\mathsf{ct}}/p + e_{\mathsf{scale}}.$$

Note in particular that we use e_{ct} to denote the noise present in the ciphertext before the procedure applied to it. The exact values and/ or distributions of e_{KS} and e_{scale} depends on the variant of the CKKS scheme that is used, see for example [CCH+24, KPP22]. We stress that we do not focus on the exact value and/ or distribution of the noise, but rather on how much *additional* noise our modified procedures incur.

Lemma B.2. *Let* ct *be the output of our* $\mathsf{scale}(\mathsf{ct}', p)$ *algorithm, and let* $\mathsf{sk} = (1, s)$ *be its corresponding secret key. Then, we have that*

$$\langle \mathsf{ct}, \mathsf{sk} \rangle = m/p + (e_{\mathsf{ct}} + \epsilon)/p + e_{\mathsf{scale}},$$

where $\epsilon = \epsilon_0 + s \cdot \epsilon_1$*, and* $\|\epsilon_i\|_\infty = u \cdot p$*, for some polynomial* u *with coefficients in* $\{-1, 0, 1\}$ *and for* $i \in \{0, 1\}$*.*

Proof. Recall that the scale procedure proceeds as:

$$\left(c'_i - [c'_i]_p\right) p^{-1},$$

where $i \in \{0, 1\}$ and we have written $\mathsf{ct} = (c_0, c_1)$.

Now, recall that the prover is rather sending a value $\left\|c'_{i,M}\right\|_\infty \leq M$, instead of the value $[c'_i]_p$. Write

$$c'_{i,M} = [c'_i]_p + \epsilon_i,$$

where $\epsilon_i = u \cdot p$, for u a polynomial with coefficients in $\{-1, 0, 1\}$. Then, we can rewrite the procedure as:

$$\begin{aligned}\left(c'_i - c'_{i,M}\right) p^{-1} &= \left(c'_i - ([c'_i]_p + \epsilon_i)\right) p^{-1} \\ &= \left(c'_i - [c'_i]_p\right) p^{-1} + \epsilon_i \cdot p^{-1}.\end{aligned}$$

The statement follows. □

Lemma B.3. *Let* ct *be the output of our key-switching procedure. Then, we have that*

$$\langle \mathsf{ct}, \mathsf{sk} \rangle = m + e_{\mathsf{ct}} + e_{\mathsf{KS}} + \epsilon',$$

where $\epsilon' = \epsilon^{(0)} + s \cdot \epsilon^{(1)}$*, and* $\left\|\epsilon^{(i)}\right\|_\infty \leq \|e_{\mathsf{KS}}\|_\infty$*.*

Proof. Recall that, as shown in Equation 5, the key switching procedure computes:

$$\begin{aligned}\tilde{c}_0 &:= d_0 + \langle \mathsf{CRT}_{\omega_l}^{-1}(d_2), \mathsf{evk}_{l,0} \rangle \\ \tilde{c}_1 &:= d_1 + \langle \mathsf{CRT}_{\omega_l}^{-1}(d_2), \mathsf{evk}_{l,1} \rangle.\end{aligned}$$

Recall first of all that

$$\mathsf{CRT}_{\omega_l}^{-1}(d_2) = \big([d_2]_{p_0}, \ldots, [d_2]_{p_l}\big).$$

Now, further recall that the prover does not (necessarily) provide $\mathsf{CRT}_{\omega_l}^{-1}(d_2)$, but rather some vector of l values $\widehat{d_{2,M}}$ that approximates $\mathsf{CRT}_{\omega_l}^{-1}(d_2)$. As previously, we can write, for all $i \in \{0, \ldots, l\}$

$$\widehat{d_{2,M}}[i] = [d_2]_{p_i} + \epsilon_i,$$

where $\|\epsilon_i\|_\infty \leq p_i$. It follows that, for $j \in \{0, 1\}$,

$$\begin{aligned}
\tilde{c_j'} &= \langle \widehat{\mathsf{CRT}_{\omega_l}^{-1}}(d_2), \mathsf{ek}_{l,j} \rangle \\
&= \langle \big([d_2]_{p_0} + \epsilon_0, \ldots, [d_2]_{p_l} + \epsilon_l, \big), \mathsf{ek}_{l,j} \rangle \\
&= \tilde{c_j} + \langle \big(\epsilon_0, \ldots, \epsilon_l\big), \mathsf{ek}_{l,j} \rangle.
\end{aligned}$$

Write $\epsilon^{(j)} = \langle \big(\epsilon_0, \ldots, \epsilon_l\big), \mathsf{ek}_{l,j} \rangle$. Then, we have that

$$\tilde{c_j'} = \tilde{c_j} + \epsilon^{(j)}.$$

Note in particular that, since both $\|[d_2]_{p_i}\|_\infty \leq p_i$ and $\|\epsilon_i\|_\infty \leq p_i$, it follows that $\epsilon^{(j)} \leq e_{\mathsf{KS}}$. The result follows. □

C Verifiable computation

We give the definition of verifiable computation (VC) scheme. We do so by following the original definition of Gennaro, Gentry and Parno [GGP10] with two differences. The first one is that we consider the publicly delegatable setting in which a client's input can be encoded using only the public key (as opposed to using the secret key as in [GGP10]). The second one is a generalization to capture non-deterministic computations. Namely, the delegation scenario that we consider is the following. The server is supposed to compute functions $f(x, u)$ that take two arguments: an input x that is provided by the delegator but is unknown to the server, and an input u that is known to the server but (possibly) not to the client. For example, $f(x, u)$ can represent a polynomial evaluation where x is the point of the client, $u = (u_0, \ldots, u_n)$ is a vector of coefficients and $f(x, u) = \sum_{i=0}^{n} u_i x^i$. Another example is machine learning classification where x is the sample to be classified, u is the model and $f(x, u)$ implements the classifier algorithm. In our VC syntax we assume that the non-deterministic input is encoded using an algorithm $\mathsf{WitCom}(\mathsf{pk}, u)$ that produces an encoding wen_u and a verification key vk_u. Looking ahead to our construction, vk_u can be a commitment of u and wen_u its opening. In terms of security, it is possible to consider two possible scenarios for vk_u: either it is computed by a party trusted by the verifier, or it is computed by the server (and thus it can be adversarial). Since the well-formedness of vk_u is application-dependent, we will assume that the encoding wen_u and vk_u are certified and provided to the client by a trusted party.

Definition C.1 (Verifiable Computation). *A verifiable computation scheme* VC *is a tuple of efficient algorithms* $(\mathsf{KeyGen}, \mathsf{WitCom}, \mathsf{InputEnc}, \mathsf{Compute}, \mathsf{VerDec})$ *defined as follows:*

$\mathsf{KeyGen}(1^\lambda, f) \to (\mathsf{sk}, \mathsf{pk})$ *: a probabilistic algorithm that given the security parameter* λ *and a function* f*, returns a key pair* $(\mathsf{pk}, \mathsf{sk})$*.*

$\mathsf{WitCom}(\mathsf{pk}, u) \to (\mathsf{wen}_u, \mathsf{vk}_u)$ *: a probabilistic algorithm that encodes a non-deterministic input* u *producing a value* wen_u *that is provided to the server and a verification key* vk_u *that is given to the client.*

$\mathsf{InputEnc}(\mathsf{pk}, x) \to (\mathsf{en}_x, \mathsf{vk}_x)$ *: a probabilistic algorithm that encodes a client's input* x *as public key value* en_x *that is provided to the server and a verification key* vk_x *that is given to the client.*

$\mathsf{Compute}(\mathsf{pk}, \mathsf{en}_x, \mathsf{wen}_u) \to \mathsf{en}_y$ *: a probabilistic algorithm which is run by the server on input the public key* pk*, and the encodings of a client's input* en_x *and non-deterministic input* wen_u*. The algorithm generates an output encoding* en_y*.*

$\mathsf{VerDec}(\mathsf{sk}, \mathsf{vk}_x, \mathsf{vk}_u, \mathsf{en}_y) \to (\mathsf{acc}, y)$ *: a deterministic algorithm which is run by the client on input the secret key* sk*, verification keys* vk_x *and* vk_u *for the inputs, and an output encoding* en_y*. The algorithm returns an acceptance bit* acc *and a value* y*.*

Correctness. The basic property that a VC scheme should achieve is correctness, which intuitively means that whenever client and server run honestly all the algorithms for f, x, u, then the client obtains a σ_y that successfully decodes to $y = f(x, u)$.

Definition C.2 (Correctness). *We say that a VC scheme* VC *is* correct *if for any function* f *and inputs* x, u*:*

$$\Pr\left[\begin{array}{l|ll} & (\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{Setup}(1^\lambda, f) \\ & (\mathsf{wen}_u, \mathsf{vk}_i) \leftarrow \mathsf{WitCom}(\mathsf{pk}, u) \\ acc = 1 \ \wedge \ y = f(x, w) & : \ (\mathsf{en}_x, \mathsf{vk}_x) \leftarrow \mathsf{InputEnc}(\mathsf{pk}, x) \\ & \mathsf{en}_y \leftarrow \mathsf{Compute}(\mathsf{pk}, \mathsf{en}_x, \mathsf{wen}_u) \\ & (acc, y) \leftarrow \mathsf{VerDec}(\mathsf{sk}, \mathsf{vk}_x, \mathsf{vk}_u, \mathsf{en}_y) \end{array}\right] = 1.$$

Succinctness. Intuitively, a VC scheme is succinct if the time required to run $\mathsf{InputEnc}$ (for delegating an input) and VerDec (to verify and decode an output) is asymptotically faster than that required to run f.

Definition C.3 (Succinctness). *A VC scheme* VC *is succinct if there exists a fixed polynomial* $p(\cdot)$ *such that for any function* $f(x, y)$ *running in time at most* T *we have that:* $\mathsf{InputEnc}(\mathsf{pk}, x)$ *runs in time at most* $p(\lambda, |x|)$*; any honestly computed* $\mathsf{en}_y \leftarrow \mathsf{Compute}(\mathsf{pk}, \mathsf{en}_x, \mathsf{wen}_u)$ *has size* $|\mathsf{en}_y| \leq p(\lambda, o(T), |y|)$*; for honestly computed* $\mathsf{en}_x, \mathsf{wen}_u, \mathsf{en}_y$*, the running time of* $\mathsf{VerDec}(\mathsf{sk}, \mathsf{en}_x, \mathsf{wen}_u, \mathsf{en}_y)$ *is at most* $p(\lambda, o(T), |y|)$*.*

IND-CPA security. Intuitively, a VC scheme is IND-CPA-secure if the server does not learn anything about the client's inputs. We also allow the adversary to access to an oracle $\mathcal{O}_{\mathsf{VerDec}}(\cdot)$ which is a verification oracle that the adversary can use to test an (adversarially generated) output en_y, returning only the acceptance bit of VerDec. This is formalized via the following experiment

Definition C.4. *A VC scheme* VC *is IND-CPA-secure if for any PPT adversary* Adv *and any function* f*, it holds* $\mathsf{Adv}^{\text{IND-CPA}}_{\mathsf{VC},\mathsf{Adv}}(\lambda) \leq \mathsf{negl}(\lambda)$*, where*

$$\mathsf{Adv}^{\text{IND-CPA}}_{\mathsf{VC},\mathsf{Adv}}(\lambda) = \Pr\left[b = b' \quad : \begin{array}{l} (\mathsf{pk},\mathsf{sk}) \leftarrow \mathsf{Setup}(1^\lambda, f); b \leftarrow\!\$\ \{0,1\} \\ (x_0, x_1, st) \leftarrow \mathsf{Adv}^{\mathcal{O}_{\mathsf{VerDec}}}(\mathsf{pk}) \\ (\mathsf{en}^*, \mathsf{vk}^*) \leftarrow \mathsf{InputEnc}(\mathsf{pk}, x_b) \\ b' \leftarrow \mathsf{Adv}^{\mathcal{O}_{\mathsf{VerDec}}}(st, \mathsf{pk}, \mathsf{en}^*, \mathsf{vk}^*) \end{array}\right] - \frac{1}{2}.$$

Soundness. Intuitively, soundness guarantees that a malicious server should not be able to cheat the client into accepting an output which is not the result of the computation that it was supposed to do. We formalize this notion via the following experiment in which the adversary (modeling a malicious server) receives the public key pk and has access to three oracles: $\mathcal{O}_{\mathsf{InputEnc}}(\cdot)$ that is used to obtain encoded client's inputs; $\mathcal{O}_{\mathsf{WitCom}}(\cdot)$ that is used to obtain encoded server's inputs; $\mathcal{O}_{\mathsf{VerDec}}(\cdot)$ which is a verification oracle that the adversary can use to test an adversarially generated output en_y with respect to verification keys generated by the previous other oracles. The goal of the adversary is to produce an encoded output that successfully decodes to a wrong result.

Experiment $\mathsf{SND}_{\mathsf{VC},\mathsf{Adv}}(\lambda, f)$

$\mathsf{X}, \mathsf{U} \leftarrow \emptyset; \mathsf{cnt}_\mathsf{X}, \mathsf{cnt}_\mathsf{U} \leftarrow 0$

$(\mathsf{pk},\mathsf{sk}) \leftarrow \mathsf{Setup}(1^\lambda, f)$

$(\mathsf{en}_y, i, j) \leftarrow \mathsf{Adv}^{\mathcal{O}_{\mathsf{WitCom}}(\cdot),\mathcal{O}_{\mathsf{InputEnc}}(\cdot),\mathcal{O}_{\mathsf{VerDec}}(\cdot,\cdot,\cdot)}(\mathsf{pk})$

$(x_i, \mathsf{vk}_i) \leftarrow (\mathsf{X}[i][1], \mathsf{X}[i][3]); (u_j, \mathsf{vk}_j) \leftarrow (\mathsf{U}[j][1], \mathsf{U}[j][3])$

$(\mathsf{acc}, y) \leftarrow \mathsf{VerDec}(\mathsf{sk}, \mathsf{vk}_i, \mathsf{vk}_j, \mathsf{en}_y)$

if $\mathsf{acc} = 1$ and $y \neq f(x_i, u_j)$ **return** 1 **else return** 0

Oracle $\mathcal{O}_{\mathsf{VerDec}}(i, j, \mathsf{en}_y)$

$(acc, y) \leftarrow \mathsf{VerDec}(\mathsf{pk}, \mathsf{X}[i][3], \mathsf{U}[j][3])$

return acc

Oracle $\mathcal{O}_{\mathsf{WitCom}}(u)$

$\mathsf{cnt}_\mathsf{U} \leftarrow \mathsf{cnt}_\mathsf{U} + 1$

$(\mathsf{wen}_u, \mathsf{vk}_u) \leftarrow \mathsf{WitCom}(\mathsf{pk}, u)$

$\mathsf{U}[\mathsf{cnt}_\mathsf{U}] \leftarrow (u, \mathsf{wen}_u, \mathsf{vk}_u)$

return $(\mathsf{wen}_u, \mathsf{vk}_u)$

Oracle $\mathcal{O}_{\mathsf{InputEnc}}(x)$

$\mathsf{cnt}_\mathsf{X} \leftarrow \mathsf{cnt}_\mathsf{X} + 1$

$(\mathsf{en}_x, \mathsf{vk}_x) \leftarrow \mathsf{InputEnc}(\mathsf{pk}, x)$

$\mathsf{X}[\mathsf{cnt}_\mathsf{X}] \leftarrow (x, \mathsf{en}_x, \mathsf{vk}_x)$

return en_x

Definition C.5 (Soundness). *A VC scheme* VC *is sound if for any function* f *and any PPT adversary* Adv *we have*

$$\mathsf{Adv}^{\mathsf{snd}}_{\mathsf{VC},\mathsf{Adv}}(\lambda) = \Pr[\mathsf{SND}_{\mathsf{VC},\mathsf{Adv}}(\lambda, f) = 1] \leq \mathsf{negl}(\lambda).$$

D Additional preliminaries

D.1 SNARKs

We recall the definition of succinct non-interactive arguments of knowledge (SNARKs).

Definition D.1. *A SNARK for an indexed relation $\mathcal{R}$ is a tuple of algorithms $\Pi := (\mathsf{Setup}, \mathsf{Index}, \mathsf{Prove}, \mathsf{Verify})$ defined as follows:*

$\mathsf{Setup}(1^\lambda) \to (\mathsf{srs}, \mathsf{vk})$ *takes the security parameter λ and returns a structured reference* srs *and a verification key* vk.

$\mathsf{Index}(\mathsf{srs}, \mathsf{vk}, \mathtt{i}) \to (\mathsf{pk}_\mathtt{i}, \mathsf{vk}_\mathtt{i})$ *is a deterministic algorithm that takes an index $\mathtt{i}$ and outputs a proving key $\mathsf{pk}_\mathtt{i}$ and a verification key $\mathsf{vk}_\mathtt{i}$.*

$\mathsf{Prove}(\mathsf{pk}_\mathtt{i}, \mathtt{x}, \mathtt{w}) \to \pi$ *takes the prover key $\mathsf{pk}_\mathtt{i}$, a statement-witness pair $(\mathtt{x}, \mathtt{w})$ such that $(\mathtt{i}, \mathtt{x}, \mathtt{w}) \in \mathcal{R}$, and outputs a proof π.*

$\mathsf{Verify}(\mathsf{vk}_\mathtt{i}, \mathtt{x}, \pi) \to 0/1$ *takes the verification key $\mathsf{vk}_\mathtt{i}$, a statement $\mathtt{x}$ and proof π and returns a boolean value.*

Correctness: *For any $(\mathtt{i}, \mathtt{x}, \mathtt{w}) \in \mathcal{R}$*

$$\Pr\left[\mathsf{Verify}(\mathsf{vk}_\mathtt{i}, \mathtt{x}, \pi) = 1 \;\middle|\; \begin{array}{l}(\mathsf{srs}, \mathsf{vk}) \leftarrow \mathsf{Setup}(1^\lambda),\\ (\mathsf{pk}_\mathtt{i}, \mathsf{vk}_\mathtt{i}) \leftarrow \mathsf{Index}(\mathsf{srs}, \mathsf{vk}, \mathtt{i})\\ \pi \leftarrow \mathsf{Prove}(\mathsf{pk}_\mathtt{i}, \mathtt{x}, \mathtt{w})\end{array}\right] = 1$$

Knowledge soundness: *For any PPT adversary* Adv *there is an efficient extractor* Ext *(running on the same input of* Adv*, including its random coins) such that*

$$\Pr\left[\begin{array}{l}\mathsf{Verify}(\mathsf{vk}_\mathtt{i}, \mathtt{x}, \pi) = 1\\ \wedge\ \ (\mathtt{i}, \mathtt{x}, \mathtt{w}) \notin \mathcal{R}\end{array} \;\middle|\; \begin{array}{l}(\mathsf{srs}, \mathsf{vk}) \leftarrow \mathsf{Setup}(1^\lambda),\\ (\mathtt{i}, \mathtt{x}, \pi), \mathtt{w}) \leftarrow \mathsf{Adv}\|\mathsf{Ext}(\mathsf{srs})\\ (\mathsf{pk}_\mathtt{i}, \mathsf{vk}_\mathtt{i}) \leftarrow \mathsf{Index}(\mathsf{srs}, \mathsf{vk}, \mathtt{i})\end{array}\right] = \mathsf{negl}(\lambda)$$

Succinctness: *The complexity of* Verify *is* $\mathrm{poly}(\lambda + |\mathtt{x}| + f(|\mathtt{w}|))$ *and the proof size is* $\mathrm{poly}(\lambda + f(|\mathtt{w}|))$ *for a sublinear function $f(\cdot) = o(\cdot)$, e.g., $\sqrt{\cdot}$ or $\log(\cdot)$.*

D.2 Multilinear Polynomial Commitments for Rings

Definition D.2 (Multilinear Polynomial Commitment). *A polynomial commitment scheme for multilinear polynomials over a ring R is a tuple* $\mathsf{PC} = (\mathsf{Setup}, \mathsf{Com}, \mathsf{VerCom}, , \mathsf{ProveEval}, \mathsf{VerEval})$

$\mathsf{Setup}(1^\lambda, \ell) \to (\mathsf{ck}, \mathsf{vk})$ *takes the security parameter and (possibly) an upper bound on the number of variables ℓ, and outputs a commitment key* ck *and a verification key* vk.

$\mathsf{Com}(\mathsf{ck}, f) \to (\mathsf{cm}_f, \mathsf{opn}_f)$ *takes a multilinear polynomial $f \in R^{(\leq 1)}[X_1, \ldots, X_\ell]$, and outputs a commitment cm_f and an opening opn_f.*

$\mathsf{VerCom}(\mathsf{vk}, \mathsf{cm}_f, f, \mathsf{opn}_f) \to b$ *checks if* opn_f *is a valid opening for the commitment* cm_f *to a polynomial* f*, by accepting* ($b = 1$) *or not* ($b = 0$).

$\langle \mathsf{ProveEval}(\mathsf{ck}, \mathsf{cm}_f, \boldsymbol{a}, y, \ell; f, \mathsf{opn}_f), \mathsf{VerEval}(\mathsf{vk}, \mathsf{cm}_f, \boldsymbol{a}, y, \ell) \rangle \to b$ *is a protocol in which the prover* ProveEval *aims to convince the verifier* VerEval *that* $y = f(\boldsymbol{a})$ *for the polynomial* $f \in R^{(\leq 1)}[X_1, \ldots, X_\ell]$ *committed in* cm_f.

Correctness. PC *is correct if for any multilinear* ℓ*-variate polynomial* f

$$\Pr\left[\mathsf{VerCom}(\mathsf{vk}, \mathsf{cm}_f, f, \mathsf{opn}_f) = 1 \;\middle|\; \begin{matrix}(\mathsf{ck}, \mathsf{vk}) \leftarrow \mathsf{Setup}(1^\lambda, \ell) \\ (\mathsf{cm}_f, \mathsf{opn}_f) \leftarrow \mathsf{Com}(\mathsf{ck}, f)\end{matrix}\right] = 1.$$

Binding. PC *is computationally binding if for all PPT adversaries* Adv *the following probability is* $\mathsf{negl}(\lambda)$*:*

$$\Pr\left[\begin{matrix}\mathsf{VerCom}(\mathsf{vk}, \mathsf{cm}, f, \mathsf{opn}_f) = 1 \\ \wedge \mathsf{VerCom}(\mathsf{vk}, \mathsf{cm}, f', \mathsf{opn}'_f) = 1 \\ \wedge f \neq f'\end{matrix} \;\middle|\; \begin{matrix}(\mathsf{ck}, \mathsf{vk}) \leftarrow \mathsf{Setup}(1^\lambda, \ell) \\ (\mathsf{cm}, f, \mathsf{opn}_f, f', \mathsf{opn}'_f) \leftarrow \mathsf{Adv}(\mathsf{ck})\end{matrix}\right]$$

Knowledge Soundness. *We say that* PC *is knowledge-sound if* $\langle \mathsf{ProveEval}, \mathsf{VerEval} \rangle$ *is a complete and knowledge-sound argument of knowledge for the NP relation* $\mathcal{R}_{\mathsf{Eval}} = \{(\mathbb{x} = (\mathsf{cm}_f, \boldsymbol{a}, y, \ell); \mathbb{w} = (f, \mathsf{opn}_f))\}$ *of tuples such that*

$$f \in R^{(\leq 1)}[X_1, \ldots, X_\ell] \wedge y = f(\boldsymbol{a}) \wedge \mathsf{VerCom}(\mathsf{vk}, \mathsf{cm}_f, f, \mathsf{opn}_f) = 1$$

Succinctness. *We say that* PC *is succinct if the commitments produced by* Com *have size sublinear in the size of a polynomial and if* $\langle \mathsf{ProveEval}, \mathsf{VerEval} \rangle$ *is a succinct argument.*

D.3 Compiling PIOPs into SNARKs

Given a multilinear polynomial commitment it is possible to turn a PIOP for a relation $\mathcal{R}$ into a public-coin interactive argument of knowledge for the same relation. The compilation consists in two steps: replace every oracle sent by the prover with a commitment to the underlying polynomial, and replace every verifier's query to an oracle $[\![p]\!]$ at a point $\boldsymbol{a}$ with sending the actual value $p(\boldsymbol{a})$ and proving its correctness w.r.t. the commitment of p by using the $\langle \mathsf{ProveEval}, \mathsf{VerEval} \rangle$ protocol. To this standard compilation technique we add the observation that if we start from a PIOP for an oracle relation $\mathcal{R}$ then we can easily build a (preprocessing) commit-and-prove argument. Finally, if the PIOP is public-coin so is the resulting argument system and thus it can be further compiled into a non-interactive argument using the Fiat-Shamir transform.

We summarize the compiler in the following theorem whose proof follows from previous work, e.g., [BFS20].

Theorem D.3. *Let* PC *be a multilinear polynomial commitment scheme and let* Π *be a PIOP for an oracle relation* $\mathcal{R}$ *over tuples* $(\mathbb{i}, \mathbb{x}, \mathbb{w}) = (\{[\![p_j^{\mathbb{i}}]\!]\}_j, \{[\![p_i^{\mathbb{x}}]\!]\}_i, (\{p_j^{\mathbb{i}}\}_j, \{p_i^{\mathbb{x}}\}_i))$

with negligible soundness error. There exists a public-coin interactive argument for the commit-and-prove relation

$$\begin{aligned}&\mathcal{R}' = \{(\mathbb{i}', \mathbb{x}', \mathbb{w}') = \\ &(\{\mathsf{cm}_j^{\mathbb{i}}\}_j, \{\mathsf{cm}_i^{\mathbb{x}}\}_i, (\{p_i^{\mathbb{x}}, \mathsf{opn}_i^{\mathbb{x}}\}_i\{[\![p_j^{\mathbb{i}}]\!]\}_j, \{[\![p_i^{\mathbb{x}}]\!]\}_i, (\{p_j^{\mathbb{i}}\}_j, \{p_i^{\mathbb{x}}\}_i))) \in \mathcal{R} \\ &\bigwedge_i \mathsf{VerCom}(\mathsf{vk}, \mathsf{cm}_i^{\mathbb{x}}, p_i^{\mathbb{x}}, \mathsf{opn}_i^{\mathbb{x}}) = 1\}\end{aligned}$$

where all the commitments $\mathsf{cm}_j^{\mathbb{i}} = \mathsf{Com}(\mathsf{ck}, p_j^{\mathbb{i}})$ *are honestly computed in the preprocessing phase.*

The efficiency of the resulting argument depends on the efficiency of both Π and PC as follows. For Π, let $\mathsf{T}_{\mathcal{P}}$ and $\mathsf{T}_{\mathcal{V}}$ be its prover and verifier running times, and let $\mathsf{r}, \mathsf{q}, \mathsf{o}, \mathsf{s}$ be, respectively, its round complexity, number of verifier's queries, number of prover's oracles, and total size of the oracles. For PC, let $\mathsf{T}_{\mathsf{Com}}, \mathsf{T}_{\mathsf{ProveEval}}, \mathsf{T}_{\mathsf{VerEval}}$ be the running times of the corresponding algorithms, and let $|\mathsf{cm}|$ and $|\pi_{\mathsf{PC}}|$ be the size of a commitment and a proof respectively.

- Proving time is $\mathsf{T}_{\mathcal{P}} + \mathsf{T}_{\mathsf{Com}} \cdot \mathsf{s} + \mathsf{T}_{\mathsf{ProveEval}} \cdot \mathsf{q}$
- Verifier time is $\mathsf{T}_{\mathcal{V}} + \mathsf{T}_{\mathsf{VerEval}} \cdot \mathsf{q}$
- Proof size is $|\mathsf{cm}| \cdot \mathsf{o} + |\pi_{\mathsf{PC}}| \cdot \mathsf{q}$

E Auxiliary material for PIOP for CKKS

E.1 Auxiliary material for PIOP for $\mathcal{R}_{AC}$

The following predicates are referenced in the main body:

$$\mathtt{add}(\mathbf{z}, \mathbf{x}) = \begin{cases} 1, & \text{if wire } \mathbf{x} \text{ holds the } d_0 \text{ or } d_1 \text{ value corresponding to } \mathbf{z}. \\ 0, & \text{otherwise.} \end{cases} \tag{30}$$

$$\mathtt{evk}(\mathbf{z}, \mathbf{y}) = \begin{cases} \mathsf{evk}_{l,b}[i], & \text{if wire } \mathbf{y} \text{ belongs to the } l\text{-th HE layer and} \\ & \text{holds the } i\text{-th decomposition of the } d_2 \text{ value} \\ & \text{corresponding to the } \mathbf{z}\text{-th } c'[b], \text{ where } b \in \{0,1\}. \\ 0, & \text{otherwise.} \end{cases} \tag{31}$$

$$\mathtt{Xmult}(\mathbf{z}, \mathbf{x}, \mathbf{y}) = \begin{cases} 1, & \text{if } \ V(\mathbf{x}) \cdot V(\mathbf{y}) \text{ is added to } V_2(\mathbf{z}). \\ 0, & \text{otherwise.} \end{cases} \tag{32}$$

The Π_{AC} PIOP, which appears next, uses the following equation:

$$\begin{aligned}s_{quot}\tilde{V}_{3,quot}(\mathbf{r}_{quot}) + s'_{quot}\tilde{V}_{3,quot}(\mathbf{r}'_{quot}) + s_{BD}\tilde{V}_{3,BD}(\mathbf{r}_{BD}) + s_{I_{\mathcal{P}}}\tilde{V}_{3,I_{\mathcal{P}}}(\mathbf{r}_{I_{\mathcal{P}}}) \stackrel{?}{=} \\ \tilde{V}_{aux,2^u}(\mathbf{r}_{a,2^u}) \cdot \left(s_{BD}\widetilde{\mathsf{C}}_{a,2^u,BD}(\mathbf{r}_{BD}, \mathbf{r}_{a,2^u}) + s_{I_{\mathcal{P}}}\widetilde{\mathsf{C}}_{a,2^u,I_{\mathcal{P}}}(\mathbf{r}_{I_{\mathcal{P}}}, \mathbf{r}_{a,2^u})\right)\end{aligned}$$

$$+\sum_{j=1}^{D-1}\tilde{V}_{aux,j}(\mathbf{r}_{a,j})\cdot\Big(s_{quot}\widetilde{\mathtt{C}}_{a,j,q}(\mathbf{r}_{quot},\mathbf{r}_{a,j})+s'_{quot}\widetilde{\mathtt{C}}_{a,j,q}(\mathbf{r}'_{quot},\mathbf{r}_{a,j})+s_{I_{\mathcal{P}}}\widetilde{\mathtt{C}}_{a,j,I_{\mathcal{P}}}(\mathbf{r}_{I_{\mathcal{P}}},\mathbf{r}_{a,j})\Big)$$
$$+\tilde{V}_{out}(\mathbf{r}_{out})\big(s_{quot}\widetilde{\mathtt{C}}_{o,quot}(\mathbf{r}_{quot},\mathbf{r}_{out})+s'_{quot}\widetilde{\mathtt{C}}_{o,quot}(\mathbf{r}'_{quot},\mathbf{r}_{out})+s_{I_{\mathcal{P}}}\widetilde{\mathtt{C}}_{o,I_{\mathcal{P}}}(\mathbf{r}_{I_{\mathcal{P}}},\mathbf{r}_{out})\Big). \tag{33}$$

PIOP Π_{AC} for $\mathcal{R}_{AC}$:

1. $\mathcal{V}$ sends $\mathbf{r}_0 \leftarrow\!\$\ S^{\#(V_0)}$. Run a sum-check protocol on Equation (16) evaluated at $\mathbf{r}_0$, which ends with prover claims $\tilde{V}_1(\mathbf{r}_1)$ $\tilde{V}_{2,d_2}(\mathbf{r}_2)$ and $\tilde{V}_{3,I_{\mathcal{P}}}(\mathbf{r}_{I_{\mathcal{P}}})$. $\mathcal{V}$ queries the oracles in $\mathtt{i}$ at a random $\mathbf{r}_0$ to verify whether $0 \stackrel{?}{=} \widetilde{\mathtt{rescon}}_1(\mathbf{r}_0,\mathbf{r}_1)\cdot\tilde{V}_1(\mathbf{r}_1)+\widetilde{\mathtt{rescon}}_3(\mathbf{r}_0,\mathbf{r}_{I_{\mathcal{P}}})\cdot\tilde{V}_{3,I_{\mathcal{P}}}(\mathbf{r}_{I_{\mathcal{P}}})+\widetilde{\mathtt{bdcon}}_2(\mathbf{r}_0,\mathbf{r}_2)\cdot\tilde{V}_{2,d_2}(\mathbf{r}_2)+\widetilde{\mathtt{bdcon}}_3(\mathbf{r}_0,\mathbf{r}_{I_{\mathcal{P}}})\cdot\tilde{V}_{3,I_{\mathcal{P}}}(\mathbf{r}_{I_{\mathcal{P}}})$. $\mathcal{V}$ only proceeds if equality holds.

2. Run a sum-check protocol on Equation (17) evaluated at $\mathbf{r}_1$ which ends with prover claims $\tilde{V}_{2,d_{01}}(\mathbf{r}_2)$ and $\tilde{V}_{3,BD}(\mathbf{r}_{BD})$. $\mathcal{V}$ queries $[\![\widetilde{\mathtt{add}}]\!]$ and $[\![\widetilde{\mathtt{evk}}]\!]$ to verify whether $\tilde{V}_1(\mathbf{r}_1) \stackrel{?}{=} \widetilde{\mathtt{add}}(\mathbf{r}_1,\mathbf{r}_2)\cdot\tilde{V}_{2,d_{01}}(\mathbf{r}_2)+\widetilde{\mathtt{evk}}(\mathbf{r}_1,\mathbf{r}_{BD})\cdot\tilde{V}_{3,BD}(\mathbf{r}_{BD})$. $\mathcal{V}$ only proceeds if equality holds.

3. Combine claims $\tilde{V}_{2,d_{01}}(\mathbf{r}_2)$ $\tilde{V}_{2,d_2}(\mathbf{r}_2)$ as follows:

- $\mathcal{V}$ sends $s_{01}, s_2 \leftarrow\!\$\ S$.
- $\mathcal{P}$ and $\mathcal{V}$ run a sum-check protocol on Equation (19) which ends with a prover claim $\tilde{V}_2(\mathbf{s})$. $\mathcal{V}$ queries the oracles in $\mathtt{i}$ to verify whether $s_{01}\tilde{V}_{2,d_{01}}(\mathbf{r}_{01})+s_2\tilde{V}_{2,d_2}(\mathbf{r}_2) \stackrel{?}{=} \tilde{V}_2(\mathbf{s})(s_{01}\widetilde{\mathtt{C}}_{2,d_{01}}(\mathbf{r}_{01},\mathbf{s})+s_2\widetilde{\mathtt{C}}_{2,d_2}(\mathbf{r}_2,\mathbf{s}))$. $\mathcal{V}$ only proceeds if equality holds.

4. $\mathcal{P}$ and $\mathcal{V}$ run a sum-check protocol on Equation (18) evaluated at $\mathbf{s}$, which ends with prover claims $\tilde{V}_{in}(\mathbf{r}_{in}), \tilde{V}_{in}(\mathbf{r}'_{in}), \tilde{V}_{3,quot}(\mathbf{r}_{quot})$ and $\tilde{V}_{3,quot}(\mathbf{r}'_{quot})$. $\mathcal{V}$ queries $[\![\widetilde{\mathtt{add}}]\!]$ and $[\![\widetilde{\mathtt{evk}}]\!]$ to verify whether $\tilde{V}_2(\mathbf{s}) \stackrel{?}{=} \widetilde{\mathtt{Xmult}}(\mathbf{s},\mathbf{r}_{in},\mathbf{r}'_{in})\cdot\tilde{V}_{in}(\mathbf{r}_{in})\cdot\tilde{V}_{in}(\mathbf{r}'_{in})+\widetilde{\mathtt{Xmult}}(\mathbf{s},\mathbf{r}'_{quot},\mathbf{r}_{quot})\cdot\tilde{V}_{3,quot}(\mathbf{r}'_{quot})\cdot\tilde{V}_{3,quot}(\mathbf{r}_{quot})+\widetilde{\mathtt{Xmult}}(\mathbf{s},\mathbf{r}_{in},\mathbf{r}_{quot})\cdot\tilde{V}_{in}(\mathbf{r}_{in})\cdot\tilde{V}_{3,quot}(\mathbf{r}_{quot})$. $\mathcal{V}$ only proceeds if equality holds.

5. Reduce claims about $\tilde{V}_{3,quot}, \tilde{V}_{3,BD}(\mathbf{r}_{BD})$ and $\tilde{V}_{3,I_{\mathcal{P}}}$ to claims about $[\![\tilde{V}_{out}]\!]$ and $[\![\tilde{V}_{aux}]\!]$:

- $\mathcal{V}$ sends $s_{quot}, s'_{quot}, s_{BD}, s_{I_{\mathcal{P}}} \leftarrow\!\$\ S$.
- Run a sum-check protocol on Equation (20), which ends with prover claims $\tilde{V}_{aux,2^u}(\mathbf{r}_{a,2^u}), \{\tilde{V}_{aux,j}(\mathbf{r}_{a,j})\}_{j=1}^{D-1}$ and $\tilde{V}_{out}(\mathbf{r}_{out})$. $\mathcal{V}$ queries the oracles in $\mathtt{i}$ to verify whether Equation (33) holds. $\mathcal{V}$ only proceeds if equality holds.

6. $\mathcal{V}$ queries oracles $[\![\tilde{V}_{in}]\!], [\![\tilde{V}_{out}]\!], \{[\![\tilde{V}_{aux,j}]\!]\}_{j=1}^{D-1}$ and $[\![\tilde{V}_{aux,2^u}]\!]$ to verify that $\mathcal{P}$'s claims $\tilde{V}_{in}(\mathbf{r}_{in}), \tilde{V}_{in}(\mathbf{r}'_{in}), \tilde{V}_{out}(\mathbf{r}_{out}), \{\tilde{V}_{aux,j}(\mathbf{r}_{a,j})\}_{j=1}^{D-1}$ and $\tilde{V}_{aux,2^u}(\mathbf{r}_{a,2^u})$ are true. If so $\mathcal{V}$ accepts, otherwise rejects.

Theorem E.1 (Theorem 4.2, restated). *Let $S \subseteq R_q$ be an exceptional set. Let Π_{sum} be a PIOP for $\mathcal{R}_{sum}$. Then Π_{AC} is a PIOP for $\mathcal{R}_{AC}$ with perfect completeness and knowledge soundness error $O(|\mathbf{C}|/|S|)$.*

Proof. Completeness follows from the completeness of the sum-check protocol.

Regarding soundness, assume that $\mathbf{C}(\mathbf{x},\mathbf{w}) \neq \mathbf{0}$. This means that the prover must have sent an incorrect message at some point, be it within a sum-check executions or as the claimed evaluations with which those finish. The soundness error for each of these events is as follows:

- For Equation (16), the soundness error is $2(\#(c') + \#(I_{\mathcal{P}}) + \#(d_2))/|S|$.
- For Equation (17), the soundness error is $2(\#(d_{01}) + \#(BD))/|S|$.
- For Equation (18), the soundness error is $4(\#(in) + \#(quot))/|S|$.
- For Equation (19), the soundness error is bounded by $2(1+\#(d_{01})+\#(d_2))/|S|$, where the $2/S$ term comes from the random linear combination using s_{01}, s_2 and $2(\#(d_{01}) + \#(d_2))/|S|$ is greater or equal than $2(\#(V_2))/|S|$.
- For Equation (20), the soundness error is $2(2+\#(aux, 2^u)+\sum_{j=1}^{D-1}(\#(aux, j))+ \#(out))/|S|$, where the $4/S$ term comes from the random linear combination using $s_{quot}, s'_{quot}, s_{BD}, s_{I_{\mathcal{P}}}$.

For an HE circuit of depth D and width W, let us apply the following approximate simplifications:

- $\#(d_{01}) + \#(d_2) \simeq \log(3WD)$.
- $\#(c') \simeq 1 + \log(WD)$, $\#(quot) \simeq 1 + \log(WD)$.

Applying an union bound and the above simplifications, we obtain that:

$$\frac{7 + \#(I_{\mathcal{P}}) + 2(\log(3WD)) + \#(BD) + 2\#(in) + 4\log(WD) + \#(out) + \#(aux, 2^u) + \sum_{j=1}^{D-1} \#(aux, j)}{2^{-1}|S|}$$

Which we further simplify into the asymptotic $O(|\mathbf{C}|/|S|)$. Applying Remark 2.5, the soundness error equals the knowledge soundness one. □

E.2 Auxiliary material for PIOP for $\mathcal{R}_{range}$

Proof of Theorem 4.5

Proof. By Remark 2.5, we argue the soundness error of the protocol. Consider a $([\![\tilde{p}_{\beta,N}]\!], ([\![\tilde{v}]\!], [\![\tilde{h}]\!]); (\tilde{p}_{\beta,N}, \tilde{v}, \tilde{h})) \notin \mathcal{R}_{decomp}$, then we have that

$$f(X_1, \ldots, X_\ell) = \tilde{v}(X_1, \ldots, X_\ell) - \sum_{(\boldsymbol{j},\boldsymbol{k})\in\{0,1\}^{\nu+\gamma}} \tilde{h}(X_1, \ldots, X_\ell, \boldsymbol{j}, \boldsymbol{k}) \cdot \tilde{p}_{\beta,N}(\boldsymbol{j}, \boldsymbol{k})$$

is a nonzero polynomial, and by generalized Schwartz-Zippel (Lemma 2.2) we have that $f(\boldsymbol{r}_i) = 0$ holds with probability $\leq \ell/|S|$ over the random choice of $\boldsymbol{r}_i$. On the other hand, by the soundness of the sum-check PIOP (Theorem 2.6) and the fact that the virtual oracle p^* has degree 2 we get that the probability that the verifier accepts a tuple $((y_v, [\![p^*]\!]); \tilde{p}^*) \notin \mathcal{R}_{sum}$ is at most $2(\nu + \gamma)/|S|$. □

Proof of Lemma 4.6

Proof. The proof follows as a simpler case of Claims 2, Remark 3, Claim 3 in [STW24].

For the first direction, we show how to construct the polynomials read_ts and final_cts. For every $\boldsymbol{j} \in \{0,1\}^{b/c}$ define counters $ctr_{\boldsymbol{j}}$ initialized to 0. Next, for $\boldsymbol{i} = \mathbf{0}$ to $\boldsymbol{i} = \mathbf{1}$ (e.g., for all $\boldsymbol{i} \in \{0,1\}^{\ell^*}$ in lexicographic order), if $\tilde{h}(\boldsymbol{i}) = \tilde{t}_\beta(\boldsymbol{j})$

set $\mathsf{read_ts}(\boldsymbol{i}) = ctr_{\boldsymbol{j}}$, and increase $ctr_{\boldsymbol{j}} \leftarrow ctr_{\boldsymbol{j}} + 1$. Finally set $\mathsf{final_cts}(\boldsymbol{j}) = ctr_{\boldsymbol{j}}$ for every $\boldsymbol{j}$. For such polynomials, it can be checked that $\mathsf{WS} = \mathsf{RS} \cup \mathsf{S}$.

For the reverse direction, assume by contradiction there exists $\boldsymbol{i}^*$ such that $\tilde{h}(\boldsymbol{i}^*) \notin \boldsymbol{t}_\beta$. The goal is to show that for any choice of the polynomials $\mathsf{read_ts}$, $\mathsf{final_cts}$ the sets we build from them are such that $\mathsf{WS} \neq \mathsf{RS} \cup \mathsf{S}$.

Let us partition $\mathsf{WS} = \mathsf{WS}_t \cup \mathsf{WS}_h$ where WS_t includes the pairs from the table, and WS_h the pairs from the polynomial $\tilde{h}$.

Let V be the set of all the *distinct* values $\tilde{h}(\boldsymbol{i})$. For every $v \in V$ let us define $\mathsf{RS}_v = \{(\eta, r) \in \mathsf{RS} \text{ s.t. } \eta = v\}$, and let us define analogously WS_v. The sets RS_v and WS_v form a partition of RS and WS_h respectively.

We claim that for every $v \in V$, we have $\mathsf{RS}_v \neq \mathsf{WS}_v$. To see this, assume by contradiction that $\mathsf{RS}_v = \mathsf{WS}_v$. Since the first components of all their pairs have the same value, it must hold that the sets of their second components are equal as multisets, namely $A = \{\mathsf{read_ts}(\boldsymbol{i}) \text{ s.t. } \tilde{h}(\boldsymbol{i}) = v : \boldsymbol{i}\} = \{(\mathsf{read_ts}(\boldsymbol{i})+1) \text{ s.t. } \tilde{h}(\boldsymbol{i}) = v : \boldsymbol{i}\} = B$. This equality is however not possible since $0 < |\mathsf{WS}_v| \leq |\mathsf{WS}_h| \leq 2^{\ell^*} < p$. To see this fact in more detail, if the multisets A and B are equal, there is a permutation ϕ between their elements (i.e., such that $b_k = a_{\phi(k)}$ for all k) and the sum of their elements must also be equal, i.e., $\sum_k b_k = \sum_k a_k$. However, by the existence of the permutation let $\bar{b} = \sum_k b_k = \sum_k a_{\phi(k)} = \sum_k a_k$, and by the definition of b_k let $\bar{b} = \sum_k b_k = \sum_k (a_k + 1) = (\sum_k a_k) + |B|$. This implies that $|B| = 0 \bmod p$ which is a contradiction as the cardinality of the multisets is $0 < |\mathsf{WS}_v| \leq |\mathsf{WS}_h| \leq 2^{\ell^*} < p$.

Consider the element $v^* = \tilde{h}(\boldsymbol{i}^*) \notin \boldsymbol{t}_\beta$. Since $v^* \notin t_\beta$, pairs with first coordinate v^* occur only in WS_h and RS. Hence, $\mathrm{WS} = \mathrm{RS} \cup \mathrm{S}$ would force $\mathrm{WS}_{v^*} = \mathrm{RS}_{v^*}$, but that would contradict the inequality $\mathsf{RS}_v \neq \mathsf{WS}_v$ we just showed. □

Proof of Theorem 4.8

Proof. By Remark 2.5, it is sufficient to bound the soundness error of the protocol, since the witness of $\mathcal{R}_{range}$ consists only of the polynomial $\tilde{h}$ that is provided as an oracle in the statement. Also, recall that since $\mathbb{F}$ is a field, $\mathbb{F}$ itself is an exceptional set of size p, from which all the verifier's challenges are sampled.

Consider a pair $(\boldsymbol{t}_\beta, [\![\tilde{h}]\!]) \notin \mathcal{L}(\mathcal{R}_{range})$, i.e., there exists $\boldsymbol{i}^* \in \{0,1\}^{\ell^*}$ such that $\tilde{h}(\boldsymbol{i}^*) \notin \boldsymbol{t}_\beta$, and let $[\![\mathsf{read_ts}]\!], [\![\mathsf{final_cts}]\!]$ be the (oracle) polynomials sent by the prover in the first round, i.e., before σ, τ are sent by the verifier. Let $\mathsf{WS}, \mathsf{RS}, \mathsf{S}$ be the multisets of pairs defined from $\tilde{t}_\beta, \tilde{h}, \mathsf{read_ts}, \mathsf{final_cts}$ as in Lemma 4.6, and let $\mathsf{WS}_t = \{(\tilde{t}_\beta(\boldsymbol{j}), 0) : \boldsymbol{j} \in \{0,1\}^{b/c}\}$ and $\mathsf{WS}_h = \{(\tilde{h}(\boldsymbol{i}), \mathsf{read_ts}(\boldsymbol{i}) + 1) : \boldsymbol{i} \in \{0,1\}^{\ell^*}\}$ be the partition of WS into the pairs coming from the table and those coming from $\tilde{h}$, respectively. For $\alpha \in \{\mathsf{WS}_t, \mathsf{WS}_h, \mathsf{RS}, \mathsf{S}\}$ let

$$P_\alpha(Z, Z') = \prod_{(a_0, a_1) \in \alpha} (a_0 + a_1 \cdot Z - Z') \in \mathbb{F}[Z, Z'],$$

and let

$$P(Z, Z') = P_{\mathsf{WS}_t}(Z, Z') \cdot P_{\mathsf{WS}_h}(Z, Z') - P_{\mathsf{RS}}(Z, Z') \cdot P_{\mathsf{S}}(Z, Z').$$

Notice that the coefficients of these polynomials are determined before σ, τ are sent by the verifier, and that P has total degree at most $2^{\ell^*} + 2^{b/c}$.

First round. Since $\beta = 2^{b/c} < p$, the elements of $\boldsymbol{t}_\beta = \{0, \ldots, \beta - 1\}$ are pairwise distinct in $\mathbb{F}$, and since $2^{\ell^*} < p$ we can apply Lemma 4.6 to $\tilde{h}$ and $\tilde{t}_\beta$, which gives us that $\mathsf{WS} \neq \mathsf{RS} \cup \mathsf{S}$ as multisets, *for every* choice of the polynomials $\mathsf{read_ts}, \mathsf{final_cts}$ sent by the prover. By [Set20, Lemmas 7.2–7.4], this implies that $P(Z, Z')$ is a nonzero polynomial in $\mathbb{F}[Z, Z']$, and thus by the Schwartz-Zippel lemma

$$\Pr_{\sigma,\tau \leftarrow\$ \mathbb{F}}[P(\sigma, \tau) = 0] \leq \frac{2^{\ell^*} + 2^{b/c}}{p}.$$

In the rest of the proof we assume that $P(\sigma, \tau) \neq 0$.

Second round. Consider the oracle polynomials $[\![f^{(1)}_{\mathsf{WS}_h}]\!], [\![f^{(1)}_{\mathsf{RS}}]\!], [\![f^{(1)}_{\mathsf{WS}_t}]\!], [\![f^{(1)}_{\mathsf{S}}]\!]$ sent by the prover in the second round, let $f^{(0)}_\alpha$ be defined as in Figure 1 and, for every α, let $f_\alpha(X_0, \boldsymbol{X}) = (1 - X_0) \cdot f^{(0)}_\alpha(\boldsymbol{X}) + X_0 \cdot f^{(1)}_\alpha(\boldsymbol{X})$ and let $\bar{\alpha} = f^{(1)}_\alpha(\boldsymbol{1}, 0) = f_\alpha(1, \ldots, 1, 0)$ be the values queried by $\mathcal{V}$ in the last step of the protocol. Let us call GP the event that, for every α and every $\boldsymbol{i}$ in the corresponding hypercube ($\{0,1\}^{\ell^*}$ for $\alpha = \mathsf{WS}_h, \mathsf{RS}$ and $\{0,1\}^{b/c}$ for $\alpha = \mathsf{WS}_t, \mathsf{S}$), it holds $f_\alpha(1, \boldsymbol{i}) = f_\alpha(\boldsymbol{i}, 0) \cdot f_\alpha(\boldsymbol{i}, 1)$, i.e., that all the grand-product statements are satisfied. Notice that GP is determined by the prover's messages, independently of the challenges $\chi, \boldsymbol{\rho}, \boldsymbol{\xi}$.

If GP occurs, then for every α the polynomial f_α satisfies the conditions of Lemma 4.7 with $v = f^{(0)}_\alpha$ (indeed $f_\alpha(0, \boldsymbol{i}) = f^{(0)}_\alpha(\boldsymbol{i})$ by construction), and thus $\bar{\alpha} = \prod_{\boldsymbol{i}} f^{(0)}_\alpha(\boldsymbol{i}) = P_\alpha(\sigma, \tau)$, where the last equality follows by the definition of $f^{(0)}_\alpha$; e.g., for $\alpha = \mathsf{WS}_h$ we have $\prod_{\boldsymbol{i} \in \{0,1\}^{\ell^*}} (\tilde{h}(\boldsymbol{i}) + \sigma \cdot (\mathsf{read_ts}(\boldsymbol{i}) + 1) - \tau) = P_{\mathsf{WS}_h}(\sigma, \tau)$. Hence the final check of $\mathcal{V}$, namely $\overline{\mathsf{WS}}_t \cdot \overline{\mathsf{WS}}_h = \overline{\mathsf{RS}} \cdot \overline{\mathsf{S}}$, is equivalent to $P(\sigma, \tau) = 0$, and thus $\mathcal{V}$ rejects.

If GP does not occur, there exist a multiset α^* and a point $\boldsymbol{i}^*$ of the corresponding hypercube for which the grand-product statement fails. We analyze the case $\alpha^* \in \{\mathsf{WS}_h, \mathsf{RS}\}$; the case $\alpha^* \in \{\mathsf{WS}_t, \mathsf{S}\}$ is symmetric, replacing $\boldsymbol{\rho}, \ell^*$ and h^* with $\boldsymbol{\xi}, b/c$ and t^*, respectively. For $\boldsymbol{i} \in \{0,1\}^{\ell^*}$ let

$$A(\boldsymbol{i}) = f_{\mathsf{WS}_h}(1, \boldsymbol{i}) - f_{\mathsf{WS}_h}(\boldsymbol{i}, 0) \cdot f_{\mathsf{WS}_h}(\boldsymbol{i}, 1), \quad B(\boldsymbol{i}) = f_{\mathsf{RS}}(1, \boldsymbol{i}) - f_{\mathsf{RS}}(\boldsymbol{i}, 0) \cdot f_{\mathsf{RS}}(\boldsymbol{i}, 1),$$

and let $\tilde{G}$ be the multilinear extension of the function $\boldsymbol{i} \mapsto A(\boldsymbol{i}) + \chi \cdot B(\boldsymbol{i})$. By the definition of g in Figure 1 and since $\sum_{\boldsymbol{i} \in \{0,1\}^{\ell^*}} u(\boldsymbol{i}) \cdot \tilde{eq}(\boldsymbol{\rho}, \boldsymbol{i}) = \tilde{u}(\boldsymbol{\rho})$ for the MLE $\tilde{u}$ of any function $u : \{0,1\}^{\ell^*} \to \mathbb{F}$, we have $\sum_{\boldsymbol{i} \in \{0,1\}^{\ell^*}} h^*(\boldsymbol{i}) = \tilde{G}(\boldsymbol{\rho})$. Since $A(\boldsymbol{i}^*) \neq 0$ or $B(\boldsymbol{i}^*) \neq 0$, the polynomial $A(\boldsymbol{i}^*) + X \cdot B(\boldsymbol{i}^*)$ is a nonzero polynomial of degree at most 1, hence $A(\boldsymbol{i}^*) + \chi \cdot B(\boldsymbol{i}^*) = 0$ with probability at most $1/p$ over the choice of $\chi \leftarrow\$ \mathbb{F}$. If $A(\boldsymbol{i}^*) + \chi \cdot B(\boldsymbol{i}^*) \neq 0$, then $\tilde{G}$ is a nonzero multilinear polynomial in ℓ^* variables, and by the Schwartz-Zippel lemma $\tilde{G}(\boldsymbol{\rho}) = 0$ with probability at most ℓ^*/p over the choice of $\boldsymbol{\rho} \leftarrow\$ \mathbb{F}^{\ell^*}$. Finally, if $\tilde{G}(\boldsymbol{\rho}) \neq 0$ then $((0, [\![h^*]\!]); h^*) \notin \mathcal{R}_{sum}$ and, since the virtual oracle h^* has degree 3 in each of its ℓ^* variables, by Theorem 2.6 the verifier of the sum-check PIOP accepts with

probability at most $3\ell^*/p$. Therefore, when GP does not occur, $\mathcal{V}$ accepts with probability at most $(1+\ell^*+3\ell^*)/p$ in the case $\alpha^* \in \{\mathsf{WS}_h, \mathsf{RS}\}$, and at most $(1+b/c+3b/c)/p$ in the symmetric case.

Summing up the bounds above, the soundness error of $\Pi^{\mathbb{F},\beta}_{range}$ is at most

$$\frac{2^{\ell^*}+2^{b/c}}{p} + \frac{4\max(\ell^*, b/c)+1}{p} = \delta^{\mathbb{F}}_{range}.$$

□

Efficiency of $\Pi^{\mathbb{F},\beta}_{range}$ Since $\Pi^{\mathbb{F},\beta}_{range}$ runs entirely over the field $\mathbb{F} = \mathbb{Z}_p$, all the sizes and running times below are counted in elements and operations of $\mathbb{F}$. The PIOP has the following efficiency profile:

- the number of rounds is $\mathsf{r} = \max(\ell^*, b/c) + 2$ as the protocol involves a parallel execution of two sum-checks of size 2^{ℓ^*} and $2^{b/c}$, plus the first two rounds;
- the prover sends $\mathsf{o} = 6$ oracles for a total size $\mathsf{s} = 3(2^{\ell^*} + 2^{b/c})$;
- the proving time is $\mathsf{T}_{\mathcal{P}} = O(2^{\ell^*} + 2^{b/c})$ since this is the time needed to build the 6 oracle polynomials and since the two sum-checks can be run in linear time as they involve a product of multilinear polynomials;
- the verifier time is $\mathsf{T}_{\mathcal{V}} = O(\ell^* + b/c)$ since this is verification time of the two sum-check executions. Additionally, as we show below $\mathcal{V}$ has to compute the evaluation of $\tilde{t}_\beta$ on $(\boldsymbol{r}'_t, 0), (\boldsymbol{r}'_t, 1)$, which can be done in time $O(b/c)$ since $\tilde{t}_\beta(X_1, \ldots, X_{b/c}) = \sum_{k=1}^{b/c} X_k \cdot 2^{k-1}$.
- the number of verifier's queries is $\mathsf{q} = 22$, as justified below.

Assume that the sum-check on h^* (resp. t^*) ends with a query of $\mathcal{V}$ of the virtual oracle on $\boldsymbol{r}_h$ (resp. $\boldsymbol{r}_t$). This translates into the following 22 queries to the 7 oracles below:

- $[\![\tilde{h}]\!]$ on $(\boldsymbol{r}'_h, 0), (\boldsymbol{r}'_h, 1)$;
- $[\![\mathsf{read_ts}]\!]$ on $(\boldsymbol{r}'_h, 0), (\boldsymbol{r}'_h, 1)$;
- $[\![\mathsf{final_cts}]\!]$ on $(\boldsymbol{r}'_t, 0), (\boldsymbol{r}'_t, 1)$;
- $[\![f^{(1)}_{\mathsf{WS}_h}]\!]$ on $\boldsymbol{r}_h, (\boldsymbol{r}'_h, 0), (\boldsymbol{r}'_h, 1), (\mathbf{1}, 0)$
- $[\![f^{(1)}_{\mathsf{RS}}]\!]$ on $\boldsymbol{r}_h, (\boldsymbol{r}'_h, 0), (\boldsymbol{r}'_h, 1), (\mathbf{1}, 0)$
- $[\![f^{(1)}_{\mathsf{WS}_t}]\!]$ on $\boldsymbol{r}_t, (\boldsymbol{r}'_t, 0), (\boldsymbol{r}'_t, 1), (\mathbf{1}, 0)$
- $[\![f^{(1)}_{\mathsf{S}}]\!]$ on $\boldsymbol{r}_t, (\boldsymbol{r}'_t, 0), (\boldsymbol{r}'_t, 1), (\mathbf{1}, 0)$

The queries above follow by the definition of g as shown below.

Let $p^* \leftarrow g_{\sigma,\tau,\chi,\boldsymbol{\rho}}(p_1, p_2, f^{(1)}_1, f^{(1)}_2, \boldsymbol{\rho})$. Then, following g's construction, an evaluation of p^* on $\boldsymbol{r}$ can be computed as

$$\begin{aligned} p^*(\boldsymbol{r}) &= (f_1(1,\boldsymbol{r}) - f_1(\boldsymbol{r},0)\cdot f_1(\boldsymbol{r},1) + \chi\cdot(f_2(1,\boldsymbol{r}) - f_2(\boldsymbol{r},0)\cdot f_2(\boldsymbol{r},1)))\cdot \tilde{eq}(\boldsymbol{\rho},\boldsymbol{r}) \\ &= (f^{(1)}_1(\boldsymbol{r}) - f_1(\boldsymbol{r},0)\cdot f_1(\boldsymbol{r},1) + \chi\cdot(f^{(1)}_2(\boldsymbol{r}) - f_2(\boldsymbol{r},0)\cdot f_2(\boldsymbol{r},1)))\cdot \tilde{eq}(\boldsymbol{\rho},\boldsymbol{r}) \end{aligned}$$

$$
\begin{aligned}
&= \Big(f_1^{(1)}(\boldsymbol{r}) - \Big((1-r_1)\cdot f_1^{(0)}(\boldsymbol{r}',0) + r_1 \cdot f_1^{(1)}(\boldsymbol{r}',0)\Big) \\
&\qquad \cdot \Big((1-r_1)\cdot f_1^{(0)}(\boldsymbol{r}',1) + r_1 \cdot f_1^{(1)}(\boldsymbol{r}',1)\Big) \\
&\qquad +\chi \cdot \Big(f_2^{(1)}(\boldsymbol{r}) - ((1-r_1)\cdot f_2^{(0)}(\boldsymbol{r}',0) + r_1 \cdot f_2^{(1)}(\boldsymbol{r}',0)) \\
&\qquad\quad \cdot((1-r_1)\cdot f_2^{(0)}(\boldsymbol{r}',1) + r_1 \cdot f_2^{(1)}(\boldsymbol{r}',1))\Big)\Big) \cdot \tilde{eq}(\boldsymbol{\rho},\boldsymbol{r}) \\
&= \Big(f_1^{(1)}(\boldsymbol{r}) - \Big((1-r_1)\cdot (p_1(\boldsymbol{r}',0) + \sigma(p_2(\boldsymbol{r}',0)+1) - \tau) + r_1 \cdot f_1^{(1)}(\boldsymbol{r}',0)\Big) \\
&\qquad \cdot \Big((1-r_1)\cdot (p_1(\boldsymbol{r}',1) + \sigma(p_2(\boldsymbol{r}',1)+1) - \tau) + r_1 \cdot f_1^{(1)}(\boldsymbol{r}',1)\Big) \\
&\qquad +\chi \cdot \Big(f_2^{(1)}(\boldsymbol{r}) - ((1-r_1)\cdot (p_1(\boldsymbol{r}',0) + \sigma p_2(\boldsymbol{r}',0) - \tau) + r_1 \cdot f_2^{(1)}(\boldsymbol{r}',0)) \\
&\qquad\quad \cdot((1-r_1)\cdot (p_1(\boldsymbol{r}',1) + \sigma p_2(\boldsymbol{r}',1) - \tau) + r_1 \cdot f_2^{(1)}(\boldsymbol{r}',1))\Big)\Big) \cdot \tilde{eq}(\boldsymbol{\rho},\boldsymbol{r})
\end{aligned}
$$

i.e., it needs the following 10 evaluations of the input polynomials

$$
p_1(\boldsymbol{r}',0), p_1(\boldsymbol{r}',1), p_2(\boldsymbol{r}',0), p_2(\boldsymbol{r}',1),
$$

$$
f_1^{(1)}(\boldsymbol{r}), f_1^{(1)}(\boldsymbol{r}',0), f_1^{(1)}(\boldsymbol{r}',1), f_2^{(1)}(\boldsymbol{r}), f_2^{(1)}(\boldsymbol{r}',0), f_2^{(1)}(\boldsymbol{r}',1)
$$

Proof of Theorem 4.11

Proof. **Correctness.** Let $\tilde{h}$ be such that $\tilde{h}(\boldsymbol{b}) \in \boldsymbol{t}_\beta$ for all $\boldsymbol{b} \in \{0,1\}^{\ell^*}$. Then the polynomial $\tilde{h}'' \in \mathbb{Z}^{(\leq 1)}[\boldsymbol{X}]$ computed by $\mathcal{P}$ in step 1 exists, and for $\tilde{h}' = \pi_p(\tilde{h}'')$ it holds $\tilde{h}'(\boldsymbol{b}) = \pi_p(\tilde{h}''(\boldsymbol{b})) \in \pi_p(\boldsymbol{t}_\beta) = \boldsymbol{t}'_\beta$ for all $\boldsymbol{b} \in \{0,1\}^{\ell^*}$. Therefore $(\boldsymbol{t}'_\beta, [\![\tilde{h}']\!]; \tilde{h}') \in \mathcal{R}_{range}$ and, by its correctness, the protocol $\Pi^{\mathbb{F},\beta}_{range}$ of step 3 accepts.

Next, we show that the checks of step 2 are satisfied. The checks $\tilde{y}_\eta(\hat{\boldsymbol{s}}) = \tilde{h}(\hat{\boldsymbol{r}}_\eta, \hat{\boldsymbol{s}})$ and $\tilde{y}'_\eta(\boldsymbol{s}') = \tilde{h}'(\boldsymbol{r}'_\eta, \boldsymbol{s}')$ hold by construction of $\tilde{y}_\eta$ and $\tilde{y}'_\eta$. Let us now focus on the last check. By construction, $\tilde{y}_\eta(\boldsymbol{\delta}) = \tilde{h}(\hat{\boldsymbol{r}}_\eta, \boldsymbol{\delta})$. The multilinear polynomials $\tilde{h}$ and $\pi_q(\tilde{h}'')$ agree on all the points of $\{0,1\}^{\ell^*}$, hence they are the same polynomial in $R_q[\boldsymbol{X}]$; since π_q is a ring homomorphism and $\hat{\boldsymbol{r}}_\eta = \pi_q(\boldsymbol{r}_\eta)$, we obtain

$$
\tilde{y}_\eta(\boldsymbol{\delta}) = \pi_q(\tilde{h}'')(\pi_q(\boldsymbol{r}_\eta), \boldsymbol{\delta}) = \pi_q\left(\tilde{h}''(\boldsymbol{r}_\eta, \boldsymbol{\delta})\right).
$$

On the other hand, by construction $\tilde{y}'_\eta(\boldsymbol{\delta}) = \tilde{h}'(\boldsymbol{r}'_\eta, \boldsymbol{\delta}) = \pi_p(\tilde{h}'')(\pi_p(\boldsymbol{r}_\eta, \boldsymbol{\delta}))$, and by Lemma 4.10 $\left|\tilde{h}''(\boldsymbol{r}_\eta, \boldsymbol{\delta})\right| < p/2$; thus, by Proposition 4.9, $\iota(\tilde{y}'_\eta(\boldsymbol{\delta})) = \tilde{h}''(\boldsymbol{r}_\eta, \boldsymbol{\delta})$. Therefore, for every $l \in \{0, \ldots, L\}$ we have

$$
\iota(\tilde{y}'_\eta(\boldsymbol{\delta})) \bmod p_l = \tilde{h}''(\boldsymbol{r}_\eta, \boldsymbol{\delta}) \bmod p_l = \pi_q\left(\tilde{h}''(\boldsymbol{r}_\eta, \boldsymbol{\delta})\right) \bmod p_l = \tilde{y}_\eta(\boldsymbol{\delta}) \bmod p_l,
$$

where the second equality holds since p_l divides q, i.e., the last check is satisfied.

Security. By Remark 2.5, it is sufficient to bound the soundness error of the protocol, since the witness of $\mathcal{R}_{range}$ consists only of the polynomial $\tilde{h}$ that is

provided as an oracle in the statement. Also, recall that δ-knowledge soundness implies δ-soundness (for a statement not in the language, no witness returned by the extractor can satisfy the relation), which is the property of $\Pi^{\mathbb{F},\beta}_{range}$ that we use.

Consider a pair $(\boldsymbol{t}_\beta, [\![\tilde{h}]\!]) \notin \mathcal{L}(\mathcal{R}_{range})$, i.e., there exist $\boldsymbol{\phi}^* \in \{0,1\}^m$ and $\boldsymbol{\delta}^* \in \{0,1\}^{\ell^*-m}$ such that $\tilde{h}(\boldsymbol{\phi}^*, \boldsymbol{\delta}^*) \notin \pi_q(\boldsymbol{t}_\beta)$, and let $[\![\tilde{h}']\!]$ be the oracle sent by the prover in step 1, where $\tilde{h}' \in \mathbb{Z}_p^{(\leq 1)}[\boldsymbol{X}]$. We distinguish two cases.

If $(\boldsymbol{t}'_\beta, [\![\tilde{h}']\!]) \notin \mathcal{L}(\mathcal{R}_{range})$, then by the soundness of $\Pi^{\mathbb{F},\beta}_{range}$ the verifier accepts in step 3 with probability at most $\delta^{\mathbb{F}}_{range}$.

Otherwise, $\tilde{h}'(\boldsymbol{b}) \in \boldsymbol{t}'_\beta$ for all $\boldsymbol{b} \in \{0,1\}^{\ell^*}$. Let $\tilde{h}'' \in \mathbb{Z}^{(\leq 1)}[\boldsymbol{X}]$ be the multilinear polynomial such that $\tilde{h}''(\boldsymbol{b}) = \iota(\tilde{h}'(\boldsymbol{b}))$ for all $\boldsymbol{b} \in \{0,1\}^{\ell^*}$ (when $\mathcal{P}$ is honest, this is the polynomial computed in step 1). Since $\boldsymbol{t}'_\beta = \pi_p(\boldsymbol{t}_\beta)$ and $\beta - 1 < p/2$ by equation (27), we have $\iota(\boldsymbol{t}'_\beta) = \boldsymbol{t}_\beta$, and thus $\tilde{h}''(\boldsymbol{b}) \in \boldsymbol{t}_\beta$ for all $\boldsymbol{b} \in \{0,1\}^{\ell^*}$; moreover, $\pi_p(\tilde{h}'') = \tilde{h}'$ since they are multilinear polynomials that agree on $\{0,1\}^{\ell^*}$. Consider the multilinear polynomial $f \in R_q[X_1, \ldots, X_m]$ defined as

$$f(X_1, \ldots, X_m) = \tilde{h}(X_1, \ldots, X_m, \boldsymbol{\delta}^*) - \pi_q(\tilde{h}'')(X_1, \ldots, X_m, \boldsymbol{\delta}^*),$$

which is determined by $\tilde{h}$ and $\tilde{h}'$, i.e., before the verifier samples $\boldsymbol{r}_1, \ldots, \boldsymbol{r}_\kappa$. We have $f(\boldsymbol{\phi}^*) \neq 0$, since $\tilde{h}(\boldsymbol{\phi}^*, \boldsymbol{\delta}^*) \notin \pi_q(\boldsymbol{t}_\beta)$ while $\pi_q(\tilde{h}''(\boldsymbol{\phi}^*, \boldsymbol{\delta}^*)) \in \pi_q(\boldsymbol{t}_\beta)$. Hence f is a nonzero polynomial of total degree at most m and, since $\pi_q(\Theta)$ is an exceptional set of size θ, by Lemma 2.2 for every $\eta \in [\kappa]$ we have $\Pr_{\boldsymbol{r}_\eta \leftarrow\$ \Theta^m}[f(\hat{\boldsymbol{r}}_\eta) = 0] \leq m/\theta$, where $\hat{\boldsymbol{r}}_\eta = \pi_q(\boldsymbol{r}_\eta)$. As the κ challenges are sampled independently,

$$\Pr_{\boldsymbol{r}_1, \ldots, \boldsymbol{r}_\kappa \leftarrow\$ \Theta^m}[\forall \eta \in [\kappa] : f(\hat{\boldsymbol{r}}_\eta) = 0] \leq \left(\frac{m}{\theta}\right)^\kappa.$$

Let us now assume that there exists $\bar{\eta} \in [\kappa]$ such that $f(\hat{\boldsymbol{r}}_{\bar{\eta}}) \neq 0$, and consider the polynomials $(\tilde{y}_1, \tilde{y}'_1, \ldots, \tilde{y}_\kappa, \tilde{y}'_\kappa)$ sent by the prover in step 2. Let $\boldsymbol{X}_{>m}$ denote $(X_{m+1}, \ldots, X_{\ell *})$. If $\tilde{y}_{\bar{\eta}}(\boldsymbol{X}_{>m}) \neq \tilde{h}(\hat{\boldsymbol{r}}_{\bar{\eta}}, \boldsymbol{X}_{>m})$ as polynomials in $R_q[\boldsymbol{X}_{>m}]$, then their difference is a nonzero polynomial of total degree at most $\ell^* - m$, and by Lemma 2.2 the check $\tilde{y}_{\bar{\eta}}(\hat{\boldsymbol{s}}) = \tilde{h}(\hat{\boldsymbol{r}}_{\bar{\eta}}, \hat{\boldsymbol{s}})$ passes with probability at most $(\ell^* - m)/|S|$ over the choice of $\hat{\boldsymbol{s}} \leftarrow\$ S^{\ell^*-m}$. Similarly, if $\tilde{y}'_{\bar{\eta}}(\boldsymbol{X}_{>m}) \neq \tilde{h}'(\boldsymbol{r}'_{\bar{\eta}}, \boldsymbol{X}_{>m})$ as polynomials in $\mathbb{Z}_p[\boldsymbol{X}_{>m}]$, the check $\tilde{y}'_{\bar{\eta}}(\boldsymbol{s}') = \tilde{h}'(\boldsymbol{r}'_{\bar{\eta}}, \boldsymbol{s}')$ passes with probability at most $(\ell^* - m)/p$ over the choice of $\boldsymbol{s}' \leftarrow\$ \mathbb{Z}_p^{\ell^*-m}$. Finally, if both $\tilde{y}_{\bar{\eta}}$ and $\tilde{y}'_{\bar{\eta}}$ are the correct partial evaluations, then $\tilde{y}_{\bar{\eta}}(\boldsymbol{\delta}^*) = \tilde{h}(\hat{\boldsymbol{r}}_{\bar{\eta}}, \boldsymbol{\delta}^*)$ and $\tilde{y}'_{\bar{\eta}}(\boldsymbol{\delta}^*) = \tilde{h}'(\boldsymbol{r}'_{\bar{\eta}}, \boldsymbol{\delta}^*) = \pi_p(\tilde{h}'')(\pi_p(\boldsymbol{r}_{\bar{\eta}}, \boldsymbol{\delta}^*))$. By Lemma 4.10 we have $\left|\tilde{h}''(\boldsymbol{r}_{\bar{\eta}}, \boldsymbol{\delta}^*)\right| < p/2$, and thus by Proposition 4.9 $\iota(\tilde{y}'_{\bar{\eta}}(\boldsymbol{\delta}^*)) = \tilde{h}''(\boldsymbol{r}_{\bar{\eta}}, \boldsymbol{\delta}^*)$. Therefore,

$$\tilde{y}_{\bar{\eta}}(\boldsymbol{\delta}^*) - \pi_q\left(\iota(\tilde{y}'_{\bar{\eta}}(\boldsymbol{\delta}^*))\right) = \tilde{h}(\hat{\boldsymbol{r}}_{\bar{\eta}}, \boldsymbol{\delta}^*) - \pi_q(\tilde{h}'')(\hat{\boldsymbol{r}}_{\bar{\eta}}, \boldsymbol{\delta}^*) = f(\hat{\boldsymbol{r}}_{\bar{\eta}}) \neq 0,$$

where we used that π_q is a ring homomorphism. By the CRT, there exists $\bar{l} \in \{0, \ldots, L\}$ such that $\tilde{y}_{\bar{\eta}}(\boldsymbol{\delta}^*) \bmod p_{\bar{l}} \neq \pi_q\left(\iota(\tilde{y}'_{\bar{\eta}}(\boldsymbol{\delta}^*))\right) \bmod p_{\bar{l}} = \iota(\tilde{y}'_{\bar{\eta}}(\boldsymbol{\delta}^*)) \bmod p_{\bar{l}}$, namely the last check of step 2 fails for $(\bar{\eta}, \boldsymbol{\delta}^*, \bar{l})$ and $\mathcal{V}$ rejects.

By a union bound over the events analyzed above, the verifier accepts with probability at most

$$\delta^{\mathbb{F}}_{range} + \left(\frac{m}{\theta}\right)^{\kappa} + \frac{\ell^* - m}{|S|} + \frac{\ell^* - m}{p},$$

which concludes the proof. □

F Formal description of commitment schemes in Section 5

F.1 Polynomial commitment for $R_q[X]$

Let $\mathsf{PC}^{(i)} = (\mathsf{Setup}^{(i)}, \mathsf{Com}^{(i)}, \mathsf{VerCom}^{(i)}, \mathsf{ProveEval}^{(i)}, \mathsf{VerEval}^{(i)})$ be a multilinear polynomial commitment for polynomials in $\mathbb{F}^{(i)}[\boldsymbol{X}]$, for $i \in [\mu]$. We construct $\mathsf{PC} = (\mathsf{Setup}, \mathsf{Com}, \mathsf{VerCom}, \mathsf{ProveEval}, \mathsf{VerEval})$ for $R_q[\boldsymbol{X}]$ as follows:

$\mathsf{Setup}(1^\lambda, \ell) \to (\mathsf{ck}, \mathsf{vk})$: run $\mathsf{Setup}^{(i)} \to (\mathsf{ck}^{(i)}, \mathsf{vk}^{(i)})$ for $i \in [\mu]$ and output $\mathsf{ck} = (\mathsf{ck}^{(i)})_{i=1}^{\mu}, \mathsf{vk} = (\mathsf{vk}^{(i)})_{i=1}^{\mu}$.

$\mathsf{Com}(\mathsf{ck}, f) \to (\mathsf{cm}_f, \mathsf{opn}_f)$: compute $\mathsf{Com}^{(i)}(\mathsf{ck}^{(i)}, \Phi^{(i)}(f)) \to (\mathsf{cm}^{(i)}, \mathsf{opn}^{(i)})$ and output $\mathsf{cm}_f := (\mathsf{cm}^{(i)})_{i=1}^{\mu}$, $\mathsf{opn}_f := (\mathsf{opn}^{(i)})_{i=1}^{\mu}$.

$\mathsf{VerCom}(\mathsf{vk}, \mathsf{cm}_f, f, \mathsf{opn}_f) \to b$:
output $b \leftarrow \bigwedge_{i=1}^{\mu} \mathsf{VerCom}^{(i)}(\mathsf{vk}^{(i)}, \mathsf{cm}^{(i)}, \Phi^{(i)}(f), \mathsf{opn}^{(i)})$.

$\langle \mathsf{ProveEval}(\mathsf{ck}, \mathsf{cm}_f, \boldsymbol{a}, y, \ell; f, \mathsf{opn}_f), \mathsf{VerEval}(\mathsf{vk}, \mathsf{cm}_f, \boldsymbol{a}, y, \ell) \rangle \to b$:
let $f^{(i)} = \Phi^{(i)}(f)$, $\boldsymbol{a}^{(i)} = \Phi^{(i)}(\boldsymbol{a})$ and $y^{(i)} = \Phi^{(i)}(y)$; prover and verifier run $b_i \leftarrow \langle \mathsf{ProveEval}^{(i)}, \mathsf{VerEval}^{(i)} \rangle$ with inputs $(\mathsf{ck}^{(i)}, \mathsf{cm}^{(i)}, \boldsymbol{a}^{(i)}, y^{(i)}, \ell; f^{(i)}, \mathsf{opn}^{(i)})$ and $(\mathsf{vk}^{(i)}, \mathsf{cm}^{(i)}, \boldsymbol{a}^{(i)}, y^{(i)}, \ell)$ and the verifier outputs $b \leftarrow \bigwedge_{i=1}^{\mu} b_i$.

F.2 Brakedown Polynomial commitment

Below $MT.\mathsf{Com}$, $MT.\mathsf{opn}$ specifies a Merkle tree commitment to matrices in $\mathbb{F}^{m \times M}$ seen as vectors in $(\mathbb{F}^m)^M$, i.e. each of the M leaves is (the hash of) a vector in $\mathbb{F}^m$.

$\mathsf{Setup}(1^\lambda, \ell) \to (\mathsf{ck}, \mathsf{vk})$: Outputs Enc and parameter t.

$\mathsf{Com}(\mathsf{ck}, f) \to (\mathsf{cm}_f, \mathsf{opn}_f)$: does the following
- Represents f as the matrix $U_f \in \mathbb{F}^{m \times m}$ as explained in Section 5.
- Computes the matrix $\hat{U}_f \in \mathbb{F}^{m \times M}$, whose i-th row is the encoding with Enc of the i-th row of U_f.
- Compute $(\mathsf{cm}_f, \mathsf{opn}_f) = MT.\mathsf{Com}(\hat{U}_f)$.

$\mathsf{VerCom}(\mathsf{vk}, \mathsf{cm}_f, f, \mathsf{opn}_f)$: compute U_f and $\hat{U}_f$ as in Com. Output 1 iff $(\mathsf{cm}_f, \mathsf{opn}_f)$ is the Merkle commitment of $\hat{U}_f$, 0 otherwise.

$\langle \mathsf{ProveEval}(\mathsf{ck}, \mathsf{cm}_f, \boldsymbol{a}, y, \ell; f, \mathsf{opn}_f), \mathsf{VerEval}(\mathsf{vk}, \mathsf{cm}_f, \boldsymbol{a}, y, \ell) \rangle \to b$: We separate this proof in two phases, the testing phase and the evaluation phase.

Testing phase:

$\mathcal{V} \to \mathcal{P}$: $\mathcal{V}$ samples $\boldsymbol{r} \in \mathbb{F}^m$ and sends it to $\mathcal{P}$.
$\mathcal{P} \to \mathcal{V}$: computes and sends $\boldsymbol{v} = \boldsymbol{r} \cdot U_f$.
$\mathcal{V} \to \mathcal{P}$: $\mathcal{V}$ uniformly samples and sends Q a subset of $[M]$ of size $\mathfrak{t}$.
$\mathcal{P} \to \mathcal{V}$: for $i \in [m], j \in Q$, sends $(MT.\mathsf{opn}_j)_{j \in Q}$, the opening of columns of $\hat{U}_f$ indexed by $j \in Q$. We call this submatrix $\hat{U}_f^Q$.

$\mathcal{V}$:

- verifies the correctness of the Merkle commitment openings.
- from $\boldsymbol{v}$, computes $\hat{\boldsymbol{v}}_Q$ the subvector of $\hat{\boldsymbol{v}} = \mathsf{Enc}(\boldsymbol{v})$ of coordinates indexed by Q.
- verifies that $\hat{\boldsymbol{v}}_Q =^? \boldsymbol{r} \cdot \hat{U}_f^Q$. If any checks fail, output $b = 0$ and stop. Otherwise continue to evaluation phase.

Evaluation phase: let $\boldsymbol{q}_1, \boldsymbol{q}_2 \in \mathbb{F}^m$ be such that $f(\boldsymbol{a}) = \langle \boldsymbol{q}_1 \cdot U_f, \boldsymbol{q}_2 \rangle$
$\mathcal{P} \to \mathcal{V}$: computes and sends $\boldsymbol{v'} = \boldsymbol{q}_1 \cdot U_f$.
$\mathcal{V} \to \mathcal{P}$: $\mathcal{V}$ uniformly samples and sends Q' a subset of $[M]$ of size $\mathfrak{t}$.
$\mathcal{P} \to \mathcal{V}$: for $i \in [m], j \in Q'$, sends $(MT.\mathsf{opn}_j)_{j \in Q'}$, the opening of columns of $\hat{U}_f$ indexed by $j \in Q'$. We call this submatrix $\hat{U}_f^{Q'}$.

$\mathcal{V}$:

- verifies the correctness of the Merkle commitment openings.
- from $\boldsymbol{v'}$, computes $\hat{\boldsymbol{v}}'_{Q'}$ the subvector of $\hat{\boldsymbol{v}}' = \mathsf{Enc}(\boldsymbol{v'})$ of coordinates indexed by Q'.
- verifies that $\hat{\boldsymbol{v}}'_{Q'} =^? \boldsymbol{q}_1 \cdot \hat{U}_f^{Q'}$ and $y =^? \langle \boldsymbol{v'}, \boldsymbol{q}_2 \rangle$. If any checks fail, output $b = 0$. Otherwise output $b = 1$.

If the evaluation phase is run several times for different $\boldsymbol{a}$, the testing phase only needs to be run once. On the other hand, if the evaluation phase is only run once, Q and Q' can be the same, and therefore the prover only needs to open the Merkle commitment at one set of columns.

F.3 Choice of Enc

As described in Section 5 using Reed-Solomon codes of length up to $N/2$ is beneficial because in that case we can define the set of evaluation points to be contained in the set of $N/2$-th roots of unity contained in the base field $\mathbb{F}_p$ and use the NTT in good conditions. If $m \geq N/2$, we can unfortunately not define a Reed-Solomon code with length $N/2$, dimension m and nontrivial minimum distance. In this case we define the encoding

$$\begin{aligned} \mathsf{Enc} : \mathbb{F}^m &\longrightarrow (\mathbb{F}^{m/\tau})^\tau &&\longrightarrow (\mathbb{F}^{N/2})^\tau \\ \boldsymbol{x} &\longrightarrow (\boldsymbol{x}_1, \ldots, \boldsymbol{x}_\tau) &&\longrightarrow (\mathsf{Enc}'(\boldsymbol{x}_1), \ldots, \mathsf{Enc}'(\boldsymbol{x}_\tau)) \end{aligned}$$

where the first map simply splits $\boldsymbol{x}$ in τ blocks of length m/τ (for simplicity we always take τ dividing m) and the second map encodes each block with a $[N/2, m/\tau]$-Reed Solomon code with encoding function Enc'. Call ρ the rate of this Reed Solomon code. Then $m/\tau = \rho N/2$ and its minimum distance is $(1-\rho)N/2 + 1 > (1-\rho)N/2$.

In these conditions, Enc defines a code of length $\tau N/2 = m/\rho$, dimension m (hence rate also ρ) and relative minimum distance $\gamma = \frac{(1-\rho)N/2}{m/\rho} = \frac{\rho(1-\rho)N}{2m}$. For fixed N and m, γ is maximized for $\rho = 1/2$.

In summary, in order to maximize the minimum distance of Enc, we choose $\rho = 1/2$ and define Enc' to be a $[N/2, N/4, N/4+1]_{p^4}$-RS code whose evaluation points are $N/2$-th roots of unity in $\mathbb{F}_p$, leading to $\gamma = \frac{N}{8m}$.

G Complete results

		2^{16}	2^{18}	2^{20}	2^{22}	2^{24}	2^{26}
Single Commit	Expander code	3.2ms	6.3ms	11.7ms	46.7ms	166.3ms	615.6ms
	RS code	48.7μs	89.0μs	313.0μs	1.32ms	4.8ms	21.5ms
	RS code (ST)	349μs	1.9ms	5.5ms	31.5ms	151.9ms	635.7ms
$(2^{13}, 3)$-R_{q_0} commit	RS code	0.3s	0.5s	1.9s	8.1s	29.3s	2m11s
	Expander code	1m19s	2m36s	4m47s	19m7s	1h8m	4h12m
$(2^{14}, 6)$-R_{q_0} commit	RS code	1.2s	2.2s	7.7s	32.4s	1m57s	8m47s
	Expander code	5m17s	10m24s	19m10s	1h16m	4h32m	16h48m

Table 4: Brakedown using Expander and RS codes. **(ST)** denotes single-threaded execution, all other numbers are for a multithreaded execution with up to 48 threads.